AP HOGESCHOOL ANTWERPEN

Web Technology

Syllabus 2026-2027

Kristof Michiels
AP Hogeschool Antwerpen
versie van 25 oktober 2026

Inleiding

Dit is de gloednieuwe syllabus voor het vak Web Technology. Ze vergezelt de slides van elk hoorcollege en de labo's waarin je elke week de leerstof inoefent.

In zo'n tekst kruipt veel werk. Telkens een nieuw hoofdstuk klaar is, wordt hier een nieuwe versie van de syllabus gepost. Er wordt naar gestreefd dat de tekst tegen het einde van het semester is uitgegroeid tot een volwaardig handboek: cursustekst bij de lessen, zelfstandig doorwerkbaar studiemateriaal, en een naslagwerk dat ook in de volgende opleidingsjaren bruikbaar blijft. Het niveau ligt bewust wat hoger dan wat je van een eerste semester verwacht; wat dieper gaat dan de les, staat in de kaders en is als zodanig gemarkeerd.

Bij de ontwikkeling van deze syllabus werd generatieve AI gebruikt als ondersteunend hulpmiddel voor onder meer redactie, structurering en uitwerking. Alle inhoud werd door de auteur gecontroleerd, bewerkt en gevalideerd. De auteur draagt de volledige inhoudelijke verantwoordelijkheid.

Inhoud

Hoofdstuk 1 - Het web als systeem

In een notendop
Wat je na dit hoofdstuk kan
1.1 Waarom dit vak bestaat
1.2 Het web als systeem
1.3 Je werkinstrumenten
1.4 HTML: elementen, tags en attributen
1.5 Het skelet van elk HTML-document
1.6 De DOM: jouw document is een boomstructuur
1.7 Tekst structureren
1.8 Semantiek: betekenis geven aan delen van de pagina
1.9 Links: de essentie van het web
1.10 Afbeeldingen
1.11 Meer dan tekst en beeld: video, audio en iframe
1.12 Namen voor later: id en class
1.13 Je pagina online
Onder de motorkap: van adresbalk tot pixels
Veelgemaakte fouten
Met AI aan je zij: gegenereerde HTML lezen
Oefeningen
Verder lezen

Hoofdstuk 2 - CSS: het geordende conflict

In een notendop
Wat je na dit hoofdstuk kan
2.1 Wat CSS is, en hoe ver het reikt
2.2 De anatomie van een stijlregel
2.3 CSS aan je pagina koppelen
2.4 Selectors, deel 1: element, class en id
2.5 Selectors, deel 2: combinators en pseudo-classes
2.6 Overerving
2.7 De cascade: wie wint en waarom
2.8 Meeteenheden: waarin meet het web?
2.9 Custom properties: je eerste designtokens
2.10 Het conflict lezen in DevTools
Onder de motorkap: hoe de browser één waarde bepaalt
Veelgemaakte fouten
Met AI aan je zij: gegenereerde CSS lezen
Oefeningen
Verder lezen

Hoofdstuk 3 - Alles is een box

In een notendop
Wat je na dit hoofdstuk kan
3.1 De anatomie van de box
3.2 De maat van de box, en de rekensom die iedereen fout heeft
3.3 De projectreset: waar elk project begint
3.4 Padding: ademruimte binnen de box
3.5 Borders, ronde hoeken en schaduw: de box aankleden
3.6 Margin: afstand tussen boxen
3.7 Display: het karakter van de box
3.8 Grenzen stellen: min, max en overflow
3.9 Afbeeldingen zijn ook boxen, met een eigen wil
3.10 Boxen lezen in DevTools
Onder de motorkap: waar boxen hun maat vandaan halen
Veelgemaakte fouten
Met AI aan je zij: gegenereerde layout-CSS lezen
Oefeningen
Verder lezen

Hoofdstuk 4 - Layout 1: de gereedschapskist

In een notendop
Wat je na dit hoofdstuk kan
4.1 De derde laag van display: containers die hun kinderen schikken
4.2 De vraag die elk layoutprobleem opent
4.3 Flexbox: verdelen langs één as
4.4 Groeien en krimpen: flex
4.5 Grid: een raster over twee assen
4.6 Plaatsen op het raster
4.7 Uitlijnen, centreren, en het grondplan van een pagina
4.8 Kiezen, en combineren
4.9 Layouts lezen in DevTools
Onder de motorkap: hoe fr verdeelt
Veelgemaakte fouten
Met AI aan je zij: gegenereerde layouts lezen
Oefeningen
Verder lezen

Hoofdstuk 5 - Layout 2: verdieping en plaatsing

In een notendop
Wat je na dit hoofdstuk kan
5.1 Named areas: het raster uittekenen
5.2 Flexbox verdiept: wrap, en wat flex: 1 echt betekent
5.3 Items die afwijken: align-self, order en de auto-marge
5.4 Positioning: uit de flow stappen
5.5 Stapelen: z-index
5.6 Afbeeldingen in het gelid: object-fit en aspect-ratio
5.7 De ouder selecteren: :has() in gebruik
5.8 Plaatsing lezen in DevTools
Onder de motorkap: ankers, stapels en plakkers
Veelgemaakte fouten
Met AI aan je zij: gegenereerd plaatsingswerk lezen
Oefeningen
Verder lezen

Hoofdstuk 6 - Responsive: het onbekende scherm

In een notendop
Wat je na dit hoofdstuk kan
6.1 Responsive is geen feature
6.2 De viewport: hoe breed is een telefoon?
6.3 Mobile-first, en de plattegrond die kantelt
6.4 Media queries: ingrijpen waar het breekt
6.5 Alles samen: het skelet van een pagina
6.6 De vloeibare basis: wat je al hebt
6.7 clamp(): vloeiend, maar met grenzen
6.8 Het zelforganiserende raster
6.9 Queries voorbij de breedte
6.10 Responsive afbeeldingen: het juiste bestand voor het juiste scherm
6.11 Testen op het onbekende scherm
Onder de motorkap: pixels die geen pixels zijn
Veelgemaakte fouten
Met AI aan je zij: gegenereerde responsiveness lezen
Oefeningen
Verder lezen

Hoofdstuk 1 - Het web als systeem

In een notendop

Het web is een gesprek tussen een client en een server, en een webpagina is een boomstructuur van elementen die betekenis geven aan inhoud. Dit hoofdstuk legt uit hoe dat gesprek verloopt (URL, HTTP, bestanden op een server), welke drie instrumenten je dit semester dagelijks gebruikt (VS Code, de browser met zijn DevTools, GitHub) en hoe je correct en betekenisvol HTML schrijft: het skelet, de boom, tekst, structuur, links en media. Aan het einde staat je eerste pagina online, en je weet waarom een fout in HTML niet luid faalt maar stil.

Wat je na dit hoofdstuk kan

Aan het einde van deze eerste week staat er een webpagina door jou gemaakt op het internet. Een echte pagina, met een echt adres, die van waar ook ter wereld geraadpleegd kan worden. Ze zal er nog eenvoudig uitzien - voor vormgeving is het wachten tot hoofdstuk 2 - maar ze bevat alles wat elke webpagina ter wereld bevat, van de site van je bank tot de startpagina van Wikipedia: een gestructureerde **boom van HTML-elementen** die betekenis geven aan informatie.

Onderweg leer je hoe het **web als systeem** in elkaar zit: wat er gebeurt tussen het intikken van een adres en het verschijnen van een pagina, hoe je met de drie kerninstrumenten van dit vak werkt (Visual Studio Code, de browser met zijn DevTools, en GitHub), en vooral hoe je **correct en betekenisvol HTML** schrijft. Dat laatste lijkt de eenvoudigste vaardigheid van dit vak. Het is ook de vaardigheid waarop alles rust wat volgt; wie ze slordig aanleert, betaalt daar in elk volgend hoofdstuk een prijs voor.

Het hoofdstuk heeft drie delen. Eerst het systeem en je gereedschap (1.1 tot 1.3): waarom dit vak bestaat, hoe client, server en browser samenwerken, en welke drie programma's je installeert. Dan de taal (1.4 tot 1.12): elementen, het skelet, de boom, en element per element wat je nodig hebt voor tekst, structuur, links en media. Ten slotte de laatste stap (1.13): je pagina online.

1.1 Waarom dit vak bestaat

Begin bij de vraag die je je waarschijnlijk al stelde: waarom zou je in 2026 nog leren om HTML, CSS en JavaScript met de hand te schrijven, als AI dat in seconden voor je doet?

Het antwoord: omdat AI dat inderdaad in seconden voor je doet. Elke opdracht in de cursus en de labo's kan een taalmodel oplossen. Als je dit vak benadert als een verzameling opdrachten die "af" moeten, dan kan je elk labo uitbesteden aan een chatbot en zal je elke week het gevoel hebben dat je

goed bezig bent. Tot de eerste toets die je zonder AI maakt. Dit overkomt elk jaar studenten. Laat het niet jouw ervaring zijn.

Waarom leggen we de lat dan net hoger in plaats van lager? Omdat er iets fundamenteels veranderd is aan wat een webdeveloper doet. Code produceren is goedkoop geworden. Code begrijpen, beoordelen en corrigeren is dat niet - en dat is precies waar het beroep naartoe beweegt. Een developer typt vandaag zelden nog een layout from scratch, maar leest voortdurend code die een model genereerde, en moet dan kunnen zien: dit klopt, dit klopt bijna, en dit is subtiel fout op een manier die over drie weken een probleem wordt. Dat zien vereist een model in je hoofd van hoe het web werkt. Wie dat model niet heeft, kan alleen maar hopen dat de chatbot gelijk heeft.

Dit vak bouwt dat model, laag per laag: eerst de boom van elementen (dit hoofdstuk), dan het "geordende conflict" dat CSS heet, dan het boxmodel, de layouts, en met JavaScript uiteindelijk de code die op gebruikers reageert. Handmatig kunnen bouwen is in dit vak geen nostalgisch ritueel; het is het bewijs dat het model in je hoofd zit. Daarom werken we bovendien *vanilla-only*: puur HTML, CSS en JavaScript, zonder frameworks of libraries. Het platform zelf kan vandaag meer dan ooit, en wie het platform kent, ziet later moeiteloos dat elke framework-laag daarbovenop dun is. Frameworks krijg je volgend jaar, in het vak Web Frameworks. Eerst dus het fundament.

Hoe we in dit vak met AI werken

We gaan niet doen alsof AI niet bestaat; we maken er expliciete afspraken over. Elke oefening in deze syllabus en elk labo draagt één van drie labels:

- **zonder AI**: je gebruikt geen AI. Dit label staat op de vaardigheidstoetsen en op oefeningen die daarop voorbereiden. Tijdens toetsen en examens is dit een feit: daar is geen AI beschikbaar
- **zonder AI aanbevolen**: je mag AI gebruiken, maar je traint dan minder jezelf. Je haalt er meer uit door het niet te doen. Dit label staat op de meeste drills
- **met AI**: je gebruikt AI en dat is het punt van de oefening. Je genereert, beoordeelt, verbetert en verantwoordt. Wat we oefenen, is jouw kritiek op wat een model maakte

Nog één afspraak: vanaf labo 2 wordt alles wat je in je repository zet automatisch nagekeken door onze feedback-tooling. In de loop van de week die daarop volgt hoor je waar je staat, met verwijzingen naar de sectie in deze syllabus waar de oplossing wordt uitgelegd. Die feedback is er om jou te helpen en telt niet mee voor je punten; je punten komen uit de toetsen en het examen. We zien als docenten wel wie meedoet en wie stilvalt - niet om te controleren, maar om je op tijd te kunnen helpen in plaats van pas in januari.

1.2 Het web als systeem

Voordat we HTML schrijven, moet je begrijpen wat er eigenlijk gebeurt als je een website bezoekt. Nog geen expertkennis - daar hebben we andere vakken en semesters voor - maar goed genoeg om te weten waar jouw werk straks past in het geheel.

Het web bestaat uit twee soorten spelers. Een **server is een computer die bestanden bewaart en op verzoek uitdeelt**; hij staat ergens in een datacenter (maar het kan ook jouw computer zijn) en doet niets anders dan wachten op verzoeken. Een **client is het programma dat die verzoeken doet**: in ons geval de browser op jouw laptop of smartphone. Het hele web is een eindeloze herhaling van hetzelfde gesprek: de client vraagt een bestand op, de server stuurt het terug, de client toont het resultaat.



Figuur 1.1 - De client (je browser) vraagt met een URL een bestand aan de server; de server antwoordt met een statuscode en het bestand, en de browser toont het.

Het adres: de URL

Het adres waarmee de client vraagt, ken je al jaren: de **URL (Uniform Resource Locator)**. Een URL heeft een vaste anatomie, en je kan hem lezen zoals je een adres op een envelop leest:

```
https://www.ap.be/opleidingen/toegepaste-informatica
└──┘   └─────────┘└──────────────────────────────┘
schema     host                  pad
```

Het **schema** (`https`) zegt welke taal client en server spreken. De **host** (`www.ap.be`) identificeert de server: een naam die via het DNS-systeem wordt vertaald naar het ip-adres van een machine. Het **pad** wijst naar één specifiek bestand of één specifieke pagina op die server. Onthoud vooral dat laatste: een URL wijst altijd ergens naartoe. Straks, wanneer je links en afbeeldingen schrijft, is elke `href` en elke `src` gewoon zo'n adres - lang of kort geschreven, maar altijd een wegbeschrijving naar een bestand.

Het gesprek: HTTP en HTTPS

De taal van het gesprek zelf heet **HTTP** (HyperText Transfer Protocol), en voor dit vak volstaat het in vijf zinnen. Eén: de browser stuurt een verzoek naar de server, met daarin de URL en wat context. Twee: de server zoekt het gevraagde bestand op. Drie: de server stuurt een antwoord terug, met een statuscode (200 betekent "hier is het", 404 betekent "bestaat niet") en het bestand zelf. Vier: de browser leest het bestand en ontdekt daarin verwijzingen naar méér bestanden - stylesheets, afbeeldingen, scripts - en herhaalt het gesprek voor elk daarvan. Vijf: uit al die bestanden samen bouwt de browser de pagina die jij ziet. Meer HTTP heb je dit semester niet nodig; in het tweede semester ga je zelf zulke gesprekken voeren vanuit code.

De **S** in `https` staat voor **secure**: hetzelfde gesprek, maar versleuteld. Zonder die S reist alles wat client en server elkaar sturen in leesbare tekst over het netwerk, en iedereen die ergens onderweg zit - de wifi van de koffiebar, je internetprovider, een router in een datacenter - kan meelezen én de inhoud onderweg veranderen. Met HTTPS gebeuren twee dingen. De verbinding wordt versleuteld, zodat meelezen zinloos wordt. En de server bewijst met een **certificaat** wie hij is, zodat je zeker weet dat `www.ap.be` echt de server van AP is en niet een kopie die iemand ertussen zette. Daarom toont je browser bij `http://` de waarschuwing *Niet beveiligd*, en daarom is HTTPS vandaag de norm en niet de uitzondering: ook voor een pagina zonder wachtwoorden of betalingen, want de bezoeker moet erop kunnen vertrouwen dat hij jouw pagina ziet en niet een aangepaste versie. Voor jou verandert dat weinig: GitHub Pages serveert je site automatisch via HTTPS, en de enige regel die je moet onthouden is dat alle adressen die je in je pagina's schrijft met `https://` beginnen. Een `http://`-afbeelding op een `https://`-pagina blokkeert de browser gewoon.

De bestanden op de server

Waar komen die bestanden op de server vandaan? Een website is - op het niveau waarop we dit semester werken - letterlijk een map met bestanden: HTML-bestanden voor de structuur en inhoud, CSS-bestanden voor de vormgeving, afbeeldingen, en straks JavaScript-bestanden voor de interactie. "Een site online zetten" betekent: die map op een server plaatsen. Dat is precies wat jij deze week doet, want een pagina staat online of ze bestaat niet. Een oefening die alleen op jouw laptop leeft, kan niemand bezoeken.

Eén bestandsnaam heeft een speciale rol: `index.html`. Wanneer een URL naar een map wijst in plaats van naar een bestand (zoals `https://www.ap.be/`), zoekt de server in die map naar `index.html` en stuurt dat terug. Daarom heet de startpagina van vrijwel elke website zo, en daarom heten de startbestanden van al jouw oefeningen zo.

De drie bouwstenen

Elke webpagina, hoe complex ook, is opgebouwd uit maximaal drie talen, elk met een strikte taakverdeling:

- **HTML** (HyperText Markup Language) beschrijft de *structuur en betekenis* van de inhoud: dit is een titel, dit is een paragraaf, dit is een lijst, dit is navigatie
- **CSS** (Cascading Style Sheets) beschrijft de *vormgeving*: lettertypes, kleuren, witruimte, layout, tot en met animatie
- **JavaScript** beschrijft het *gedrag*: wat er gebeurt als de gebruiker klikt, typt, scrolt

Die taakverdeling is een ontwerpprincipe dat je vanaf vandaag hanteert. HTML gaat nooit over vorm. Je kiest een element nooit omdat het er groot of vet uitziet, maar omdat het de juiste betekenis draagt; hoe het eruitziet, beslis je later in CSS, en dat kan er compleet anders uitzien dan wat de browser standaard toont.

Een beknopte geschiedenis van het web

Het (wereldwijde) web - ook www genoemd - is geen ontwerp van één bedrijf; het groeide in ruim dertig jaar uit een handvol beslissingen die vandaag nog altijd gelden. Twee begrippen worden daarbij vaak door elkaar gehaald. Het **internet** is het netwerk: de kabels, routers en afspraken (TCP/IP) waarmee computers wereldwijd pakketjes data naar elkaar sturen. Het bestaat sinds de jaren zeventig en regelde lang vóór het web al e-mail en bestandsoverdracht. Het **web** is één toepassing op dat netwerk: documenten met links ertussen, opgevraagd via HTTP en geschreven in HTML. Het is "uitgevonden" in 1989 door Tim Berners-Lee, een informaticus bij het CERN in Genève, die een manier zocht om onderzoekers documenten te laten delen. Zijn drie afspraken - de URL als adres, HTTP als gesprek, HTML als documenttaal - zijn precies de drie die je zonet leerde, en de beslissing van het CERN in 1993 om die technologie gratis en zonder patent vrij te geven, is de reden dat het web van iedereen is en niet van een bedrijf.

De tijdlijn hieronder bevat de gebeurtenissen die je als IT-er moet kunnen plaatsen, met telkens waarom ze voor jou nog relevant zijn.

JAAR	GEBEURTENIS	WAAROM HET VOOR JOU BELANGRIJK IS
1969	ARPANET, de voorloper van het internet, verbindt de eerste computers	het netwerk is ouder dan het web
1983	TCP/IP wordt de standaard van het internet; het DNS wordt ingevoerd	namen zoals `www.ap.be` bestaan sinds dan
1989	Tim Berners-Lee stelt bij het CERN het World Wide Web voor	URL, HTTP en HTML worden in één ontwerp geboren
1991	de eerste website gaat online; het web verlaat het CERN	een website is nog altijd wat het toen was: een map met documenten
1993	het CERN geeft het web vrij; de browser Mosaic toont afbeeldingen in de pagina	het web wordt van iedereen, en het wordt visueel
1994	het W3C wordt opgericht; Netscape Navigator verschijnt	er is een orgaan dat standaarden schrijft, en een browser die ze al dan niet volgt
1995	JavaScript wordt in tien dagen gebouwd bij Netscape; HTML 2.0 is de eerste officiële HTML-standaard	de derde bouwsteen bestaat, met haar eigenaardigheden (hoofdstuk 10)
1996	CSS1 wordt een aanbeveling van het W3C	vorm wordt losgekoppeld van structuur: de taakverdeling van 1.2
1998	DOM Level 1; Google wordt opgericht	de boom van 1.6 krijgt een standaard-API; zoekmachines worden een lezer van je pagina
1997-2001	de browseroorlog tussen Netscape en Internet Explorer	jarenlang moest elke site per browser anders geschreven worden; de reden dat standaarden uiterst belangrijk zijn
2004	Firefox 1.0; Gmail toont wat een pagina kan zonder te herladen (Ajax); de WHATWG wordt opgericht	de browser wordt een platform voor toepassingen
2007	de iPhone	vanaf nu is hét scherm een veelheid aan schermen: hoofdstuk 6
2008	Google Chrome verschijnt, met een snelle JavaScript-engine	JavaScript wordt een serieuze taal; de DevTools van 1.3 zijn Chromes erfenis
2014	HTML5 wordt de officiële standaard	de semantische elementen van 1.8 (header, nav, main, article) en video en audio (1.11)
2015	ES2015 (ES6) vernieuwt JavaScript grondig; HTTP/2	het moderne JavaScript-idioom van deze syllabus
2017	CSS Grid werkt in alle grote browsers	layout krijgt eindelijk een speciaal daarvoor ontwikkelde CSS-techniek (hoofdstuk 4)

JAAR	GEBEURTENIS	WAAROM HET VOOR JOU BELANGRIJK IS
2019	W3C en WHATWG spreken af: HTML is één "living standard"	versienummers zijn voorbij; er is alleen nog "HTML"
2022-2023	container queries, `:has()` en CSS nesting; ChatGPT verschijnt; Baseline wordt ingevoerd	het platform kan meer dan ooit, code genereren wordt triviaal, en "kan ik dit al gebruiken?" krijgt een helder antwoord
2025	de European Accessibility Act treedt in werking	toegankelijkheid is een wettelijke eis (1.8)

Twee lessen uit die tijdlijn. Ten eerste: de afspraken van 1989 zijn nooit vervangen, alleen uitgebreid. Wie URL, HTTP en HTML begrijpt, begrijpt het web van vandaag. Ten tweede: het web is geen product maar een standaard, geschreven door organisaties waarin browsermakers samen beslissen (het **W3C** voor CSS en toegankelijkheid, de **WHATWG** voor HTML en de DOM, **Ecma** voor JavaScript). Daarom kan je op MDN per feature opzoeken of ze al in alle browsers werkt, en daarom bestaat de code die je dit semester schrijft over tien jaar nog.

1.3 Je werkinstrumenten

In dit vak gebruik je drie instrumenten, en je moet ze alle drie goed onder de knie krijgen.

Visual Studio Code is je editor: het programma waarin je code schrijft. Installeer daarnaast meteen de extensie *Live Server*: die toont je pagina in de browser en ververst automatisch bij elke wijziging die je bewaart. Code voor het web is tekst; een editor is gebouwd om tekst exact te behandelen. Schrijf nooit code in een tekstverwerker - die "verbetert" je aanhalingstekens en breekt daarmee je bestanden.

De browser (met DevTools). We gebruiken in de lessen Chrome, en kijken er je werk mee na, maar Firefox of Edge werken net zo goed. Belangrijker dan de browserkeuze is één toetscombinatie die je vandaag nog uit je hoofd leert: die voor de **DevTools** (rechtsklik op een pagina en kies *Inspect*, of F12 op Windows en Cmd-Option-I op Mac). De DevTools zijn het instrumentenpaneel van de browser, en het tabblad *Elements* toont je iets essentieels: niet de tekst die de developer schreef, maar de boom die de browser ervan bouwde. Vanaf vandaag is dit je eerste reflex bij elke vraag over een pagina - de jouwe of die van iemand anders. Open de inspector, kijk naar de boom, en je ziet wat de browser ziet. In dit vak leren we de DevTools niet "erbij"; het is een kerninstrument dat in elk hoofdstuk terugkomt.

GitHub is de plek waar jouw werk leeft en vanwaar het gepubliceerd wordt. Het concept in drie zinnen. Op je computer heb je een map met je bestanden; op GitHub staat een kopie van die map, je **repository** (kortweg: repo). Telkens je een betekenisvolle stap hebt gezet, breng je beide in overeenstemming en schrijf je in één zin wat je deed. GitHub bewaart daarbij de volledige geschiedenis, en publiceert de inhoud via **GitHub Pages** automatisch als website op een echt adres.

Meer Git-theorie dan dit heb je dit semester niet nodig - geen terminal, geen commando's, geen branches. In labo 1 zet je je eerste pagina's online zoals het eenvoudigst kan: je maakt in de browser een repository `webtech` aan, sleept je map erin en zet Pages aan. Vanaf labo 2 gaat het rechtstreeks

vanuit VS Code, met één klik (*Commit & Sync*); de volledige stappenkaart met schermafbeeldingen daarvoor vind je in **appendix A**, en vanaf dan is publiceren een routine van tien seconden. Waarom we dit vanaf week 1 doen en niet "later, als het er echt toe doet": ten eerste omdat een gepubliceerde site het vak zichtbaar en echt maakt, ten tweede omdat je repository vanaf labo 2 ook de manier is waarop je labowerk wordt ingeleverd en automatisch van feedback voorzien, en ten derde omdat werken via repositories dé samenwerkingsvorm is van moderne development - ook, en steeds meer, de samenwerking tussen developers en AI-agents.

Andere manieren om te publiceren

GitHub Pages is niet de enige weg. Elke dienst die een map met bestanden op een server zet, doet hetzelfde werk als in 1.2 beschreven, en het loont om er minstens één andere te kennen. **Netlify** is de snelste: ga naar app.netlify.com, kies *Drop*, sleep je map in het venster, en enkele seconden later staat ze online op een adres als `https://willekeurige-naam.netlify.app`. Geen repository, geen instellingen; de gratis versie volstaat voor een studentensite. Netlify kan ook aan een GitHub-repository gekoppeld worden, zodat het net als Pages bij elke wijziging opnieuw publiceert. Verwante diensten zijn Cloudflare Pages en Vercel, en de klassieke variant is een hostingpakket waarbij je de map met een FTP-programma naar de server kopieert - de manier waarop het web twintig jaar werkte en waarop veel kleine sites nog altijd draaien. Voor dit vak kiezen we GitHub Pages, omdat publiceren dan samenvalt met het bewaren van je geschiedenis en met inleveren. Maar onthoud het principe: een site is een map, en publiceren is die map ergens neerzetten waar een server ze kan uitdelen.

1.4 HTML: elementen, tags en attributen

Tijd voor de taal zelf. Kijk naar deze vier regels; ze zijn stuk voor stuk HTML-elementen:

```
<h1>Ik ben een hoofding</h1>
<p>Ik ben een paragraaf met daarin <em>benadrukte</em> tekst.</p>
<a href="https://www.ap.be">Ik ben een link naar de AP-site</a>
<img src="logo.png" alt="Het logo van de opleiding">
```

Een **element** is de combinatie van tags en inhoud. De meeste elementen bestaan uit een **openingstag** (`<p>`), inhoud, en een **sluittag** (`</p>` - let op de slash). De tags geven betekenis aan de inhoud ertussen: dezelfde woorden worden een hoofding, een paragraaf of een link, puur door de tags die je kiest. Sommige elementen hebben geen inhoud en dus ook geen sluittag; `<img>` is er zo één - er bestaat geen "tekst binnen een afbeelding", dus het element is in zichzelf compleet. We noemen dat een **void element**.

In de openingstag kunnen **attributen** staan: naam-waardeparen die extra informatie aan het element koppelen. Ze hebben altijd dezelfde vorm - naam, gelijkheidsteken, waarde tussen aanhalingstekens - en staan gescheiden door spaties:

```
<a href="https://www.ap.be">...</a>
<img src="logo.png" alt="Het logo van de opleiding">
```

Sommige attributen zijn verplicht (`href` bij een link, `src` en `alt` bij een afbeelding), de meeste zijn optioneel. Welke attributen een element kent, zoek je op; niemand kent ze allemaal uit het hoofd, en de referentie die we consequent gebruiken is **MDN Web Docs** (developer.mozilla.org) - de documentatie van het webplatform zelf.

Elementen kunnen ten slotte in elkaar **genest** worden: de paragraaf hierboven bevat een `em`-element. Nesting is geen detail; het is het hele organisatieprincipe van HTML, en het brengt ons bij de belangrijkste denkstap van dit hoofdstuk. Maar eerst het vaste vertrekpunt van elk document.

1.5 Het skelet van elk HTML-document

Elk HTML-bestand dat je dit semester schrijft, begint met exact dit skelet:

```
<!doctype html>
<html lang="nl">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Titel van deze pagina</title>
  </head>
  <body>
    <!-- your page content goes here -->
  </body>
</html>
```

Regel voor regel. De **doctype** vertelt de browser: dit is modern HTML, behandel het volgens de huidige standaard. Het **html-element** is het root-element dat al de rest omvat; het `lang`-attribuut vertelt aan browsers, zoekmachines en schermlezers in welke taal de pagina geschreven is. Dat lijkt een detail, maar een schermlezer die Nederlands voorleest met een Engelse stem produceert brij - vul `lang` dus altijd correct in.

Binnen het root-element leven precies twee kinderen. De **head** bevat informatie *over* de pagina, die niet op de pagina zelf verschijnt: de karaktercodering (`utf-8`, zodat accenten en emoji correct werken), de viewport-instelling (die vertelt mobiele browsers dat je pagina zich aan hun schermbreedte aanpast - het volledige verhaal volgt in hoofdstuk 6, maar hij staat er vanaf dag één), en de **title**, de tekst die in het browsertabblad en in zoekresultaten verschijnt. De **body** bevat alles wat de bezoeker wél ziet: daarin schrijf jij vanaf nu je pagina's.

De regel `` is **commentaar**: tekst voor menselijke lezers die de browser volledig negeert. Je zet er notities mee in je code, of je schakelt er tijdelijk een stuk opmaak mee uit. Gebruik het om te verhelderen wat niet vanzelf spreekt - voor de anderen die jouw code lezen, en voor jezelf over drie weken.

Merk ten slotte de insprongen op: elk genest element springt één niveau in. De browser is daar blind voor - HTML is ongevoelig voor extra witruimte - maar mensen niet. Insprongen zijn hoe jij, je medestudenten, je docenten en straks je collega's de structuur van een document in één oogopslag

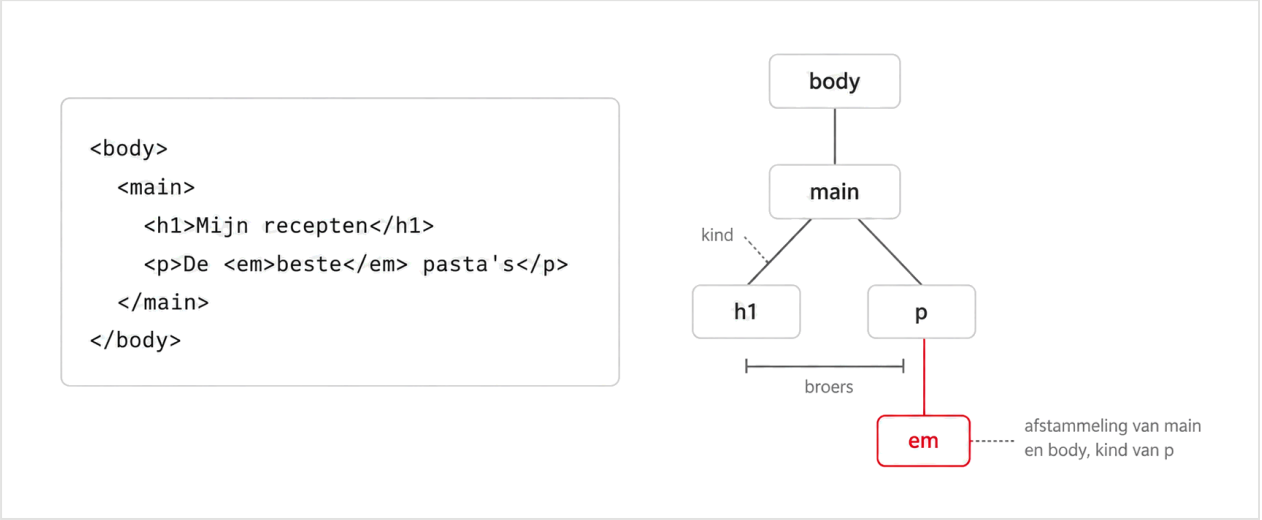
lezen. Netjes inspringen is in dit vak geen stilistische voorkeur maar een vereiste, om dezelfde reden als leesbaar handschrift bij een wiskundebewijs: wie de structuur niet toont, maakt fouten die hij zelf niet meer ziet.

1.6 De DOM: jouw document is een boomstructuur

Nu de denkstap die dit hoofdstuk zijn naam geeft. Kijk nog eens naar het skelet, en let op wie in wie genest zit: `title` zit in `head`, `head` zit in `html`. Nesting maakt van een document een hiërarchie, en die hiërarchie heeft een naam: het **Document Object Model**, kortweg de **DOM**.

Wanneer de browser jouw HTML-bestand ontvangt, leest hij de tekst en bouwt er een boomstructuur van. Elk element wordt een knoop in die boom. We gebruiken daarvoor de woordenschat van een stamboom: `head` en `body` zijn de **kinderen** (children) van `html`, dat hun **ouder** (parent) is; `head` en `body` zijn elkaars **broers** (siblings); en alles wat ergens onder een element hangt, hoe diep ook, is een **afstammeling** (descendant) van dat element.

```
<body>
  <main>
    <h1>Mijn recepten</h1>
    <p>De <em>beste</em> pasta's van het studentenleven.</p>
  </main>
</body>
```



Figuur 1.2 - Hetzelfde fragment als code en als boom: `body` heeft één kind `main`, `main` heeft de kinderen `h1` en `p`, en `em` is een kind van `p` en een afstammeling van `main` en `body`.

Teken dit voorbeeld als boom en je krijgt: `body` heeft één kind (`main`); `main` heeft twee kinderen (`h1` en `p`); `p` heeft één elementkind (`em`). De `em` is een afstammeling van `main` én van `body`, maar een kind van alleen `p`.

Waarom is dit het belangrijkste concept van het hoofdstuk? Omdat álles wat volgt in dit vak op deze boom werkt. CSS-regels selecteren knopen in de boom ("elke `em` die ergens in een `li` zit") en stijlen erven langs de takken naar beneden - dat wordt hoofdstuk 2. JavaScript vraagt knopen op, verandert

ze, maakt nieuwe knopen en hangt ze in de boom - dat wordt hoofdstuk 11. En de DevTools die je in 1.3 leerde openen, tonen je niet jouw tekstbestand maar precies deze boom, zoals de browser hem gebouwd heeft. Wie in bomen denkt, leest CSS-selectors straks als gewone zinnen. Wie HTML als "tekst met haakjes" blijft zien, leert elke regel apart uit het hoofd.

Een praktisch gevolg om nu al te onthouden: nesting is alles-of-niets. Een element zit volledig binnen zijn ouder of volledig erbuiten; tags mogen elkaar nooit kruisen. `<p><em>fout</p></em>` is geen boom meer en dus geen geldig document - de browser zal er iets van proberen te maken (daarover meer in het kader *Onder de motorkap*), maar wat hij ervan maakt, is niet meer wat jij bedoelde.

1.7 Tekst structureren

Met het skelet en het boommodel op zak kunnen we het binnenste van de body vullen. De elementen in deze sectie zijn de broodelementen van elk tekstdocument op het web.

Paragrafen en hoofdingen

Het **p-element** (paragraaf) is het werkpaard van tekst. Elke alinea lopende tekst hoort in een eigen `<p>`; tekst die "los" in de body staat, zonder omhullend element, noemen we naakte tekst en is een teken van slordige opmaak - elk stuk informatie verdient een element dat zegt wat het is.

Hoofdingen geven je document zijn geleding, in zes niveaus van `<h1>` tot en met `<h6>`. De niveaus vormen een inhoudsopgave, geen maten-en-gewichtentabel: `h1` is de titel van de pagina (één per pagina), `h2`-elementen zijn de hoofdstukken, `h3`-elementen de onderdelen daarvan, enzovoort. Je kiest een niveau dus nooit omdat het "de juiste grootte" heeft - grootte is CSS - maar omdat het de juiste plaats in de inhoudsopgave uitdrukt, en je slaat geen niveaus over. Test jezelf: als je alleen de hoofdingen van je pagina onder elkaar zet, moet er een logische inhoudsopgave staan. Schermlezers doen letterlijk dit - gebruikers ervan springen van hoofding naar hoofding - en zoekmachines lezen je pagina op dezelfde manier.

```
<h1>Mijn kookblog</h1>
<h2>Pasta carbonara voor beginners</h2>
<p>Deze klassieker vraagt maar vijf ingrediënten.</p>
<h3>Ingrediënten</h3>
<p>Spaghetti, guanciale, eieren, pecorino en zwarte peper.</p>
<h2>Over deze blog</h2>
<p>Koken op kot, zonder oven en zonder budget.</p>
```

Lijsten

Alles wat meer dan één keer voorkomt, is een kandidaat-lijst - en op het echte web zijn lijsten overal, veel breder dan boodschappenlijstjes: navigatiebalken, zoekresultaten, productoverzichten en fotogalerijen zijn allemaal lijsten. HTML geeft je er twee soorten. Een **ongeordende lijst** (`<ul>`, unordered list) gebruik je wanneer de volgorde er niet toe doet; een **geordende lijst** (`<ol>`, ordered

list) wanneer de volgorde betekenis draagt, zoals bij de stappen van een recept. Binnenin bestaan beide uit **lijstitems** (`<li>`), en dat is meteen een harde regel: een `ul` of `ol` heeft als directe kinderen uitsluitend `li`-elementen.

```
<h3>Bereiding</h3>
<ol>
  <li>Kook de pasta beetgaar.</li>
  <li>Bak de guanciale krokant.</li>
  <li>Meng alles van het vuur af met het eimengsel.</li>
</ol>
```

Binnen een lijstitem mag je vrijwel alles kwijt: tekst, maar ook hoofdingen, paragrafen, afbeeldingen of een nieuwe lijst. Met dat laatste maak je geneste lijsten, en daar toont zich het boommodel opnieuw - let goed op wáár de binnenlijst staat: als kind van een `li`, niet als rechtstreeks kind van de buitenlijst:

```
<ul>
  <li>Pasta's</li>
  <li>
    Sauzen
    <ul>
      <li>Carbonara</li>
      <li>Arrabbiata</li>
    </ul>
  </li>
</ul>
```

De bolletjes en nummers die de browser toont, zijn standaardvormgeving; vanaf hoofdstuk 2 bepaal jij hoe een lijst eruitziet, tot en met lijsten die er helemaal niet meer als "lijstjes" uitzien. De structuur blijft; het uitzicht is vrij. Dat is de taakverdeling van 1.2 in de praktijk.

Accenten binnen tekst: em en strong

Binnen lopende tekst kan je woorden een speciale lading geven. Het **em-element** (emphasis) legt klemtoon zoals je dat sprekend zou doen: "dit was *niet* de bedoeling". Het **strong-element** markeert belang of urgentie: "bewaar dit bestand **voor** je het sluit". De browser toont em standaard cursief en strong standaard vet, maar - je voelt het patroon al - dat is uitzicht en dus aanpasbaar; je kiest deze elementen om wat ze betekenen. Wil je alleen maar dat iets er cursief of vet *uitziet*, zonder betekenislading, dan is dat een taak voor CSS, niet voor markup.

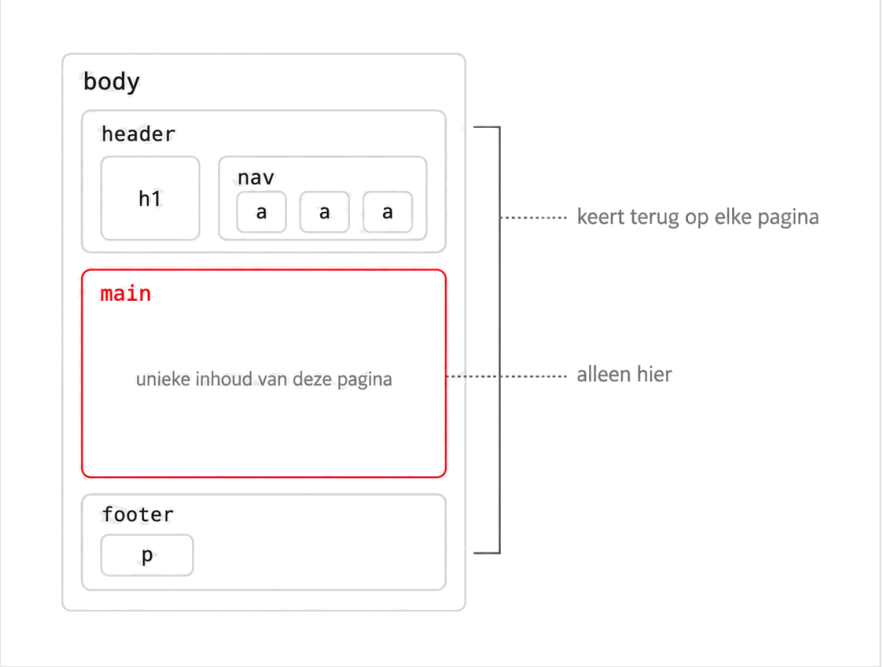
`em` en `strong` gedragen zich anders dan de elementen hierboven: ze leven *binnen* een tekstregel in plaats van er een eigen blok mee te vormen. Dat onderscheid - elementen die als blokken onder elkaar vallen tegenover elementen die in de tekststroom meelopen - zie je op elke pagina, en het krijgt in hoofdstuk 3 een precieze verklaring in het boxmodel. Voor nu volstaat de observatie.

1.8 Semantiek: betekenis geven aan delen van de pagina

De elementen tot nu toe beschrijven kleine eenheden: een titel, een alinea, een lijst. Maar een pagina heeft ook een grofstructuur - een kop, een navigatie, een hoofdinhoud, een voet - en ook daarvoor heeft HTML betekenisdragende elementen. We noemen ze de **semantische verzamelelementen**: containers die andere elementen bundelen én benoemen wat die bundel is.

Het **main-element** trekt een cirkel rond de hoofdinhoud van de pagina: de informatie die deze pagina uniek maakt. Elke pagina heeft er precies één. Het **header-element** bevat de inleidende bovenkant - vaak een logo, de sitenaam en de navigatie - en het **footer-element** de afsluitende onderkant, met bijvoorbeeld auteurs- en contactinformatie. Header en footer horen niet in main thuis: zij bevatten wat op elke pagina van de site terugkeert, main bevat wat alleen hier staat. Het **nav-element** ten slotte markeert een blok navigatielinks - en omdat navigatie een verzameling gelijkaardige links is, herken je er als vanzelf een lijst in:

```
<body>
  <header>
    <h1>Mijn kookblog</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="recepten.html">Recepten</a></li>
        <li><a href="over-mij.html">Over mij</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <!-- unique content of this page -->
  </main>
  <footer>
    <p>Gemaakt door Tim Peeters, student Toegepaste Informatica.</p>
  </footer>
</body>
```



Figuur 1.3 - Het grondplan van een pagina: `header` (met `h1` en `nav`) en `footer` keren op elke pagina terug; `main` bevat wat alleen op deze pagina staat.

Dit patroon - header, main, footer als kinderen van body - is het **grondplan** van vrijwel elke webpagina, en het is het startpunt van al je oefeningen vanaf nu. Voor fijnere geleding bestaan nog drie verwanten die je moet kennen: **article** voor een op zichzelf staand stuk inhoud dat ook los van de pagina betekenis zou hebben (een blogpost, een nieuwsbericht, een productkaart), **section** voor een thematisch deel van een pagina dat typisch een eigen hoofding heeft, en **aside** voor inhoud die naast het hoofdverhaal staat (een zijbalk, een "lees ook"-blok). De grenzen tussen die drie zijn soms een kwestie van oordeel; dat oordeel ontwikkel je door te oefenen, en de vuistregels op MDN helpen bij twijfel.

En wat als geen enkel betekenisdragend element past? Dan - en pas dan - grijp je naar de twee neutrale elementen: **div** als betekenisloze blokcontainer en **span** als betekenisloos stukje binnen een tekstregel. Ze zeggen niets over hun inhoud; het zijn haakjes waarmee je elementen kan groeperen om er straks met CSS of JavaScript iets mee te doen. Onthoud de rangorde: eerst een semantisch element zoeken, div en span als laatste redmiddel. In hoofdstuk 2 wordt duidelijk waarom je desondanks véél divs zal zien - en in het kader *Met AI aan je zij* hieronder waarom je er in gegenereerde code nog meer zal zien dan nodig.

Waarom semantiek geen schoonheidsprijs is

Je zou kunnen denken: de bezoeker ziet toch niet welke tags ik gebruikte - waarom zou ik moeite doen als een stapel divs er hetzelfde uitziet? Omdat mensen met ogen op een scherm maar één soort lezer van je pagina zijn, en de andere lezers uitsluitend je markup hebben.

Schermlezers - de voorleessoftware waarmee blinde en slechtziende gebruikers het web gebruiken - navigeren via jouw structuur: van hoofding naar hoofding, rechtstreeks naar main of nav. Een pagina zonder semantiek is voor hen een muur van ongedifferentieerde tekst. Dit is bovendien sinds juni 2025 geen vrijblijvende kwestie meer: de **European Accessibility Act** verplicht digitale toegankelijkheid

voor commerciële websites, webshops en apps in de hele EU, op basis van de WCAG-richtlijnen. Toegankelijkheid is daarmee een wettelijke eis en een gevraagde beroepscompetentie geworden, en in dit vak is het een rode draad: elk hoofdstuk behandelt de toegankelijkheidskant van zijn onderwerp. Voor dit hoofdstuk is dat: correcte semantiek, een kloppende hoofdingenstructuur, `lang` op het html-element, en straks het alt-attribuut.

Zoekmachines lezen je pagina op dezelfde manier: je title, je hoofdingen en je structuur bepalen mee hoe je gevonden wordt. En de jongste lezersgroep groeit het snelst: **AI-systemen en agents** die pagina's samenvatten, ernaar verwijzen of er taken op uitvoeren, doen dat betrouwbaarder naarmate de structuur betekenis draagt. Goede semantiek is dus geen beleefdheid; het is schrijven voor álle lezers van je pagina - en drie van de vier lezersgroepen zien alleen je boom.

1.9 Links: de essentie van het web

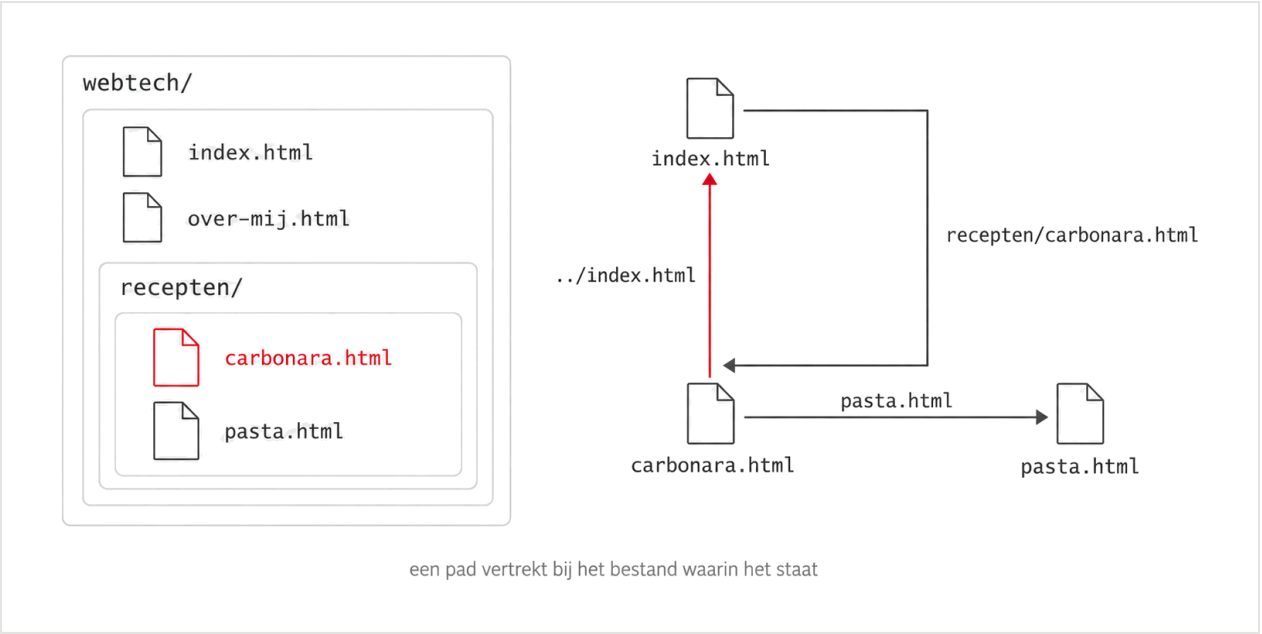
De "HT" in HTML en HTTP staat voor hypertext: tekst met sprongen erin. Het **a-element** (anchor) maakt die sprongen. Het verplichte `href`-attribuut bevat het adres van de bestemming; de inhoud tussen de tags is wat aanklikbaar wordt:

```
<a href="https://www.ap.be">Bezoek de site van AP</a>
```

Voor de waarde van `href` grijpen we terug naar de URL-anatomie van 1.2, want er zijn twee manieren om een adres te schrijven. Een **absolute URL** is het volledige adres, met scheme en host. Die gebruik je om te linken naar een andere website dan de jouwe. Absolute URL's kunnen lang en intimiderend ogen; laat je niet afschrikken, het blijft één attribuut met één waarde.

Een **relatieve URL** beschrijft de weg vanaf het huidige bestand, en die gebruik je voor alles binnen je eigen site. De regels zijn dezelfde als die van mappen op je computer. Een bestandsnaam alleen (`recepten.html`) wijst naar een bestand in dezelfde map. Een pad met mapnamen (`recepten/carbonara.html`) daalt af in submappen. En `../` klimt één map omhoog - herhaalbaar, dus `../../index.html` klimt er twee:

```
<a href="recepten.html">Alle recepten</a>
<a href="recepten/carbonara.html">Carbonara</a>
<a href="../index.html">Terug naar de homepagina</a>
```



Figuur 1.4 - Een relatief pad vertrekt bij het bestand waarin het staat: vanuit `carbonara.html` wijst `pasta.html` naar een buur en klimt `../index.html` één map omhoog; vanuit `index.html` daalt `recepten/carbonara.html` af.

Relatieve paden zijn de eerste plek waar het vak precisie afdwingt: één letter verschil, een vergeten map of een hoofdletter te veel, en de link is dood. Twee gewoontes voorkomen vrijwel alle ellende. Geef bestanden en mappen namen zonder spaties, zonder accenten en volledig in kleine letters (`mijn-recepten.html`, niet `Mijn Recepten.html`) - op jouw computer werkt het misschien allebei, op de server niet, en het verschil ontdek je pas na het publiceren. En denk vanaf het begin na over je mappenstructuur, want de paden schrijf je vanuit die structuur.

Een derde vorm zal je vaak tegenkomen in andermans code: paden die met `/` beginnen (`/recepten/carbonara.html`). Die starten vanaf de wortel van de site in plaats van vanaf het huidige bestand. Handig op grote sites, maar met een addertje: ze werken alleen op een echte server, en anders dan je zou verwachten wijst `/` op jouw GitHub Pages-adres niet naar jouw project maar naar het domein erboven. We gebruiken dit semester daarom consequent relatieve paden zonder beginnende slash; herken de vorm, gebruik hem nog niet.

1.10 Afbeeldingen

Het **img-element** plaatst een afbeelding in de pagina. Het is een void element (geen inhoud, geen sluittag) met twee verplichte attributen. **src** bevat het adres van het afbeeldingsbestand en werkt exact zoals href: absoluut of relatief, met dezelfde padregels - afbeeldingen krijgen in jouw projecten een eigen map:

```
<img src="afbeeldingen/carbonara.jpg" alt="Bord spaghetti carbonara met verse pecorino">
```

Het tweede verplichte attribuut verdient meer dan een vermelding. **alt** bevat het tekstalternatief voor de afbeelding: wat een schermlezer voorleest, wat de browser toont als het bestand niet laadt, en wat zoekmachines en AI-systemen lezen. Een goed alt-attribuut beschrijft wat de afbeelding

communiceert, in de context van de pagina - niet dat het een afbeelding is (dat weet de lezer al). "Foto" is waardeloos; "bord spaghetti carbonara met verse pecorino" laat een blinde bezoeker hetzelfde weten als een ziende. Twee bijzondere gevallen: is de afbeelding puur decoratief en voegt ze geen informatie toe, schrijf dan een leeg alt-attribuut (`alt=""`) zodat schermlezers haar overslaan - het attribuut weglaten mag nooit. En bevat de afbeelding tekst (een logo met naam), dan is die tekst je alt.

Over de bestandsformaten hoef je dit semester maar dit te weten: **jpg** voor foto's, **png** voor afbeeldingen met scherpe randen of transparantie, **svg** voor logo's en iconen (het schaalt zonder kwaliteitsverlies, waarom hoor je in het motorkap-kader van hoofdstuk 3), en **webp** en **avif** als moderne, compactere vervangers voor jpg en png die je in professionele sites steeds vaker ziet. De browser toont ze allemaal met hetzelfde img-element.

Wil je een afbeelding een zichtbaar bijschrift geven, dan bestaat daar - geheel in de geest van dit hoofdstuk - een betekenisdragend duo voor: **figure** omhult de afbeelding, **figcaption** draagt het bijschrift:

```
<figure>
  <img src="afbeeldingen/carbonara.jpg" alt="Bord spaghetti carbonara">
  <figcaption>Het eindresultaat na twintig minuten werk.</figcaption>
</figure>
```

De width- en height-attributen die je in oudere code op img ziet, bespreken we pas in hoofdstuk 3 en 6: afmetingen zijn vormgeving, en vormgeving doen we in CSS. Eén uitzondering op die regel komt daar aan bod - ze heeft met laadgedrag te maken - maar tot dan schrijf je een img met src en alt, en niets meer.

1.11 Meer dan tekst en beeld: video, audio en iframe

Drie elementen maken de mediacatalogus compleet, en je zal ze sneller nodig hebben dan je denkt - de kans is groot dat je al in week twee een video op je persoonlijke site wil.

Het **video-element** speelt een videobestand af dat jij zelf serveert, precies zoals img een afbeeldingsbestand toont. Anders dan img heeft het wél een sluittag, en één attribuut is in de praktijk verplicht: **controls**, dat de afspeelknoppen van de browser toont. Zonder controls staat er een video die niemand kan starten:

```
<video src="media/demo.mp4" controls>
  <p>Je browser kan deze video niet afspelen.
     <a href="media/demo.mp4">Download het bestand.</a></p>
</video>
```

De inhoud tussen de tags is de reservetekst voor browsers die het bestand niet aankunnen - hetzelfde principe als alt bij img. Handige extra's: **poster** wijst naar een afbeelding die vóór het afspelen wordt getoond, en wie per se automatisch wil starten, moet weten dat browsers **autoplay** alleen toestaan in

combinatie met **muted** - het web heeft collectief besloten dat pagina's die ongevraagd geluid maken een plaag zijn, en de browser handhaaft dat. Voor geluid zonder beeld bestaat het **audio-element**, dat identiek werkt: src, controls, sluittag, reservetekst. Als formaat volstaat dit semester mp4 voor video en mp3 voor audio; die spelen overal.

Maar eerlijk is eerlijk: de meeste video op het web serveer je niet zelf. Videobestanden zijn groot, en platforms als YouTube en Vimeo doen het hosten, comprimeren en streamen beter dan jouw GitHub Pages-site ooit zal doen. Daarvoor dient het derde element. Een **iframe** (inline frame) is een venster naar een ándere pagina, midden in de jouwe - letterlijk een pagina in een pagina. Kies je op YouTube voor *Delen* - *Insluiten*, dan krijg je zo'n iframe-snippet om te plakken:

```
<!-- ...replace VIDEO-ID with the id from the YouTube address... -->
<iframe src="https://www.youtube.com/embed/VIDEO-ID"
        title="Demo van het eindproject"
        allowfullscreen></iframe>
```

Twee dingen moet je daaraan zelf doen. Ten eerste het **title-attribuut**: een iframe is voor een schermlezer een gesloten doos, en title is het etiket erop - dezelfde rol als alt bij img, en net zo verplicht in dit vak. Ten tweede zal je merken dat een geplakte iframe een vast formaat heeft dat zich nergens iets van aantrekt; hoe je hem netjes laat meeschalen met zijn omgeving, is een layoutkwestie die in hoofdstuk 5 haar gereedschap krijgt (aspect-ratio). Tot dan mag hij er wat stijfjes bijstaan.

Eén waarschuwing tot slot: een iframe voert andermans pagina uit binnen de jouwe. Voor YouTube en soortgelijke gevestigde diensten is dat prima; sluit geen pagina's in van bronnen die je niet vertrouwt, om dezelfde reden dat je geen onbekende USB-stick in je laptop steekt.

1.12 Namen voor later: id en class

Tot slot twee attributen die elk HTML-element mag dragen en die je nu al moet herkennen, ook al bewijzen ze hun nut pas vanaf hoofdstuk 2: **id** en **class** geven een element een naam waarmee CSS en JavaScript het kunnen vastnemen.

Het verschil zit in uniciteit. Een **id** identificeert: de waarde mag op de hele pagina maar één keer voorkomen, zoals een studentennummer. Een **class** classificeert: dezelfde waarde mag op zoveel elementen staan als je wil, zoals een klasgroep, en één element mag meerdere classes dragen, gescheiden door spaties:

```
<main>
  <p class="intro">De eerste paragraaf, met een eigen rol.</p>
  <p>Een gewone paragraaf.</p>
  <p class="intro belangrijk">Deze paragraaf draagt twee classes.</p>
</main>
```

Kies betekenisvolle namen, in kleine letters en zonder spaties, en beschrijf de rol van het element, niet het gewenste uitzicht: `class="intro"` veroudert nooit, `class="rode-tekst"` wordt een leugen zodra het design verandert. Waarom je meestal naar class grijpt en maar zelden naar id, wordt duidelijk in hoofdstuk 2, wanneer we zien hoeveel gewicht een id in de schaal legt bij het geordende conflict van CSS.

1.13 Je pagina online

Daarmee is de cirkel van dit hoofdstuk rond: je weet wat een server is (1.2), je hebt een repository op GitHub (1.3), en je kan een correcte pagina schrijven (1.4 tot en met 1.12). De laatste stap is de drie samenbrengen, en die stap is verrassend klein.

Jouw repository `webtech` is gekoppeld aan **GitHub Pages**: telkens er iets verandert in je repository, plaatst GitHub de bestanden automatisch op zijn servers, en enkele ogenblikken later staat de nieuwe versie live op jouw adres - `https://jouwgebruikersnaam.github.io/webtech/`. De server zoekt daar, zoals elke webserver (1.2), naar een `index.html`; de pagina's van labo 1 staan in de submap `labo-01/`, dus hun adres is `https://jouwgebruikersnaam.github.io/webtech/labo-01/index.html`. In labo 1 breng je je bestanden in de repository door je map in de browser te uploaden. Vanaf labo 2 wordt dat de vaste werkstroom van elke week in dit vak: bewerk je bestanden in VS Code, bekijk het resultaat live met Live Server, en als je tevreden bent: *Commit & Sync* met een korte boodschap die zegt wat je deed. Hoe je dat één keer opzet, staat stap voor stap in labo 2; daarna is het een routine van twee klikken.

Eén gewoonte geven we je vanaf dag één mee, omdat ze het verschil maakt tussen een geschiedenis die je helpt en een die niets zegt: commit per afgewerkte stap, niet per week. "Navigatie toegevoegd aan alle pagina's" is een commit; "van alles" is er geen. Je geschiedenis wordt zo een logboek van je werk - voor jezelf, voor je docenten, en voor de feedback-tooling die vanaf labo 2 met elke wijziging meekijkt.

Het visitekaartje van je pagina

Nu je pagina een publiek adres heeft, verdient ze ook een visitekaartje: drie regels in de head die niets aan de pagina zelf veranderen, maar alles aan hoe ze de buitenwereld tegemoet treedt. De **favicon** is het pictogrammetje in het browsertabblad: één regel (`<link rel="icon" href="favicon.svg">`) en jouw site is geen naamloos tabblad meer tussen de twintig andere. De **meta description** (`<meta name="description" content="...">`) is de samenvatting van één of twee zinnen die zoekmachines onder je title tonen in de resultatenlijst; schrijf je haar niet, dan knipt de zoekmachine zelf een willekeurig fragment uit je pagina. En de **Open Graph-tags** bepalen hoe je pagina eruitziet wanneer iemand de link deelt in een chat of op sociale media - het voorbeeldkaartje met titel, omschrijving en afbeelding:

```
<!-- ...inside head... -->
<meta property="og:title" content="Portfolio van Tim Peeters">
<meta property="og:description" content="Student Toegepaste Informatica aan AP.">
<meta property="og:image"
content="https://timpeeters.github.io/webtech/afbeeldingen/kaart.png">
```

Let bij og:image op het absolute adres: het kaartje wordt opgebouwd door de server van de chat-app, niet door jouw pagina, dus een relatief pad wijst voor hém nergens naartoe. Voor je persoonlijke site is dit drietal geen verplichting van week één, maar wel de afwerking die het verschil maakt zodra je je adres begint rond te sturen - en rondsturen is precies de bedoeling.

De validator

Eén instrument nog, en het is er één voor je hele loopbaan. Je leest in het motorkap-kader hieronder dat foute HTML niet luid faalt maar stil: de browser herstelt zwijgend, en jij ziet niets. Er bestaat gelukkig een lezer die wél luid faalt: de **HTML-validator** van het W3C (validator.w3.org). Plak er het adres van je gepubliceerde pagina in en je krijgt een lijst van alles wat formeel niet klopt, met regelnummers. Maak er een gewoonte van om elke pagina die je publiceert één keer door de validator te halen - het is de snelste manier om fouten te vinden die je oog nooit zou zien, en in de review-oefeningen van deze syllabus is het een vast onderdeel van de verificatiestap.

Onder de motorkap: van adresbalk tot pixels

Dit kader gaat dieper dan het hoorcollege. Je hebt het niet nodig om de oefeningen te maken - wel om te begrijpen waarom de regels van dit hoofdstuk bestaan.

Wat gebeurt er precies tussen het moment waarop je op enter drukt en het moment waarop de pagina er staat? De reis heeft vijf etappes.

Opzoeken. De browser kent van `www.ap.be` alleen de naam, geen locatie. Via het **DNS** (Domain Name System, een wereldwijd gedistribueerd adresboek) vertaalt hij de naam naar een IP-adres: het nummer van de machine die de site serveert.

Ophalen. Nu volgt het HTTP-gesprek uit 1.2: verzoek, antwoord, statuscode, en als alles goed gaat een stroom bytes die jouw HTML-bestand vormen. Bij HTTPS gaat daar nog een korte handdruk aan vooraf, waarin de server zijn certificaat toont en beide kanten een sleutel afspreken waarmee de rest van het gesprek versleuteld wordt.

Parsen. De browser leest die bytes van boven naar beneden en hakt ze in stukjes: openingstag, tekst, sluittag. Dit heet **parsen**, en het resultaat is de boom uit 1.6 - de DOM. Vanaf hier bestaat jouw bestand voor de browser niet meer; alles wat volgt, werkt op de boom. Dat is de diepere reden waarom de DevTools je de boom tonen en niet je bestand: je kijkt naar wat de browser daadwerkelijk heeft.

Aankleden en meten. Op de boom past de browser vervolgens de stijlregels toe (van jou, of zijn eigen standaardstijlen - dat verhaal begint in hoofdstuk 2) en rekent hij voor elke knoop uit waar hij komt te staan en hoe groot hij wordt (het boxmodel van hoofdstuk 3).

Schilderen. Ten slotte zet de browser het resultaat om in pixels op je scherm. Elke keer dat er iets verandert - jij past CSS aan, JavaScript wijzigt de boom - doorloopt hij de laatste etappes opnieuw. Hoe je die herberekeningen bewust gebruikt (en soms bewust vermijdt), komt terug bij transities en animaties in hoofdstuk 8.

Er zit in de parse-etappe nog een venijnig detail dat elke beginner een keer in de val lokt. HTML-parsers zijn radicaal vergevingsgezind: bij fouten stopt de browser niet met een foutmelding, maar *herstelt* hij je document volgens vaste regels tot een geldige boom. Vergeet je een sluittag, kruis je twee elementen, zet je een element op een plek waar het niet mag staan - de browser bouwt zwijgend de boom die volgens de herstelregels het dichtst in de buurt komt. Dat klinkt vriendelijk, maar het betekent: **foute HTML faalt niet luid, ze faalt stil.** De pagina "werkt", alleen is de boom niet wat jij dacht - en over drie weken snap je niet waarom een CSS-regel het ene lijstitem wel raakt en het andere niet. Daarom zijn de regels van dit hoofdstuk (correcte nesting, li's direct in ul, elk stuk informatie in een element) geen etiquette, en daarom is de Elements-tab je waarheidsinstrument: daar zie je de boom ná herstel. Wijkt hij af van wat je schreef, dan heeft de parser iets voor je "opgelost".

Veelgemaakte fouten

Dit katern keert in elk hoofdstuk terug: de fouten die we elk jaar zien, met symptoom, oorzaak en remedie. De debug-oefeningen en de feedback op je labo's putten er rechtstreeks uit en verwijzen naar deze nummers.

F1.1 - Naakte tekst. *Symptoom*: tekst staat op de pagina maar gedraagt zich vreemd zodra je gaat stylen. *Oorzaak*: de tekst staat rechtstreeks in body of main, zonder omhullend element. *Remedie*: elk stuk informatie krijgt een element dat zegt wat het is; lopende tekst hoort in een p.

F1.2 - Hoofdingen op uitzicht gekozen. *Symptoom*: een h4 pal onder een h1, want "dat was de juiste grootte". *Oorzaak*: hoofdingen behandeld als lettergroottes in plaats van als inhoudsopgave. *Remedie*: niveaus volgen de structuur (geen niveaus overslaan, één h1); groottes regel je in hoofdstuk 2 met CSS.

F1.3 - Gekruiste tags. *Symptoom*: styling of gedrag "lekt" naar tekst waar het niet hoort. *Oorzaak*: `<p><em>tekst</p></em>` - elementen overlappen in plaats van te nesten; de diepere oorzaak is dat er niet in bomen gedacht is. *Remedie*: een element zit volledig in zijn ouder; controleer de herstelde boom in de Elements-tab.

F1.4 - Andere kinderen dan li in een lijst. *Symptoom*: bullets op rare plaatsen, items die niet uitlijnen. *Oorzaak*: tekst of elementen rechtstreeks in ul of ol geplaatst. *Remedie*: ul en ol hebben uitsluitend li-kinderen; al de rest zit ín een li - ook geneste lijsten.

F1.5 - Vergeten of nietszeggende alt. *Symptoom*: geen zichtbaar symptoom, en dat is precies het probleem. *Oorzaak*: alt weggelaten of ingevuld als "foto" of "afbeelding1". *Remedie*: beschrijf wat de afbeelding communiceert; decoratief beeld krijgt alt="".

F1.6 - Spaties en hoofdletters in bestandsnamen. *Symptoom*: alles werkt lokaal, maar na publicatie zijn links dood en afbeeldingen kapot. *Oorzaak*: `Mijn Foto.JPG` - jouw besturingssysteem is soepel, de server niet. *Remedie*: kleine letters, koppeltekens, geen spaties of accenten - vanaf het eerste bestand.

F1.7 - Verdwaalde relatieve paden. *Symptoom*: een link of afbeelding werkt op de ene pagina wel en op de andere niet. *Oorzaak*: het pad is geschreven vanuit de verkeerde map; relatieve paden vertrekken bij het bestand waarín ze staan. *Remedie*: teken je mappenstructuur en wandel het pad na, map voor map, vanaf het bestand zelf.

F1.8 - Structuurelementen in de verkeerde rol. *Symptoom*: sitenavigatie in main, of paginatitel in de footer. *Oorzaak*: de vraag "keert dit terug op elke pagina of is dit uniek voor deze pagina?" is niet gesteld. *Remedie*: stel exact die vraag per blok; terugkerend hoort in header of footer, uniek in main.

F1.9 - Div-soep. *Symptoom*: een technisch werkende pagina die volledig uit divs bestaat. *Oorzaak*: de neutrale container gebruikt zonder eerst naar een betekenisdragend element te zoeken - een fout die je ook in veel gegenereerde code ziet. *Remedie*: de rangorde van 1.8: eerst semantiek, div als laatste redmiddel.

F1.10 - De boom niet gecontroleerd. *Symptoom*: "het ziet er goed uit, dus het is goed." *Oorzaak*: alleen naar het gerenderde resultaat gekeken, nooit naar de structuur. *Remedie*: maak de Elements-tab en de validator een vast onderdeel van je afwerkroutine; het motorkap-kader legt uit waarom het oog hier niet volstaat.

F1.11 - Media zonder etiket of bediening. *Symptoom*: een video die niemand kan starten, of een iframe dat voor schermlezers een naamloos gat in de pagina is. *Oorzaak*: controls vergeten op video of audio, title vergeten op iframe. *Remedie*: video en audio krijgen controls, elk iframe krijgt een beschrijvende title - dezelfde reflex als alt bij img.

Met AI aan je zij: gegenereerde HTML lezen

Vraag een taalmodel om een webpagina en je krijgt er binnen seconden een die werkt. In dit terugkerende kader leer je per hoofdstuk waar je dan op moet letten; bij HTML zijn dat deze vier patronen.

Div-soep en overbodige lagen. Modellen zijn getraind op miljoenen bestaande pagina's, en het gemiddelde van het web is geen goed voorbeeld. Je krijgt vaak stapels geneste divs met classes als `container`, `wrapper` en `content` waar één semantisch element volstond, en `<div class="header">` waar `<header>` bestaat. De structuur wérkt, maar drie van je vier lezersgroepen (1.8) verliezen erbij. Jouw controlevraag bij elke gegenereerde container: welk betekenisdragend element had hier gekund?

Plausibele maar lege alt-teksten. Gegenereerde afbeeldingen krijgen keurig een alt-attribuut - modellen "weten" dat het moet - maar de inhoud is vaak generiek ("een afbeelding van het product") of gokt naar wat het model niet kan zien. Formeel correct, functioneel waardeloos. Alt-teksten schrijven blijft mensenwerk: jij weet wat de afbeelding in déze context communiceert.

Verouderd idioom. Trainingsdata bevat twintig jaar web, dus gegenereerde code bevat geregeld het web van tien jaar geleden: `<b>` en `<i>` waar strong en em horen, `<br>`-tags als witruimte-gereedschap, inline styles in een style-attribuut (daarover meer in hoofdstuk 2). Het werkt allemaal; het is allemaal een teken dat je de output niet blind moet overnemen.

Hoofdingen op uitzicht. Ook modellen maken fout F1.2: ze kiezen wel eens hoofdingniveaus op verwachte grootte in plaats van op structuur. Jouw inhoudsopgave-test (alleen de hoofdingen lezen) werkt op gegenereerde pagina's net zo goed als op je eigen werk.

De rode draad, hier en in elk volgend hoofdstuk: het model produceert, jij beoordeelt - en beoordelen kan alleen wie de regels zelf kent. In de oefeningen hieronder doe je dit voor het eerst systematisch.

Oefeningen

Labo 1 bevat de opgaven van deze week, met de precieze bestandsnamen en de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen. De AI-labels staan in 1.1.

Drills (zonder AI aanbevolen)

- **D1.1** Schrijf het volledige skelet van een HTML-document uit het hoofd, inclusief lang, charset, viewport en title. Dit moet je op het einde van deze week foutloos kunnen, zonder na te denken.
- **D1.2** Je krijgt een platte tekst (een recept: titel, inleiding, ingrediënten, stappen, bronvermelding). Zet hem om in correct gestructureerde HTML met hoofdingen, paragrafen en de juiste lijstsoorten.
- **D1.3** Teken op papier het boomdiagram van een gegeven HTML-fragment, met de begrippen parent, child en sibling. Controleer jezelf daarna in de Elements-tab.
- **D1.4** Bouw een geneste lijst van drie niveaus diep (bv. opleidingen, met per opleiding vakken, met per vak onderwerpen) zonder één nesting-fout.
- **D1.5** Bouw een minisite van drie pagina's (index, recepten in een submap, over mij) met een gedeelde navigatie en werkende relatieve links in beide richtingen, inclusief een `../`-pad.
- **D1.6** Voeg aan een gegeven pagina drie afbeeldingen toe vanuit een afbeeldingenmap: één informatieve (met goede alt), één decoratieve (met leeg alt) en één met bijschrift via figure en figcaption.
- **D1.7** Sluit een YouTube-video van je keuze in via een iframe met een beschrijvende title, en voeg daarnaast een eigen (kort) videobestand toe met controls en reservetekst. Haal daarna de gepubliceerde pagina door de W3C-validator en los alles op wat hij meldt.

Debug en review (zonder AI)

- **R1.1** Je krijgt een pagina met tien fouten uit het foutenkatern. Vind ze alle tien, benoem per fout wát er mis is en waaróm het een probleem is (ook als de pagina "werkt"), en verbeter ze. Gebruik de Elements-tab om je diagnoses te staven.
- **R1.2** Je krijgt twee versies van dezelfde pagina: identiek gerenderd resultaat, verschillende markup. Beargumenteer in vijf zinnen welke versie beter is, met de vier lezersgroepen uit 1.8 als toetssteen.

Bouwen en verantwoorden (met AI of zonder AI, jouw keuze, mét verantwoording)

- **B1.1** De pagina's die je in labo 1 online zet, zijn versie één van je persoonlijke site: `index.html` met het grondplan uit 1.8 (header met navigatie, main, footer), een voorstelling van jezelf, lijsten, minstens één afbeelding met goede alt en één externe link. Deze site groeit elke week mee met het vak; dit is je fundament, dus bouw hem zoals iemand die weet dat er twaalf verdiepingen bovenop komen. Labo 1 is zonder AI. Kies je daarna, bij het uitbreiden van je site, voor de AI-route, lever dan naast de site ook in: je prompt, de onbewerkte eerste output, en een lijst van wat je verbeterde en waarom - beoordeeld wordt de kwaliteit van je kritiek.

Verder lezen

- MDN, *How the web works*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works
- MDN, *Getting started with HTML*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax
- MDN, *HTML text fundamentals*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Headings_and_paragraphs
- MDN, *Document and website structure*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Structuring_documents
- MDN, *Images in HTML*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/HTML_images

Hoofdstuk 2 - CSS: het geordende conflict

In een notendop

CSS is de taal die bepaalt hoe de boom van hoofdstuk 1 eruitziet. Een stylesheet bestaat uit regels: een selector die elementen aanwijst en declaraties die zeggen wat er met die elementen gebeurt. Omdat meerdere regels voortdurend hetzelfde element willen stylen, en omdat de browser zelf ook al regels meebrengt, is stijl een geordend conflict. Dit hoofdstuk leert je de regels van dat conflict: hoe je elementen raak selecteert, hoe stijlen langs de boom naar beneden erven, wie wint als regels botsen (herkomst, dan specificiteit, dan volgorde), waarin het web meet, en hoe je met custom properties je eerste designtokens zet. Aan het einde lees je in DevTools de uitslag van elk conflict af in plaats van te gokken.

Wat je na dit hoofdstuk kan

Vorige week bouwde je een boom van betekenisvolle elementen en zette je hem online. Deze week leer je de taal waarmee je die boom vormgeeft. Aan het einde van dit hoofdstuk kan je stijlregels schrijven en aan je pagina's koppelen, elementen raak selecteren (van "elke paragraaf" tot "alleen de links in de navigatie") en, belangrijker dan beide, **voorspellen wat er gebeurt wanneer meerdere regels hetzelfde element willen stylen**. Je zet ook je eerste designtokens op: de centrale waarden waaruit je vormgeving vanaf nu wordt opgebouwd.

Dat voorspellen is de echte leerstof. CSS-syntax leer je in een kwartier. Wat beginners maandenlang doet vloeken is iets anders: een regel die "het niet doet". Vrijwel altijd doet die regel het wél; hij verliest alleen een conflict van een andere regel, volgens een rangorde die je nog niet kende. Dit hoofdstuk maakt die rangorde expliciet. Wie het conflictmodel begrijpt, ziet in seconden wat anderen met proberen-en-kijken te lijf gaan. Wie het overslaat, grijpt naar steeds grovere middelen, en hoe dat afloopt lees je in het foutenkatern.

Het hoofdstuk heeft drie delen. Eerst de taal zelf (2.1 tot 2.3): wat CSS is, hoe een regel eruitziet en hoe je een stylesheet aan je pagina's koppelt. Dan het aanwijzen en het conflict (2.4 tot 2.7): selectors in twee stappen, overerving, en de cascade als beslisladder. Ten slotte het werkmateriaal (2.8 tot 2.10): eenheden, custom properties, en het paneel in DevTools waarin je elk conflict afleest. In labo 2 komt daar de eerste **nabouw** bij: een screenshot en een HTML-bestand dat je niet mag wijzigen, en jij schrijft de CSS. Dat is exact de vorm van de toets en van het CSS-deel van het examen.

2.1 Wat CSS is, en hoe ver het reikt

CSS (Cascading Style Sheets) is de taal die beschrijft hoe HTML-elementen getoond worden: lettertypes, kleuren, witruimte, afmetingen, layout, tot en met animatie. Het is een eigen taal met een eigen syntax, geen HTML met extraatjes. De taakverdeling uit hoofdstuk 1 blijft daarbij overeind: HTML draagt betekenis en structuur, CSS draagt het volledige uitzicht. In je HTML staat nergens hoe iets eruitziet; in je CSS staat nergens wat iets betekent.

Hoe ver die scheiding reikt, bewijst een klassieker die je gezien moet hebben: **CSS Zen Garden** (csszengarden.com). Honderden designers gaven dezelfde, onaangeroerde HTML-pagina elk een compleet eigen design, van strak zakelijk tot barok geïllustreerd, louter door de stylesheet te vervangen. Zelfde boom, ander blad. Dat is ook de afspraak van dit blok: tegen de toetsweek krijg jij een kale HTML-structuur en een screenshot, en schrijf je zelf de CSS die het één in het ander verandert. In labo 2 doe je dat voor het eerst, in zijn kleinste maat.

Eén stylesheet kan bovendien een hele site bedienen: al je pagina's linken naar hetzelfde CSS-bestand, en één aanpassing daar verandert de vormgeving overal. Consistentie en efficiëntie in één beweging. Verderop in dit hoofdstuk drijven we dat met custom properties nog een niveau verder.

Nog wat context, omdat je er in documentatie en gegenereerde code voortdurend sporen van ziet. CSS ontstond in 1996 (de tijdlijn van 1.2), kende een moeizame jeugd tijdens de browseroorlog, en groeide via CSS2 en de modulaire "CSS3"-golf uit tot wat het nu is: een levende standaard die permanent evolueert, module per module. Versienummers zijn daarbij betekenisloos geworden; niemand schrijft "CSS3", men schrijft CSS. De vraag die er wél toe doet, stel je per feature: kan ik dit al gebruiken? Het antwoord vind je bij **Baseline**, de indicator die MDN en caniuse.com tonen. *Widely available* betekent zonder zorgen bruikbaar; *newly available* betekent bruikbaar in alle moderne browsers, maar recent. Alles wat deze syllabus als leerstof aanbiedt, is widely available; waar we vooruitblikken naar nieuwer werk, zeggen we dat erbij.

2.2 De anatomie van een stijlregel

Alle CSS die je ooit zal schrijven, bestaat uit variaties op deze ene vorm:

```
selector {
  property: waarde;
  andere-property: waarde;
}
```

De **selector** bepaalt op welke elementen de regel van toepassing is. Tussen de accolades volgt het **declaratieblok**: één of meer **declaraties**, elk een koppel van een **property** (eigenschap) en een **waarde**, gescheiden door een dubbelpunt en afgesloten met een puntkomma. Een concreet voorbeeld:

```css
h1 {
  color: darkslateblue;
  font-size: 2.5rem;
}

p {
  line-height: 1.6;
}
```

Lees dit als twee zinnen: "elke h1 in het document krijgt deze kleur en deze lettergrootte" en "elke p krijgt deze regelhoogte". Over de gebruikte waarden (wat is een rem? waarom heeft line-height geen eenheid?) gaat 2.8; eerst de vorm zelf.

Drie vormafspraken die in dit vak niet onderhandelbaar zijn. Elke declaratie krijgt een eigen regel en eindigt op een puntkomma, ook de laatste. Technisch mag die laatste puntkomma ontbreken, maar de declaratie die je er later onder typt, breekt dan stil. Declaraties springen één niveau in, en de sluitaccolade staat op een eigen regel. En je schrijft kleine letters, ook in selectors. Net als bij de insprongen in HTML is dit geen esthetiek: CSS geeft geen foutmeldingen (daarover meer in het motorkap-kader), dus leesbaarheid is je enige verdediging.

Commentaar schrijf je in CSS tussen `/*` en `*/`, op één regel of over meerdere. Je gebruikt het om delen van je stylesheet te benoemen en om tijdelijk declaraties uit te schakelen tijdens het experimenteren:

```css
/* === typography === */
p {
  line-height: 1.6;
  /* font-size: 1.2rem; -- disabled while testing */
}
```

Er bestaan op dit moment ruim driehonderdvijftig CSS-properties. Dat getal is er niet om je af te schrikken maar om je werkwijze te ijken: niemand kent ze allemaal, iedereen kent er een kern van enkele tientallen uit het hoofd en zoekt de rest op via MDN. Deze syllabus behandelt de kern; leren opzoeken hoort bij het vak.

2.3 CSS aan je pagina koppelen

CSS kan op drie manieren bij je HTML raken. Je moet ze alle drie herkennen, al gebruiken we er maar één.

De koninklijke weg is de **externe stylesheet**: een afzonderlijk bestand met de extensie `.css`, gekoppeld via een link-element in de head van elke pagina:

```
<!-- ...index.html, inside head... -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Mijn kookblog</title>
<link rel="stylesheet" href="css/stijl.css">
```

Het `href`-attribuut is, je herkent het uit hoofdstuk 1, gewoon een relatieve URL, met alle padregels en valkuilen van 1.9. In onze projecten krijgt CSS een eigen map `css/`, naast de map `afbeeldingen/`. Een pagina in een submap linkt dus met `../css/stijl.css`. Alle pagina's van je site linken naar hetzelfde bestand; zo koop je de site-brede consistentie uit 2.1. Dit is de enige methode die we in dit vak gebruiken.

De tweede vorm is de **ingesloten stylesheet**: CSS binnen een style-element in de head. Het werkt, maar de stijlen gelden dan enkel voor die ene pagina, en structuur en vormgeving zitten weer in één bestand verstrengeld. Je komt het tegen in voorbeelden en snelle prototypes; in projecten vermijden we het.

De derde vorm moet je vooral herkennen om ze te weigeren: **inline styles**, vormgeving in een style-attribuut rechtstreeks op een element.

Niet zo:

```
<h1 style="color: red; margin-top: 2em">Een rode hoofding</h1>
```

Zo wel:

```
<h1>Een rode hoofding</h1>
```

```
h1 {
  color: red;
  margin-top: 2em;
}
```

Inline styles vermengen niet alleen de lagen die we net gescheiden hebben. Ze krijgen in het conflict van dit hoofdstuk ook een bijna onverslaanbaar gewicht, waardoor één achteloos style-attribuut je stylesheet blijvend saboteert. Je zal ze in gegenereerde code geregeld tegenkomen (zie het AI-kader). In jouw eigen werk komen ze niet voor, met één uitzondering die pas in hoofdstuk 11 opduikt en daar haar eigen verhaal krijgt.

2.4 Selectors, deel 1: element, class en id

De selector is de helft van elke stijlregel, en raak selecteren is een vak apart. We bouwen het gereedschap in twee stappen op. Deze sectie geeft je de drie basisvormen plus twee hulpmiddelen.

De **elementselector** ken je al uit de voorbeelden: de naam van een HTML-element, en de regel geldt voor elk exemplaar in het document. Goed voor de grote lijnen ("alle paragrafen krijgen deze regelhoogte"), te grof voor detailwerk.

Voor dat detailwerk schreef je in hoofdstuk 1 al de haakjes: class en id. De **class-selector** begint met een punt en selecteert elk element dat die class draagt; de **id-selector** begint met een hekje en selecteert het ene element met dat id:

```
<p class="intro">De openingsparagraaf.</p>
<p>Een gewone paragraaf.</p>
<p id="disclaimer">Eenmalige juridische tekst.</p>
```

```
.intro {
  font-size: 1.25rem;
}

#disclaimer {
  color: dimgray;
}
```

Je kan element en class ook combineren: `p.intro` selecteert alleen p-elementen met de class intro. Dat is strenger dan `.intro`, dat elk element met die class pakt, welk element het ook is. In de praktijk zal je veruit het vaakst met kale class-selectors werken. Waarom niet met id-selectors, terwijl die toch preciezer ogen? Omdat een id per pagina uniek is (herbruikbaarheid nul), én omdat de id-selector in het conflict van 2.7 een onevenredig zwaar gewicht meedraagt dat je stylesheet stroef maakt. **Vuistregel:** style met classes; bewaar id's voor JavaScript en voor interne links. De volle verantwoording volgt bij de specificiteit.

Twee hulpmiddelen sluiten de basisset af. Met een **gegroepeerde selector** pas je één declaratieblok toe op meerdere selectors tegelijk, gescheiden door komma's. Onmisbaar om herhaling uit je stylesheet te houden:

```
h1, h2, h3 {
  font-family: Georgia, serif;
}
```

En de **universele selector** `*` selecteert elk element. Je gebruikt hem spaarzaam en gericht; zijn belangrijkste optreden komt in hoofdstuk 3, bij de reset waarmee onze projecten starten.

2.5 Selectors, deel 2: combinators en pseudo-classes

Het echte werk begint wanneer je selectors combineert met de structuur van je boom. Hier betaalt hoofdstuk 1 zich uit: elke combinator hieronder is een uitspraak over posities in de DOM, en wie de boom voor zich ziet, leest deze selectors als gewone zinnen.

De **afstammelingcombinator** is een spatie tussen twee selectors: `nav a` selecteert elke a die ergens binnen een nav zit, hoe diep ook: kind, kleinkind, dieper. Dit wordt je dagelijkse werkpaard, want het spreekt de taal van je semantische structuur: "links in de navigatie", "paragrafen in de footer", "afbeeldingen in main". Merk op hoeveel classes je hiermee níét nodig hebt. Wie zijn HTML semantisch bouwde, kan met structuur selecteren in plaats van overal etiketten te plakken.

```
nav a {
  text-decoration: none;
}

footer p {
  font-size: 0.875rem;
}
```

De **kindcombinator** `>` is de strenge broer: `ul > li` selecteert alleen li-elementen die een *direct* kind zijn van een ul, geen diepere afstammelingen. Het verschil zie je precies bij geneste structuren zoals de lijsten uit 1.7: `ul li` raakt ook de items van de binnenlijst, `ul > li` alleen die van het buitenste niveau (gegeven een selector die de juiste ul aanwijst). Verder zijn er nog twee broercombinators die je moet herkennen. `h2 + p` selecteert een p die *onmiddellijk* na een h2 komt: dezelfde ouder, direct ernaast, handig voor een inleidende paragraaf. `h2 ~ p` selecteert elke p die ergens na een h2 volgt binnen dezelfde ouder. Die laatste twee gebruik je zelden; de eerste twee voortdurend.

Pseudo-classes ten slotte selecteren op *toestand of positie* in plaats van op naam: kenmerken die niet in je markup staan maar in de situatie. Ze beginnen met een dubbelpunt. De bekendste is `:hover`, het element waar de muis nu overheen beweegt. Samen met `:focus`, het element dat via toetsenbord of klik de focus heeft, geeft hij je pagina's hun eerste interactiviteit, volledig zonder JavaScript:

```
nav a:hover,
nav a:focus {
  color: crimson;
}
```

Dat `:focus` er standaard bij hoort, is een toegankelijkheidszaak: wie met het toetsenbord door je pagina tabt, moet exact dezelfde visuele feedback krijgen als wie een muis beweegt. Vergeet je focus, dan navigeert een deel van je bezoekers blind. In hoofdstuk 9 komen we erop terug, wanneer we focus-stijlen gaan ontwerpen in plaats van ze te wissen.

Naast toestand is er positie. `:first-child` en `:last-child` selecteren een element dat het eerste of laatste kind van zijn ouder is, en `:nth-child()` doet dat op formule: `li:nth-child(odd)` pakt de oneven lijstitems (zo maak je een zebrapatroon), `li:nth-child(3)` het derde. Er bestaan er veel meer; deze vier volstaan dit semester ruim, en je herkent voortaan elke dubbelpunt in een selector als "hier wordt op toestand of positie geselecteerd". Eén nieuwkomer verdient een vooruitblik: `:has()`, waarmee je sinds kort een ouder kan selecteren op basis van zijn kinderen ("elke figure die een figcaption bevat"). Jarenlang de meest gevraagde feature van CSS, nu widely available; in hoofdstuk 5 nemen we hem in gebruik.

Dezelfde selectors, geneste spelling

Er is nog een tweede manier om deze combinaties op te schrijven, en je moet haar kunnen lézen, want gegenereerde en professionele CSS staat er vol mee. CSS kent sinds kort **nesting**: regels mogen in elkaar genest worden, waarbij de binnenregel automatisch aan de buitenselector wordt vastgeknoopt. Deze twee fragmenten betekenen exact hetzelfde:

```
/* flat — how this syllabus writes */
nav a { text-decoration: none; }
nav a:hover { color: crimson; }

/* nested — what you will often read */
nav {
  a {
    text-decoration: none;
    &:hover { color: crimson; }
  }
}
```

De `&` staat voor "de selector waar ik in genest zit": `&:hover` binnen `a` betekent `a:hover`. Zonder `&` geldt de afstammelingcombinator: `a` binnen `nav` betekent `nav a`. Meer is het niet. Nesting voegt geen enkele nieuwe selecteerkracht toe; het is dezelfde boomlogica in een andere spelling. Dat je haar zo vaak zal tegenkomen, heeft een geschiedenis: preprocessors als Sass boden dit al bijna twintig jaar aan voordat het platform het overnam, dus het idioom zit diep in de trainingsdata van elk model en in de vingers van elke senior.

Waarom schrijft deze syllabus dan plat? Omdat platte selectors je dwingen om bij elke regel de volledige selector uit te schrijven en dus te doorzien, precies de spier die dit hoofdstuk traint. En omdat diep geneste CSS hetzelfde probleem krijgt als diep geneste HTML: je verliest uit het oog welke selector er werkelijk staat, en daarmee welke specificiteit hij draagt. Herken de spelling, vertaal haar in je hoofd terug naar de platte vorm, en kies zelf pas voor nesting wanneer die vertaling een reflex is geworden.

2.6 Overerving

Voor het conflict losbarst, eerst het mechanisme dat conflicten juist voorkomt. Zet één declaratie op body en kijk wat er gebeurt:

```
body {
  color: #333333;
  font-family: Verdana, sans-serif;
}
```

Elke hoofding, paragraaf, lijst en link op de pagina neemt deze kleur en dit lettertype aan, zonder dat je er één selector extra voor schreef. Dat is **overerving** (inheritance): sommige properties stromen langs de takken van de DOM-boom naar beneden, van ouder naar kind naar kleinkind, tot een element

ze zelf overschrijft. De boom uit hoofdstuk 1 is dus niet alleen een structuurmodel; het is de infrastructuur waarlangs stijlen reizen.

Niet alles erft, en de logica erachter is goed te onthouden. **Tekstproperties erven**: kleur, lettertype, lettergrootte, regelhoogte, tekstuitlijning. Het zou ook absurd zijn om elk woord apart een lettertype te moeten geven. **Doosproperties erven niet**: randen, marges, padding, achtergronden, afmetingen. Stel je voor dat elk element de rand van zijn ouder herhaalde. Twijfel je bij een property, dan zegt MDN het onderaan elke referentiepagina (*Inherited: yes/no*).

Overerving is gereedschap, geen toeval: je zet de typografische basis van je hele site op body (of straks op :root), en werkt daarna alleen nog de uitzonderingen bij. Zo blijft je stylesheet klein en je design consistent; de stijl van een element wijzig je op één plek, niet op twintig. Geërfde waarden zijn in het conflict van de volgende sectie overigens de zwakst mogelijke partij: élke regel die het element rechtstreeks selecteert, wint van een geërfde waarde. Ook dat is logisch. Wat je expliciet zegt, verslaat wat er kwam aanwaaien.

2.7 De cascade: wie wint en waarom

Nu het kernmodel zelf. De C van CSS staat voor *cascading* (letterlijk: als een waterval), en de **cascade** is het beslissingsmechanisme dat in werking treedt zodra meerdere declaraties dezelfde property van hetzelfde element willen zetten. Dat gebeurt voortdurend; het is de normale toestand, geen uitzondering. Jouw regel concurreert minstens al met de standaardstijlen van de browser zelf. Dat een h1 groot en vet oogt en een link blauw en onderstreept, komt uit de **user agent stylesheet**, de ingebouwde stylesheet die elke browser op elk document toepast. Elke pagina die jij stylet, is dus al gestyled voor jij begint; jouw stylesheet is per definitie een verzameling overrides.

De cascade beslist elk conflict in drie stappen, in deze volgorde. Zodra een stap een winnaar aanwijst, stopt het spel; de volgende stappen dienen alleen om gelijke standen te breken.

Stap 1: herkomst. Stijlen van de auteur van de pagina (jij) verslaan de standaardstijlen van de browser. Daarom werkt je stylesheet überhaupt. Het volledige plaatje kent meer herkomsten (gebruikersstijlen, en het `!important`-keyword dat de orde omkeert), maar dat parkeren we tot onderaan deze sectie.

Stap 2: specificiteit. Binnen jouw eigen stylesheet wint de meest *specifieke* selector. De rangorde ken je impliciet al uit 2.4: een id-selector weegt zwaarder dan elke class, een class of pseudo-class weegt zwaarder dan elke elementselector, en bij samengestelde selectors tel je per categorie op. Vergelijk categorie per categorie, van zwaar naar licht, als een getal met drie posities:

```
p { color: orange; }              /* 0 ids, 0 classes, 1 element  */
.intro { color: green; }          /* 0 ids, 1 class,   0 elements */
main p.intro { color: purple; }   /* 0 ids, 1 class,   2 elements */
#disclaimer { color: dimgray; }   /* 1 id,  0 classes, 0 elements */
```

Wat gebeurt hier? Een p met class intro binnen main wordt paars: de derde selector verslaat de tweede op elementen (gelijk op classes), en beide verslaan de eerste op classes. Draagt het element ook het id disclaimer, dan wint dimgray: één id verslaat élke stapel classes en elementen, hoe hoog ook. Precies daarom luidde de vuistregel van 2.4 wat ze luidde. Wie met id's stylet, bouwt regels die alleen nog met zwaarder geschut te overschrijven zijn, en dat geschut escaleert (het foutenkatern toont hoe). Wie vlak stylet, met classes en elementen, houdt een stylesheet waarin alles nog overal overschreven kan worden door gewoon iets specifieker te zijn, of later.

Stap 3: volgorde. Bij exact gelijke specificiteit wint de regel die het laatst komt in de bron. "De laatste wint" klinkt als een detail, maar het is een instrument dat we bewust gaan bespelen. Het is waarom je overrides ná de basisregels schrijft, en het is het hele draagvlak van responsive design in hoofdstuk 6, waar media queries onderaan de stylesheet met gelijke specificiteit de regels erboven overschrijven op smallere of bredere schermen.

Dat is het hele model: herkomst, dan specificiteit, dan volgorde. Geërfde waarden doen niet eens mee zodra íéts het element rechtstreeks selecteert (2.6). Wanneer een stijl "niet werkt", loop je voortaan deze ladder af in plaats van te gokken, en 2.10 geeft je het instrument dat de hele afweging zichtbaar maakt.

Rest de valse vriend: **!important**. Een declaratie met dat achtervoegsel springt boven de normale herkomst-orde uit en verslaat alles wat we net opbouwden. Het voelt als een oplossing en het is bijna altijd het begin van het probleem: de enige remedie tegen een !important is een nog specifiekere !important, en zo escaleert een stylesheet tot niemand hem nog durft aan te raken. In dit vak gebruik je het niet. Kom je het tegen, in gegenereerde code bijvoorbeeld (zie het AI-kader), dan lees je het als een symptoom: hier heeft iemand een specificiteitsconflict niet begrepen en het dichtgeslagen in plaats van opgelost.

2.8 Meeteenheden: waarin meet het web?

Veel CSS-waarden zijn maten (lettergroottes, breedtes, afstanden), en CSS biedt een tiental eenheden om ze in uit te drukken. De keuze is geen smaakkwestie: ze bepaalt hoe je pagina zich gedraagt wanneer de omstandigheden veranderen. En op het web veranderen de omstandigheden altijd: andere schermen, andere vensters, en gebruikers die hun standaard lettergrootte aanpassen omdat ze slechter zien dan jij.

Eerst het vaste vertrekpunt. Elke browser heeft een standaard lettergrootte, vrijwel altijd **16 pixels**, en de gebruiker kan die in de browserinstellingen wijzigen. Dat gegeven is het anker van alles wat volgt.

De **pixel** (px) is de absolute eenheid: 1px is een vast, klein streepje, onafhankelijk van context. Absoluut klinkt betrouwbaar, maar het snijdt aan twee kanten. Een lettergrootte in px negeert de voorkeursinstelling van de gebruiker: wie 20px als standaard instelde omdat 16 te klein is, krijgt van jouw `font-size: 14px` toch 14. Voor tekst en voor alles wat met tekst moet meeschalen is px daarom de verkeerde keuze. Zijn nut behoudt hij voor het allerkleinste werk dat níét mag meeschalen: een rand van 1px, een subtiele schaduwverschuiving.

De **rem** (root em) is onze standaardeenheid: 1rem is de lettergrootte van het root-element, dus standaard 16px, maar hij volgt de instelling van de gebruiker, en dat is het punt. `font-size: 1.25rem` betekent "een kwart groter dan wat deze gebruiker leesbaar vindt", en dat is een fundamenteel andere uitspraak dan "20 pixels". Reken dus in verhoudingen: 1rem = 16, 1.5rem = 24, 0.875rem = 14. Even wennen, snel automatisch. Wie oudere cursussen of codebases kent, komt soms de truc tegen waarbij de root op 62,5% wordt gezet zodat 1rem = 10px "makkelijker rekent". We doen dat bewust niet. Het verplaatst het rekengemak naar een globale ingreep die elke externe code en elk voorbeeld op het web onbruikbaar maakt in jouw project. De 16-tafel leer je één keer.

De **em** is de relatieve broer met een ander referentiepunt: 1em is de lettergrootte van het element zelf (of, bij font-size, van de ouder). Dat maakt em waardevol voor wat moet meeschalen met zijn directe context, zoals padding ín een knop die mee groeit als de knoptekst groeit. Maar em is verraderlijk bij nesting, waar de waarden zich opstapelen: een lijst in een lijst in een lijst, elk op `font-size: 0.9em`, krimpt per niveau verder. **Vuistregel:** rem is je standaard; em alleen wanneer je uitdrukkelijk "relatief aan dit element hier" bedoelt.

Dan de eenheden die naar de omgeving kijken. **Percentages** zijn relatief aan het ouderelement: `width: 50%` is de helft van de breedte van de ouder. De **viewport-eenheden** vw en vh zijn procenten van het browservenster zelf: 1vw is een honderdste van de vensterbreedte, 1vh van de vensterhoogte. Deze eenheden maken layouts vloeibaar, ze ademen mee met het venster, en vanaf hoofdstuk 4 vormen ze samen met grid en flexbox het gereedschap van elke layout. Voor lettergroottes gebruik je vw nog niet: dat wordt pas veilig met de clamp()-functie van hoofdstuk 6, die er een onder- en bovengrens omheen legt.

Twee afspraken tot slot. Een eenheid plakt tegen zijn getal (`2rem`, nooit `2 rem`), maar de waarde nul schrijf je zonder eenheid (`margin: 0`). En één property wil bewust géén eenheid: `line-height: 1.6` betekent "1,6 keer de eigen lettergrootte" en schaalt daardoor overal correct mee. De eenheidloze vorm is hier de juiste.

En kleuren?

Kleuren zijn geen maten, maar je schrijft ze in dezelfde declaraties, dus hier de twee notaties die je deze week nodig hebt. De **kleurnaam** (`crimson`, `dimgray`, `darkslateblue`) is leesbaar en volstaat om te oefenen; er bestaan er een honderdvijftigtal, allemaal op MDN. De **hexadecimale notatie** (`#e2001a`) is wat je in elk echt project ziet: een hekje en zes tekens, twee voor rood, twee voor groen, twee voor blauw, elk van `00` (niets) tot `ff` (vol). `#000000` is zwart, `#ffffff` wit, `#e2001a` het rood van AP. Je hoeft die codes niet uit het hoofd te kennen; je plukt ze uit een design of uit de kleurkiezer van DevTools. In hoofdstuk 7 komen de moderne notaties met een kanaal voor doorzichtigheid en een beter kleurbereik; appendix B zet ze allemaal naast elkaar.

2.9 Custom properties: je eerste designtokens

Kijk naar een stylesheet van een gemiddelde site en je ziet dezelfde waarden tientallen keren terugkeren: het merkblauw, de standaardmarge, het lettertype. Elke herhaling is een risico (één vergeten plek bij een wijziging) en een gemiste kans (de samenhang van het design staat nergens

uitgesproken). CSS heeft daar een ingebouwde oplossing voor die we vanaf dag één gebruiken: **custom properties**, in de volksmond CSS-variabelen.

Je declareert ze als gewone declaraties, met een naam die met twee streepjes begint, en je leest ze uit met de var()-functie. De declaratie zet je op de selector `:root`, een pseudo-class die het root-element aanwijst (in HTML-documenten: het html-element), zodat elke waarde via overerving (2.6) overal in de boom beschikbaar is.

Niet zo:

```
body { color: #1a1a2e; }
h2 { color: #e94560; }
a { color: #e94560; }
footer p { color: #1a1a2e; }
```

Zo wel:

```
/* style.css */
:root {
  --color-ink: #1a1a2e;
  --color-accent: #e94560;
  --font-body: Verdana, sans-serif;
  --space: 1.5rem;
}

body {
  color: var(--color-ink);
  font-family: var(--font-body);
}

h2 {
  color: var(--color-accent);
  margin-bottom: var(--space);
}
```

De directe winst is onderhoud: de accentkleur van je site verander je op één regel, en elke plek die ernaar verwijst volgt. Maar er zit een groter idee achter, en dat idee draagt in de industrie de naam **design tokens**: een design is geen verzameling losse waarden maar een klein aantal beslissingen (dit zijn onze kleuren, dit onze maten, dit onze lettertypes) die je op één plek benoemt en overal consequent toepast. Elk professioneel designsysteem is zo gebouwd, en de custom property is de vorm waarin zulke tokens in CSS leven. Vanaf nu begint elke stylesheet in dit vak met een tokenblok op :root. In hoofdstuk 7 groeit dat blok uit tot een volwaardig systeem met schalen voor typografie en witruimte. In hoofdstuk 13 zie je waarom een goed tokenblok ook het verschil maakt wanneer je een AI aan je code laat werken: het is de plek waar jouw designbeslissingen leesbaar zijn, voor collega's én voor machines.

Twee praktische afspraken. Naamgeving: kleine letters, koppeltekens, en benoem de *rol*, niet de waarde. `--color-accent` overleeft een rebranding, `--color-rood` wordt een leugen (dezelfde les als bij classnamen in 1.12). En var() aanvaardt een terugvalwaarde voor het geval een token niet bestaat

(`var(--color-accent, crimson)`); handig om te kennen, zelden nodig in eigen code.

2.10 Het conflict lezen in DevTools

Alles uit dit hoofdstuk komt samen in één paneel dat je vanaf nu dagelijks gebruikt. Selecteer in de Elements-tab een element, en het deelvenster **Styles** ernaast toont de volledige uitslag van het conflict voor dat element: elke regel die het element raakt, uit welk bestand en welke regel hij komt, in aflopende orde van winnaar naar verliezer. De user agent stylesheet van 2.7 staat er ook tussen, en geërfde stijlen staan apart gemarkeerd onder *Inherited from*.

Het belangrijkste signaal is de **doorstreping**: een doorstreepte declaratie is een verliezer, overschreven door een regel hoger in het paneel. Daarmee is de vraag "waarom doet mijn regel het niet" geen mysterie meer maar een leesoefening. Staat je regel er, doorstreept? Dan verloor hij het conflict. Kijk wie erboven staat en je weet van wie, en met de ladder van 2.7 weet je waarom. Staat je regel er helemaal niet? Dan selecteert je selector dit element niet (typfout, verkeerde structuuraanname: check de boom) of laadt je stylesheet niet (padfout, F2.1 in het katern). Dat zijn álle mogelijkheden; er bestaat geen derde.

Twee gereedschappen in dit paneel maak je je deze week nog eigen. Je kan declaraties live aan- en uitvinken en nieuwe waarden typen: experimenteren op de levende pagina, zonder je bestand aan te raken, en pas overnemen in je CSS wat bevalt. Klik op een kleurstaal en je krijgt een kleurkiezer; klik op een getal en de pijltjestoetsen tellen op en af. En het tabblad **Computed** toont per property de uiteindelijk berekende waarde: wat 1.25rem in pixels werd, welke kleur er na het hele conflict daadwerkelijk staat. Samen met de Elements-boom uit hoofdstuk 1 vormt dit paneel je waarheidsinstrument. De boom toont wat de browser hééft, Styles toont waarom het eruitziet zoals het eruitziet.

Onder de motorkap: hoe de browser één waarde bepaalt

Dit kader gaat dieper dan de theorieles. Je hebt het niet nodig om de oefeningen te maken, wel om te begrijpen waarom de cascade zich gedraagt zoals ze zich gedraagt.

In de theorieles vertellen we de cascade als een verhaal over regels die winnen en verliezen. De browser voert in werkelijkheid een strak algoritme uit, per element en per property, voor élke property die CSS kent, ook de honderden die jij nooit noemde. Dat volledige beeld verklaart een paar dingen die anders raadsels blijven.

Stap één: verzamelen. De browser verzamelt alle declaraties die het element raken, uit alle bronnen: de user agent stylesheet, eventuele gebruikersstijlen, en al jouw stylesheets samen. Meerdere gelinkte CSS-bestanden vormen dus één wedstrijd, geen aparte competities; de volgorde van je link-elementen is gewoon bronvolgorde.

Stap twee: sorteren. De verzamelde declaraties worden gesorteerd volgens de ladder uit 2.7: herkomst en belangrijkheid eerst, dan specificiteit, dan bronvolgorde. De volledige herkomst-orde, van zwak naar sterk: browserstijlen, gebruikersstijlen, auteursstijlen, en dan, in spiegelbeeld, auteurs-!important, gebruikers-!important, browser-!important. Die omkering is geen gril maar een machtsverdeling: een gebruiker die een vergroot lettertype nódig heeft, verslaat met zijn !important de auteur die het mooier vond van niet. !important is dus ten diepste een toegankelijkheidsmechanisme voor de gebruiker; auteurs die het inzetten, lenen gereedschap dat niet voor hen bedoeld was. Specificiteit is daarna exact het drietal uit 2.7: tel (ids, classes, elementen) en vergelijk positioneel. De universele selector telt als (0,0,0) en een samengesteld gevaarte als `main ul li a.actief` als (0,1,4). Sinds kort bestaat er ook een instrument dat boven specificiteit uit een eigen orde oplegt, @layer, waarmee grote codebases hun cascade in expliciete lagen indelen. Je hoeft het dit semester niet te gebruiken, wel te herkennen als je het tegenkomt.

Stap drie: invullen. Nu pas komt het deel dat vrijwel niemand kent en dat toch elke "waar komt die stijl vandaan?"-vraag beantwoordt. Voor de meeste properties van het element leverde stap twee namelijk níéts op: niemand declareerde iets. Voor elk van die gaten valt de browser terug op een vaste rangorde. Bestaat er een geërfde waarde (is de property overerfbaar en heeft de ouder een waarde), dan die; anders de **initial value**, de fabrieksinstelling die de CSS-specificatie voor elke property vastlegt. Overerving is in het echte algoritme dus geen concurrent van de cascade maar haar vangnet: het komt pas aan bod waar de cascade zweeg. Precies daarom verliest een geërfde waarde van élke declaratie die het element rechtstreeks raakt, hoe zwak ook. De zwakste `p { }`-regel verslaat de sterkste waarde op body.

Stap vier: doorrekenen. De winnende waarden zijn nog niet de eindwaarden: `1.25rem` moet pixels worden, procenten moeten worden uitgerekend tegen de ouder, kleurnamen worden getalswaarden. Dit doorrekenen gebeurt in fasen (computed, used, actual value in het jargon), en het Computed-tabblad van 2.10 toont je het resultaat. Hier zie je ook waarom geërfde relatieve waarden soms verrassen: kinderen erven doorgaans de dóórgerekende waarde, niet je oorspronkelijke notatie.

En als je een syntaxfout schreef? CSS heeft, net als de HTML-parser uit hoofdstuk 1, geen foutmeldingen. Het herstelgedrag is wel anders, en strenger. Een declaratie die de browser niet begrijpt (typfout in de property, onmogelijke waarde) wordt in zijn geheel genegeerd, stil, terwijl de rest van het blok gewoon doorwerkt. Eén vergeten puntkomma is venijniger: de browser plakt de volgende declaratie aan de lopende vast en verwerpt het onbegrijpelijke geheel, waardoor één typfout twéé regels dooft. Ook hier faalt fout werk dus stil, en ook hier is DevTools je detector: genegeerde declaraties krijgen in het Styles-paneel een waarschuwingsteken en een doorstreping. Wie na elke werksessie door dat paneel loopt, vangt zijn typfouten op de dag zelf.

Veelgemaakte fouten

De fouten van deze week hebben één ding gemeen met die van vorige week: de browser meldt ze niet. Het Styles-paneel wel. De feedback op labo 2 verwijst naar deze nummers.

F2.1 - De stylesheet laadt niet. *Symptoom*: geen enkele stijl werkt; de pagina oogt als kaal HTML. *Oorzaak*: vrijwel altijd het href-pad in het link-element (F1.7 in nieuwe kleren), soms een typfout in `rel="stylesheet"`. *Remedie*: kijk of je bestand in het Styles-paneel überhaupt opduikt; wandel het pad na vanaf het HTML-bestand, en denk aan `../` voor pagina's in een submap.

F2.2 - Typfout in property of waarde. *Symptoom*: één declaratie doet niets, de rest werkt. *Oorzaak*: `colour`, `font-weigth`, een waarde die niet bestaat. CSS meldt fouten niet (zie motorkap). *Remedie*: zoek het waarschuwingsteken in het Styles-paneel; het staat er al voor je.

F2.3 - De vergeten puntkomma. *Symptoom*: twéé declaraties doen niets. *Oorzaak*: de puntkomma tussen beide ontbreekt; de browser verwierp het samengeplakte geheel. *Remedie*: als een declaratie dooft, controleer altijd ook de regel erbóven.

F2.4 - De selector raakt het element niet. *Symptoom*: je regel verschijnt nergens in het Styles-paneel van het element. *Oorzaak*: de punt vergeten bij een class (`intro` zoekt een element met de naam intro, en dat bestaat niet), hekje en punt verwisseld, of een structuuraanname die niet klopt (`nav > a` terwijl de links in li's zitten). *Remedie*: check de boom in Elements en lees je selector hardop als zin.

F2.5 - Specificiteitsverlies niet herkend. *Symptoom*: "mijn regel doet het niet", maar hij staat doorstreept in het paneel. *Oorzaak*: een specifiekere regel wint, vaak een eerder geschreven `#id` - of samengestelde selector die je vergat. *Remedie*: lees wie erboven staat; maak je selector gelijkwaardig of vereenvoudig de winnaar. Niet: zwaarder geschut erbij.

F2.6 - De !important-spiraal. *Symptoom*: een stylesheet vol !important die niemand nog begrijpt. *Oorzaak*: F2.5 telkens "opgelost" met het botte mes. *Remedie*: verwijder ze en los de onderliggende conflicten op met de ladder van 2.7. !important is een symptoom, geen stijlmiddel.

F2.7 - Styling op id's. *Symptoom*: overrides vragen steeds langere selectors. *Oorzaak*: id-selectors in de stylesheet leggen een specificiteitsvloer waar alles overheen moet. *Remedie*: de vuistregel van 2.4: classes voor stijl, id's voor JavaScript en interne links.

F2.8 - Spatie voor de eenheid. *Symptoom*: declaratie genegeerd. *Oorzaak*: `2 rem` met spatie is ongeldig; `margin: 0px` mag wel, maar hoeft niet. *Remedie*: eenheid plakt aan het getal; nul schrijf je kaal.

F2.9 - Tekst in pixels. *Symptoom*: niets zichtbaar, tot een gebruiker met aangepaste lettergrootte je site bezoekt en niets meeschaalt. *Oorzaak*: font-size in px negeert de gebruikersinstelling. *Remedie*: rem voor alles wat tekst is of met tekst meeschaalt; px alleen voor randen en pixelwerk.

F2.10 - Herhaalde letterlijke waarden. *Symptoom*: dezelfde kleurcode op veertien plaatsen; een rebranding wordt zoekwerk. *Oorzaak*: waarden hardcoded in plaats van benoemd. *Remedie*: het tokenblok van 2.9. Elke waarde die twee keer voorkomt, verdient een naam op :root.

F2.11 - Hover zonder focus. *Symptoom*: geen zichtbaar symptoom met de muis; wie met de tab-toets navigeert, ziet niet waar hij staat. *Oorzaak*: alleen `:hover` gestyled. *Remedie*: elke hover-regel krijgt `:focus` als tweede selector in de groep (2.5).

Met AI aan je zij: gegenereerde CSS lezen

Vraag een model om styling en je krijgt werkende CSS, met een paar voorspelbare kwalen die je na dit hoofdstuk zonder moeite herkent.

Pixel-soep. Gegenereerde stylesheets meten graag alles in px, lettergroottes incluis: het gemiddelde van twintig jaar trainingsdata. Jij weet inmiddels waarom dat de gebruikersinstelling breekt (2.8, F2.9). Vraag om rem, of reken zelf om. En let op: het model rekent braaf 16px = 1rem, maar blijft de schaallogica erachter vaak schuldig.

Overspecifieke selectors. Modellen produceren graag kettingen als `div.container ul.menu li a.link` waar `.menu a` volstond. Elke overbodige schakel verhoogt de specificiteit en maakt latere overrides zwaarder: de vloer van F2.7, maar dan gegoten in elke regel. Controlevraag per selector: welk deel doet echt werk?

!important als pleister. Wanneer gegenereerde code een bestaande stylesheet moet overschrijven, grijpt ze opvallend snel naar !important. Het model "ziet" jouw cascade niet en slaat het conflict dicht in plaats van het te winnen. Elke !important in gegenereerde output verdient een moment wantrouwen: welk conflict wordt hier ontweken, en wat is de nette winst via specificiteit of volgorde?

Inline styles en verweesde waarden. Vooral bij kleine snippets stopt een model stijlen graag in style-attributen (2.3) en strooit het letterlijke waarden waar jouw tokenblok al namen heeft. Een model dat je :root-tokens te zien krijgt, gebruikt ze overigens meestal keurig. Dit is de eerste concrete plek waar je merkt dat een goed systeem AI-output beter maakt; dat idee groeit mee tot en met hoofdstuk 13.

Geneste spelling. Veel gegenereerde CSS komt in de geneste vorm van 2.5, vaak diep genest, met lange `&`-kettingen. Dat is geen fout, maar wel een leesopdracht: vertaal per regel terug naar de platte selector en beoordeel dán specificiteit en bedoeling. Wie alleen de platte vorm kent, leest de helft van moderne output niet.

De oefening die dit alles samenbrengt, staat hieronder bij R2.3: een gegenereerde stylesheet reviewen met dit hoofdstuk als meetlat.

Oefeningen

Labo 2 bevat de opgaven van deze week, met startbestanden, de eerste nabouw van een screenshot en de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen.

Drills (zonder AI aanbevolen)

- **D2.1** Gegeven een HTML-pagina en tien beschrijvingen in woorden ("alle links in de footer", "de eerste paragraaf na elke h2", "elk oneven lijstitem", ...): schrijf de tien selectors. Daarna omgekeerd: lees tien gegeven selectors hardop als zin.
- **D2.2** Koppel één externe stylesheet aan alle pagina's van je site, met correcte relatieve paden vanuit álle pagina's. Controleer in het Styles-paneel dat het bestand op elke pagina laadt.

- **D2.3** Wie wint? Op papier, zonder browser: tien conflictsituaties (gegeven HTML-fragment plus twee of drie concurrerende regels), voorspel telkens de uitkomst en benoem de beslissende stap van de ladder.
- **D2.4** Zet een tokenblok op :root met minstens twee kleuren, één lettertype en één maat; herschrijf een gegeven stylesheet zodat elke herhaalde waarde via var() loopt.
- **D2.5** Style de navigatie van je site: geen onderstreping, eigen kleur, en een zichtbaar :hover- én :focus-effect. Test de focus met de tab-toets.
- **D2.6** Rem-gymnastiek: vertaal tien px-waarden naar rem en terug (16-basis), uit het hoofd, tot het geen rekenwerk meer is.

Debug en review (zonder AI)

- **R2.1** Een pagina met stylesheet waarin acht fouten uit het onderdeel "veelgemaakte fouten" zitten (F2.1 tot en met F2.9): vind ze, benoem per fout de diagnose én het paneel-signaal waarmee je haar vond, en herstel.
- **R2.2** Drie regels vechten om één element; de huidige uitkomst is fout. Los het conflict drie keer op: één keer via specificiteit, één keer via volgorde, één keer door te vereenvoudigen. Beargumenteer welke oplossing de beste stylesheet achterlaat.
- **R2.3** (met AI) Je krijgt een door AI gegenereerde stylesheet bij een gegeven pagina. Review ze met dit hoofdstuk als meetlat: markeer pixel-soep, overspecifieke selectors, !important-gebruik en verweesde waarden; herschrijf de stylesheet naar dezelfde visuele uitkomst met classes, rem en tokens.

Bouwen en verantwoorden (met AI of zonder AI)

- **B2.1** Versie twee van je persoonlijke site: één externe stylesheet voor alle pagina's, een tokenblok op :root, typografische basis via overerving op body, gestylede navigatie met hover en focus, en nergens een id-selector, !important of inline style.

Verder lezen

- MDN, *Basic CSS selectors*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Basic_selectors
- MDN, *Handling conflicts (cascade, specificity, inheritance)*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Handling_conflicts
- MDN, *CSS values and units*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Values_and_units
- MDN, *Using CSS custom properties*: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- MDN, *Baseline*: https://developer.mozilla.org/en-US/docs/Glossary/Baseline/Compatibility

Hoofdstuk 3 - Alles is een box

In een notendop

Voor de browser is elk element een rechthoek: een **box** met vier lagen, van binnen naar buiten content, padding, border en margin. Layout is niets anders dan boxen maat geven en tegen elkaar schikken. Dit hoofdstuk leert je de anatomie van die box, de rekensom die bepaalt hoe breed en hoe hoog ze wordt, de projectreset waarmee elk project vanaf nu begint, en de properties waarmee je stuurt: padding, border, margin, width en height, display en overflow. Je leert ook waarin je een box meet (rem, procent, viewport) en hoe je in DevTools afleest wat de browser er uiteindelijk van rekende. De browser rekent, jij stuurt. Wie het boxmodel kent, voorspelt een layout in plaats van ze te proberen.

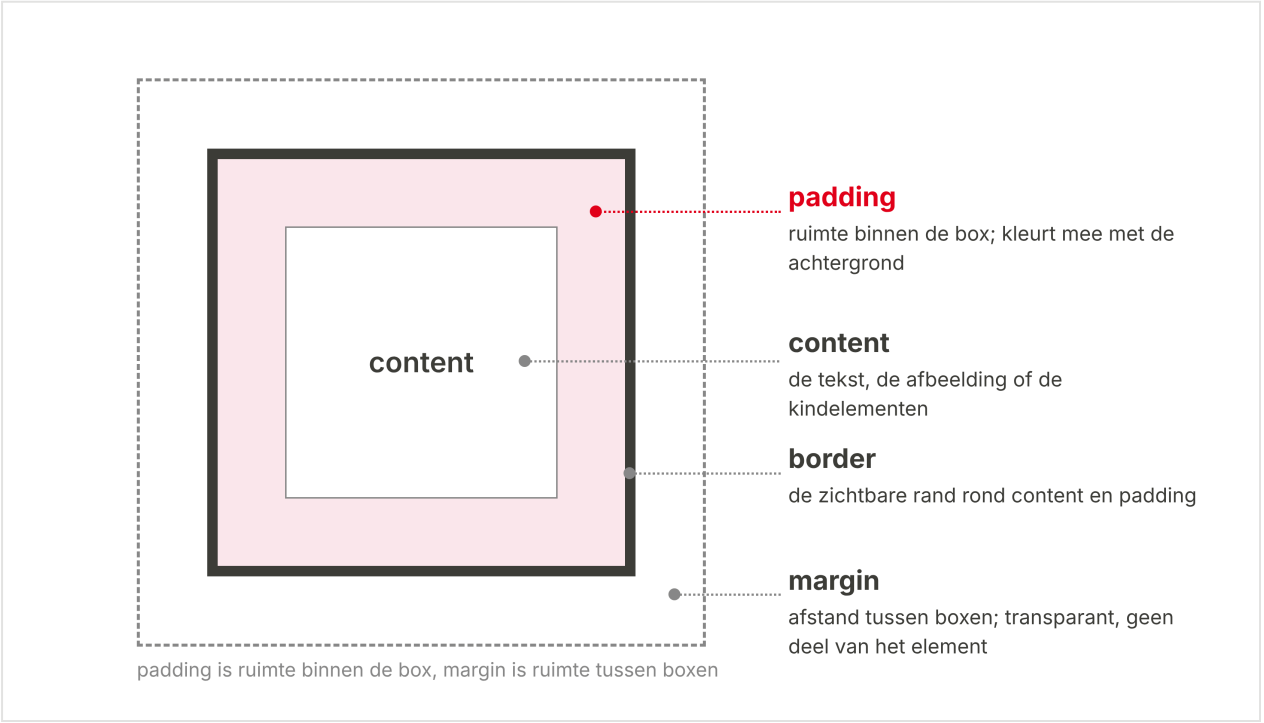
Wat je na dit hoofdstuk kan

Open een willekeurige website, open DevTools en beweeg met de inspector over de pagina: elk element licht op als een rechthoek. Koppen, paragrafen, links, afbeeldingen, hele secties. De browser ziet je pagina als een verzameling boxen, en layout is niets anders dan boxen maat geven en tegen elkaar schikken. Dit hoofdstuk leert je de anatomie van die box en de properties waarmee je haar stuurt: padding, border, margin, afmetingen, en de display-property die bepaalt hoe boxen zich tegenover elkaar gedragen.

Na dit hoofdstuk kan je van elke box op een pagina voorspellen hoe breed en hoe hoog ze wordt en waarom, kan je ruimte bínnen en tússen elementen bewust vormgeven in plaats van "tot het er goed uitziet", en start elk van je projecten met een korte reset, vier regels, die de valkuilen dichtzet waar vorige generaties developers jaren op stukliepen. Daarmee staat het fundament waarop de volgende twee hoofdstukken hun layouts bouwen: grid en flexbox schikken uiteindelijk gewoon boxen, deze boxen.

Het hoofdstuk heeft drie delen. Eerst de box zelf (3.1 tot 3.3): haar vier lagen, haar maat en de reset. Dan de lagen één voor één (3.4 tot 3.6): padding, border en margin, met het beruchte ineenvallen van marges. Ten slotte het gedrag (3.7 tot 3.10): display, grenzen en overflow, afbeeldingen, en het boxmodel-diagram in DevTools. In labo 3 bouw je vijf screenshots na, van een kleurenbalk tot een landingspagina: vijf keer de toetsvorm, met alleen hoofdstuk 1 tot 3 als gereedschap.

3.1 De anatomie van de box



Figuur 3.1 - Het boxmodel: de content in het midden, daaromheen padding, border en margin. Padding kleurt mee met de achtergrond van het element, margin is transparant.

Elk element in de DOM-boom genereert een rechthoekige **box** (in het jargon de element box), en elke box is opgebouwd uit dezelfde vier lagen, van binnen naar buiten.

De **content** is de kern: de tekst, de afbeelding of de kindelementen die erin leven. Daaromheen ligt optioneel **padding**: ademruimte tussen de content en de rand van de box, die meekleurt met de achtergrond van het element. Dan volgt optioneel een **border**: de zichtbare rand die content en padding omsluit. En helemaal buitenaan ligt optioneel **margin**: afstand tussen deze box en de buitenwereld. Margin is transparant, want het is geen deel van het element maar afstand tót andere elementen.

Eén zin om het verschil tussen de twee ruimtesoorten voorgoed vast te zetten, want vrijwel elke beginnersverwarring in dit hoofdstuk komt hierop neer: **padding is ruimte binnen de box, margin is ruimte tussen boxen.** Krijgt het element een achtergrondkleur, dan kleurt de padding mee en de margin niet. Twijfel je bij een ontwerpbeslissing, stel dan de vraag: hoort deze ruimte bij het element (padding) of bij de schikking van elementen (margin)?

Dit geldt voor élk element, ook de inline elementen uit 1.7, al speelt het model daar beperkter (daarover in 3.7). En omdat elementen genest zijn, zijn boxen dat ook: de box van main bevat de boxen van zijn kinderen, en de content-laag van een ouder is het speelveld van zijn kinderen. De boom uit hoofdstuk 1 is dus ook een box-in-box-structuur; DevTools toont je beide gezichten van dezelfde werkelijkheid.

3.2 De maat van de box, en de rekensom die iedereen fout heeft

Zonder jouw tussenkomst rekent de browser zelf: een blokelement wordt zo *breed* als zijn ouder ruimte biedt, en zo *hoog* als zijn content nodig heeft. Onthoud dat asymmetrische standaardgedrag, **breedte komt van buiten, hoogte komt van binnen**, want het verklaart straks de helft van je layoutvragen. Een lege box heeft hoogte nul; een box met tekst groeit mee met elke regel.

Met **width** en **height** neem je het stuur over. En dan komt de historische valkuil. In het oorspronkelijke boxmodel slaan width en height alleen op de *content-laag*; padding en border komen er nog bovenop. Een box van `width: 30rem` met `padding: 2rem` en een border van 0.25rem is visueel dus geen 30 maar 34,5rem breed. Tel maar: border links, padding links, content, padding rechts, border rechts. Wie dat niet weet, ziet layouts uit hun voegen barsten zodra er padding bij komt.

```
main {
  width: 30rem;
  padding: 2rem;
  border: 0.25rem solid var(--color-ink);
  /* visual width: 0.25 + 2 + 30 + 2 + 0.25 = 34.5rem */
}
```

CSS heeft hiervoor een schakelaar: **box-sizing**. De standaardwaarde `content-box` geeft het historische gedrag hierboven. De waarde `border-box` maakt width en height tot wat iedereen intuïtief verwacht: de volledige visuele maat van de box, borders en padding inbegrepen; de content krimpt dan mee naar wat overblijft. Dezelfde declaraties met `box-sizing: border-box` leveren een box die exact 30rem breed is.

	`CONTENT-BOX` (STANDAARD)	`BORDER-BOX` (WIJ)
`width: 30rem` betekent	de content is 30rem	de hele box is 30rem
padding en border	komen er bovenop	zitten erin
visuele breedte van het voorbeeld	34,5rem	30rem

De hele moderne webwereld werkt met border-box, en wij ook: niet per element, maar in één klap voor alles, via de reset in de volgende sectie. De rekensom van het content-box-model moet je desondanks kunnen maken: op oudere codebases, in examenvragen en bij het lezen van andermans CSS kom je haar tegen, en F3.1 is er de klassieker van.

Waarin meet je een box: rem, procent en viewport

In 2.8 leerde je de eenheden kennen aan de hand van tekst. Op een box zie je pas goed waarom de keuze geen smaakkwestie is: elke eenheid geeft een ander antwoord op de vraag "relatief aan wat?".

Rem is ook voor boxen je standaard. Een box die in rem gemeten is, schaalt mee met de tekst erin: wie zijn letters groter zet, krijgt een breder kaartje, en de verhouding tussen letter en box blijft kloppen. `max-width: 40rem` is een leeskolom van zowat zeventig tekens, in elke lettergrootte. De 16-tafel uit 2.8 geldt onverkort: 20rem is 320px, 0.25rem is 4px.

Procenten zijn relatief aan de ouder, en wel aan de *content-laag* van de ouder. `width: 50%` is de helft van de breedte die de ouder biedt; twee boxen van 50% vullen samen de ouder, tenminste als er geen padding of border bovenop komt (3.2 opnieuw). Voor height werkt een procent pas als de ouder zelf een bepaalde hoogte heeft; honderd procent van "zo hoog als mijn inhoud" kan de browser niet berekenen, en dan gebeurt er niets (F3.2). En één valkuil die je moet kennen omdat ze zo onlogisch oogt: **padding en margin in procenten zijn altijd relatief aan de breedte van de ouder**, ook padding-top en margin-bottom. `padding-top: 40%` is dus veertig procent van de *breedte*. Dat is geen bug maar een ontwerpkeuze (de hoogte is meestal nog niet bekend wanneer de browser de padding uitrekent), en het verklaart waarom `margin: 15%` rondom een vierkant netjes een kleiner vierkant oplevert.

De **viewport-eenheden** vw en vh zijn relatief aan het browservenster: 1vh is een honderdste van de vensterhoogte, 1vw van de vensterbreedte. Daarmee laat je een box aan het venster hangen in plaats van aan haar ouder: een band van `height: 50vh` vult de halve hoogte van het scherm, welk scherm ook; een hero van `min-height: 100vh` vult het eerste scherm volledig (min-height, geen height: de inhoud mag er altijd overheen groeien, zie 3.8). Twee kanttekeningen. `width: 100vw` rekent de verticale schuifbalk mee en geeft op Windows een horizontale schuifbalk cadeau; voor de volle breedte heb je niets nodig, want een blok neemt die al. En op telefoons schuift de adresbalk mee, waardoor 100vh soms nét te hoog is; hoofdstuk 6 geeft daar de moderne oplossing voor.

EENHEID	RELATIEF AAN	OP EEN BOX TYPISCH VOOR	LET OP
`rem`	lettergrootte van het root-element	breedtes, padding, marges, radius: je standaard	reken in de 16-tafel
`%`	de content-laag van de ouder	"een deel van mijn ouder": kolommen, afbeeldingen	hoogte alleen als de ouder er een heeft; padding en margin tegen de breedte
`vw`, `vh`	het venster	banden en hero's die aan het scherm hangen	100vw en de schuifbalk; 100vh en de adresbalk
`px`	niets	borders, een schaduwverschuiving	nooit voor tekst, zelden voor een box

Vuistregel: rem voor alles wat bij de inhoud hoort, procent voor "een deel van mijn ouder", viewport-eenheden alleen voor wat echt aan het venster hangt, px alleen voor randen. In labo 3 gebruik je alle vier, elk op zijn plaats.

3.3 De projectreset: waar elk project begint

Vanaf deze week begint elk project met twee stylesheets. De eerste heet `reset.css`, staat als eerste gelinkt in de head en raak je niet aan. De tweede is `stijl.css`, waarin je werkt en die met je tokenblok begint. We noemen die eerste de **projectreset**, en anders dan de naam doet vermoeden is het geen magie maar vier bewuste beslissingen die je regel per regel moet kunnen verantwoorden:

```css
/* reset.css */
*,
*::before,
*::after {
  box-sizing: border-box;
}

body, h1, h2, h3, h4, h5, h6,
p, figure, blockquote, dl, dd {
  margin: 0;
}

img, picture, video, canvas, svg {
  display: block;
  max-width: 100%;
}

input, button, textarea, select {
  font: inherit;
}
```

Regel één zet border-box op elk element, via de universele selector die in 2.4 werd aangekondigd (de ::before- en ::after-vermeldingen dekken pseudo-elementen af, gegenereerde mini-elementen die je in hoofdstuk 8 leert kennen; we nemen ze nu al mee zodat de reset compleet is). Merk op dat dit dankzij de specificiteitsladder een vriendelijke ingreep is: de universele selector weegt niets, dus élke regel die je zelf schrijft kan er moeiteloos van afwijken.

Regel twee neemt de marges in handen die de user agent stylesheet (2.7) uitdeelt: de acht pixels rond body die niemand ooit besteld heeft, en de marges boven en onder koppen, paragrafen, figuren en citaten. Vanaf nu is verticale witruimte een beslissing van jou, in één richting (3.6), en niet een erfenis van de browser die je per element moet wegwerken. Lijsten staan bewust niet in de lijst: een lijst houdt haar insprong en haar bolletjes tot jij beslist dat ze er anders moet uitzien, zoals bij een navigatie.

Regel drie temt media. `max-width: 100%` verhindert dat een grote afbeelding uit haar container barst: ze krimpt mee met de beschikbare ruimte, een regel die in hoofdstuk 6 de hoeksteen van responsive afbeeldingen blijkt. En `display: block` neemt afscheid van het inline-gedrag van img (1.7), dat onder elke afbeelding een raadselachtige reep witruimte veroorzaakt: inline elementen staan op een tekstregel, en onder een tekstregel hoort ruimte voor staartletters. Afbeeldingen zijn geen letters; wij behandelen ze als blokken. Video, canvas en svg krijgen hetzelfde regime.

Regel vier is een belofte aan hoofdstuk 9: formulierelementen erven standaard het lettertype van de pagina niet (een knop oogt anders dan de tekst ernaast), en `font: inherit` zet dat recht. Je hebt er deze weken niets aan, maar je wil het niet vergeten op de dag dat je je eerste formulier bouwt.

Even belangrijk is wat de reset **niet** doet. Links blijven blauw en onderstreept, strong blijft vet, koppen blijven groot, lijsten blijven lijsten, knoppen houden hun uitzicht. De reset neemt beslissingen weg die niemand bestelde; ze neemt geen beslissingen weg die betekenis dragen. Wat de browser nog uitdeelt, lees je in 3.10 af in het boxmodel-diagram en zet je per geval zelf: `padding-left: 0` en

`list-style: none` (de property die bolletjes en nummers regelt) op een lijst die geen lijst meer mag lijken. Omdat `reset.css` als eerste gelinkt staat, wint elke regel in `stijl.css` op bronvolgorde (2.7); de reset verliest dus altijd van jou, en zo hoort het.

Waar dit vandaan komt: Meyer, normalize en de moderne reset

De projectreset is de jongste telg van een familie die bijna zo oud is als CSS zelf, en je komt de oudere leden voortdurend tegen in codebases en in gegenereerde code. Rond 2007 verschilden browsers nog sterk in wat ze standaard met elementen deden, en Eric Meyer publiceerde zijn **Reset CSS**: één lange selectorlijst die vrijwel elk element op nul zet, margin, padding, border, lettergrootte, zelfs het vet van strong en de grootte van koppen (`font: inherit`), en lijsten hun bolletjes afneemt. Alles plat, en dan bouw je elk uitzicht zelf op. Dat gaf in die tijd zekerheid, en veel cursussen (ook deze, tot vorig jaar) begonnen ermee. In 2011 kwam **normalize.css** met de omgekeerde filosofie: bewaar de nuttige standaardwaarden, en corrigeer alleen de plekken waar browsers van elkaar afwijken. Vandaag zijn browsers het grotendeels eens, en is een reset nog maar een korte lijst meningen over wat een project zinvol vindt: de resets van Andy Bell en Josh Comeau zijn er bekende voorbeelden van, en de onze is er een variant van. Meyers reset herken je aan de lange selectorlijst en `font: inherit` ; als je ze tegenkomt, weet je nu dat strong daar niet vanzelf vet is. De allerkortste vorm komt ook nog voor:

Niet zo:

```
* {
  margin: 0;
  padding: 0;
}
```

Deze "nuke-reset" lijkt op onze regel twee, maar raakt ook lijsten, knoppen, tabellen en formuliervelden, waarna je alles terug opbouwt. Je vindt online nog veel zwaardere exemplaren; die lees je na dit hoofdstuk kritisch in plaats van gelovig, en het AI-kader komt erop terug. De vier regels hierboven staan als `reset.css` in de template van het vak en in de startbestanden van labo 3; na een week kan je ze uit het hoofd verantwoorden, en dat is het punt.

3.4 Padding: ademruimte binnen de box

Padding voeg je toe waar content ademruimte nodig heeft ten opzichte van de eigen rand: vrijwel altijd zodra een element een achtergrondkleur of border draagt, want tekst die tegen een rand plakt, leest als een fout. De vier zijden hebben elk hun eigen property (padding-top, padding-right, padding-bottom, padding-left), maar in de praktijk schrijf je vrijwel altijd de verkorte vorm **padding**, die één tot vier waarden aanvaardt. De volgorde bij vier waarden is met de klok mee vanaf boven (boven, rechts, onder, links), en de kortere vormen vullen spiegelend aan:

```css
.kaart {
  padding: 2rem;                /* all four sides         */
}

.banner {
  padding: 1rem 2rem;           /* top/bottom, left/right */
}

.hoekje {
  padding: 1rem 2rem 3rem 4rem; /* top right bottom left  */
}
```

De tweewaardenvorm is je werkpaard: verticale en horizontale ademruimte verschillen in goed ontwerp meestal, en `padding: 1rem 2rem` zegt dat in vier tekens. Waarden komen uit je tokenblok zodra ze zich herhalen, `padding: var(--space)` in plaats van overal opnieuw een losse maat verzinnen; in hoofdstuk 7 groeit dat uit tot een volwaardige spacing-schaal. Padding mag niet negatief zijn en kent geen `auto`; dat zijn twee verschillen met margin die je in 3.6 terugziet.

3.5 Borders, ronde hoeken en schaduw: de box aankleden

Een **border** bestaat uit drie componenten, dikte, stijl en kleur, en die schrijf je vrijwel altijd samen in de verkorte vorm:

```css
.kaart {
  border: 0.125rem solid var(--color-ink);
}

.notitie {
  border-left: 0.25rem solid var(--color-accent); /* one side only */
}
```

De stijl kent naast `solid` nog waarden als dotted, dashed en double (de volledige lijst staat op MDN; de jaren-negentig-stijlen groove, ridge, inset en outset herken je vooral in oude code). Elke zijde kan apart (border-top, border-left, ...), en dat één-zijde-patroon uit het tweede voorbeeld, een accentstreep links, is een klein designidee dat je vaak zal hergebruiken. Het omgekeerde kan ook: `border-bottom: 0` laat één zijde van een omrande box open, zodat ze als een tabblad op de box eronder lijkt te staan. En een border-bottom op een span is een onderstreping die jij stuurt, in dikte, kleur en afstand, waar text-decoration alleen een dun lijntje in de tekstkleur geeft.

Met **border-radius** rond je de hoeken af, border of niet; ook een achtergrondkleur volgt de afronding. Eén waarde rondt alle hoeken gelijk; per hoek kan het ook (border-top-left-radius en verwanten). En het extreme geval is een gezicht dat je overal ziet: een vierkante box met `border-radius: 50%` wordt een cirkel, de standaardavatar.

box-shadow ten slotte geeft de box een slagschaduw en daarmee diepte. De waarde leest als: horizontale verschuiving, verticale verschuiving, vervaging, kleur, met optioneel een vierde lengte (spread) die de schaduw groter of kleiner maakt dan de box zelf:

```css
.kaart {
  border-radius: 0.5rem;
  box-shadow: 0 0.25rem 1rem rgb(0 0 0 / 0.15);
}

.sticker {
  box-shadow: 0.5rem 0.5rem var(--color-ink); /* no blur: a hard shadow */
}
```

Die kleurnotatie in het eerste voorbeeld, zwart op vijftien procent dekking, is hoe vrijwel elke goede zachte schaduw eruitziet: subtiel, en gemaakt met transparantie in plaats van met grijs. Laat je de vervaging weg, zoals in het tweede voorbeeld, dan krijg je een harde schaduw, het papieren-knipsel-effect dat je in posters en stickers ziet. Meerdere schaduwen op één element scheid je met komma's, en negatieve verschuivingen mogen: `-0.5rem 0.5rem` werpt de schaduw naar linksonder. Het volledige kleurenverhaal (rgb, hsl, transparantie en het kiezen zelf) is designtrack-stof voor hoofdstuk 7; tot dan is de notatie hierboven je vaste recept. Samen vormen padding, border-radius en een zachte schaduw het universele "kaartje"; bouw je in D3.3 en daarna nooit meer uit het hoofd kwijt.

3.6 Margin: afstand tussen boxen

Margin werkt syntactisch exact als padding, vier aparte properties en één verkorte vorm met dezelfde kloksgewijze volgorde, maar gedraagt zich fundamenteel anders, want margin is geen deel van de box. Drie gedragingen moet je kennen, waarvan één beroemd berucht is.

Auto als waarde. Bij margin mag, anders dan bij padding, de waarde `auto`: verdeel de beschikbare ruimte zelf. Op links en rechts tegelijk is dat decennialang hét recept geweest om een blok met vaste breedte te centreren, en het werkt nog altijd:

```css
main {
  max-width: 40rem;
  margin: 0 auto; /* top/bottom 0, left/right share the leftover space */
}
```

Let op het verschil met `text-align: center`, dat de *inhoud* binnen een box centreert; margin auto centreert de *box zelf*. Beide verwarren is F3.4. Grid en flexbox geven je in de volgende hoofdstukken rijkere centreermiddelen, maar dit patroon blijft de standaard voor "een leeskolom in het midden van de pagina".

Het ineenvallen van marges. Nu de beruchte: wanneer twee blokelementen onder elkaar staan, tellen hun aangrenzende top- en bottommargin niet op. De grootste van de twee wint, de kleinste verdwijnt erin. Een h2 met `margin-bottom: 2rem` boven een p met `margin-top: 1.5rem` levert 2rem afstand, geen 3,5. Dit heet **margin collapsing**, het gebeurt alleen verticaal en alleen tussen blokken in de gewone flow, en het is geen bug: waarom het bestaat, lees je in het motorkap-kader. De praktische les

is een strategie die je veel gepieker bespaart: **regel verticale afstand consequent in één richting**, wij kiezen margin-bottom, zodat er nooit twee marges om dezelfde tussenruimte concurreren. Wie boven én onder marges strooit, speelt verstoppertje met het collapsing-mechanisme.

Negatieve marges. Margins mogen negatief zijn: het element schuift dan tégen de normale richting in, over zijn buren of over de rand van zijn ouder heen als het moet. Een fijnafstellingsinstrument voor speciale gevallen, een element dat nét over een rand moet kragen, en tegelijk een geurvlag wanneer je het overal ziet: veel negatieve marges betekenen meestal dat de onderliggende layout vecht met haar auteur.

Op inline tekstelementen tot slot doen verticale marges niets (de tekstregel bepaalt daar de hoogte); horizontale werken wel. Ook dat wordt begrijpelijk in 3.7.

Leeskader: logische properties

Nu je padding en margin per zijde kan benoemen, moet je weten dat het web daar een twéédé namenstelsel voor heeft, en dat je dat voortdurend zal tegenkomen in gegenereerde en professionele code. Naast de fysieke namen (top, right, bottom, left) bestaan de **logische properties**, die zijden benoemen ten opzichte van de schrijfrichting: **inline** is de richting waarin tekst loopt, **block** de richting waarin regels stapelen. In het Nederlands en Engels betekent dat: `margin-inline` is links-en-rechts, `margin-block` is boven-en-onder, `padding-inline-start` is padding-links, en `margin-block-end` is margin-bottom. Hetzelfde stelsel duikt overal op: `inset-block-start` voor top (hoofdstuk 5), `border-inline-start` voor border-left.

Waarom bestaat dat? Omdat niet elke taal van links naar rechts leest. Een site die ook Arabisch of Hebreeuws moet serveren, wil "padding aan de kant waar de tekst begint", niet "padding links", en logische properties draaien dan automatisch mee met de schrijfrichting. Voor jouw werk dit semester is dat zelden een eis, dus deze syllabus schrijft de fysieke namen: die houden je aandacht bij het boxmodel zelf. Maar modellen en moderne codebases schrijven overwegend logisch, dus de vertaaltabel moet in je hoofd zitten: zie je `margin-block: 1rem 2rem`, lees dan margin-top 1rem, margin-bottom 2rem. Wie de logische namen niet kent, leest moderne CSS met gaten erin, en dat is precies het soort stille kennislacune dat het reviewwerk van dit vak wil uitsluiten.

3.7 Display: het karakter van de box

Elke box heeft naast een maat ook een *karakter*: hoe ze zich tegenover haar buren gedraagt. Dat karakter zit in de **display**-property, en de twee waarden die je al kent van het gedrag uit 1.7 heten **block** (begint op een nieuwe regel, neemt de volle beschikbare breedte, aanvaardt width en height) en **inline** (loopt mee in de tekstregel, breekt niet, en negeert width, height en verticale marges; een inline box is zo groot als haar inhoud, punt). Elk element heeft een standaardkarakter uit de user agent stylesheet, p en h1 zijn block, a en em zijn inline, maar dat is een startwaarde, geen natuurwet: met display verander je het karakter van elk element. Een link met `display: block` wordt een knop met een maat, een padding en een achtergrond; een li met `display: inline` staat naast zijn broers in plaats van eronder.

```
nav a {
  display: inline-block;
  padding: 0.75rem 1.5rem;
}
```

Dit voorbeeld toont meteen de derde waarde en haar klassieke gebruiksgeval. **inline-block** is de tussenvorm: het element blijft in de regel meelopen (inline naar buiten toe), maar gedraagt zich binnenin als blok: width, height, padding en marges werken volwaardig. Precies wat een navigatielink nodig heeft: naast zijn broers blijven staan, maar wél een royaal klikvlak krijgen. Dat royale klikvlak is trouwens meer dan esthetiek: kleine klikdoelen zijn een toegankelijkheidsprobleem voor iedereen met minder fijne motoriek, van een trillende hand tot een duim op een telefoon. Ruime padding op interactieve elementen is de a11y-reflex van dit hoofdstuk.

Twee eigenaardigheden van inline-block moet je kennen voor je er kolommen mee bouwt. Ten eerste telt de witruimte in je HTML mee: tussen twee inline boxen staat een spatie, net als tussen twee woorden, dus twee boxen van `width: 50%` passen niet naast elkaar zolang er in de HTML een regeleinde tussen staat. Ten tweede lijnen inline-block boxen uit op de **baseline** van hun tekst, zodat twee boxen met verschillende inhoud niet gelijk bovenaan beginnen; `vertical-align: top` zet ze gelijk. Beide zijn precies de reden dat je volgende week flexbox en grid krijgt. Tot dan weet je waarom het gebeurt, en dat is in dit vak de helft van het werk.

De vierde waarde is **none**: het element verdwijnt volledig uit de weergave, alsof het niet in de boom stond; het neemt geen plaats meer in en de buren schuiven bij. Vergelijk met `visibility: hidden`, dat een element onzichtbaar maakt maar zijn plek laat bezetten: een gat in de pagina. Beide hebben hun gebruik, vooral straks vanuit JavaScript (tonen en verbergen op klik); verwarring tussen beide is F3.7. Voor toegankelijkheid geldt: display none verbergt ook voor schermlezers, meestal precies wat je wil, maar onthoud dat "visueel weg" en "weg" hier samenvallen en bij visibility niet helemaal.

En dan de vooruitblik die dit hoofdstuk zijn plaats in het blok geeft: display kent nog twee waarden, **flex** en **grid**, en die veranderen niet het element zelf maar het gedrag van zijn *kinderen*: ze maken van de box een layoutcontainer die haar inhoud volgens nieuwe regels schikt. Dat is het volledige onderwerp van de hoofdstukken 4 en 5. Alles wat je daar leert, staat óp dit hoofdstuk: een grid-cel en een flex-item zijn gewone boxen met padding, borders en marges, geschikt door een slimmere ouder.

3.8 Grenzen stellen: min, max en overflow

Vaste maten zijn star: een width van 40rem is te breed voor een telefoon en verloren ruimte op een breed scherm. Daarom denk je bij afmetingen vanaf nu standaard in *grenzen* in plaats van in vaste waarden, met **min-width**, **max-width**, **min-height** en **max-height**. Het patroon uit 3.6, `max-width: 40rem` op een gecentreerde leeskolom, is er het schoolvoorbeeld van: op smalle schermen vult de kolom de breedte, op brede stopt ze met groeien waar tekst onleesbaar lang zou worden. Dit is je eerste echte responsive gereedschap, drie hoofdstukken vóór het hoofdstuk dat zo heet; grenzen stellen in plaats van maten dicteren ís de responsive denkwijze.

Hoogtes verdienen daarbij een aparte waarschuwing: **zet zelden een vaste height.** De hoogte-uit-content-regel van 3.2 is je vriend: content verandert, vertaalt zich, wordt langer op een smaller scherm, en een box die meegroeit vangt dat allemaal op. Een vaste height doet dat niet, en dan komt **overflow** in beeld: wat gebeurt er als de content niet in de box past? Standaard (`overflow: visible`) stroomt hij er zichtbaar overheen, over de buren heen als het moet; lelijk, maar eerlijk. `hidden` knipt het teveel onzichtbaar weg (gevaarlijk: content verdwijnt geruisloos, al is het ook het gereedschap om een box binnen een ronde vorm af te snijden), `scroll` geeft de box altijd schuifbalken, en `auto` alleen wanneer het nodig is; vrijwel altijd de juiste keuze wanneer je er een nodig hebt. De echte les van deze property is evenwel preventief: elke keer dat je overflow nodig hebt om content in een box te proppen, is de eerste vraag of die box wel een vaste maat verdiende.

3.9 Afbeeldingen zijn ook boxen, met een eigen wil

Afbeeldingen doen in het boxmodel mee als volwaardige boxen, met één bijzonderheid: ze brengen hun eigen maat mee. Elk rasterbestand (jpg, png, webp) bestaat uit een vast aantal pixels, en die **intrinsieke afmetingen** gebruikt de browser als de natuurlijke maat van de box zolang jij niets zegt. Zeg je wél iets, een width in CSS, dan schaalt de browser mee en bewaart hij automatisch de verhouding, zolang je maar één van beide dimensies zet: width óf height. Wie beide zet in een andere verhouding dan het origineel, vervormt de afbeelding (F3.8); de moderne property `object-fit` waarmee je dat conflict elegant oplost, komt in hoofdstuk 5 aan bod bij de kaartlayouts waar je hem echt nodig hebt.

En hier lossen we een belofte uit hoofdstuk 1 in. Daar leerde je width- en height-*attributen* op img voorlopig weg te laten omdat afmetingen vormgeving zijn, met een aangekondigde uitzondering. Dit is ze: in professionele code zie je die attributen wél, ingevuld met de intrinsieke afmetingen, náást CSS die de echte weergavemaat bepaalt. De reden is laadgedrag. De browser bouwt de pagina terwijl afbeeldingen nog onderweg zijn (denk aan de vijf etappes uit het motorkap-kader van hoofdstuk 1), en zonder maatinformatie reserveert hij géén plaats: elke gearriveerde afbeelding duwt dan de hele pagina omlaag, het ergerlijke verspringen dat je kent van trage nieuwssites. De attributen geven de browser de *verhouding* vooraf, zodat hij het gat kan reserveren. De attributen zeggen dus niet "zo groot moet je zijn" maar "zo gevormd ben ik"; vormgeving blijft aan CSS, en de regel `max-width: 100%` uit onze reset behoudt het laatste woord.

3.10 Boxen lezen in DevTools

Het sluitstuk van dit hoofdstuk is, zoals altijd, het instrument. Onderaan het Styles-paneel (en bovenaan het Computed-tabblad) toont DevTools voor elk geselecteerd element het **boxmodel-diagram**: de vier lagen als geneste rechthoeken, met per zijde de berekende waarde in pixels. Beweeg erover en de overeenkomstige laag licht op in de pagina zelf, elk in zijn eigen kleur: content blauw, padding groen, margin oranje. Dit diagram is de waarheid over elke box: niet wat jij declareerde, maar wat de browser ervan berekende, alle eenheden doorgerekend, collapsing en al. Een rem, een procent en een vh staan er alle drie als pixels, en dat is precies wat je wil weten.

Daarmee is je debug-protocol uit 2.10 compleet voor layoutvragen. Ziet een afstand er verkeerd uit? Selecteer het element en lees af: is het padding of margin (kleur!), van wélk element (selecteer ook de buur), en hoeveel? Negen van de tien "mysterieuze witruimtes" zijn in dertig seconden gediagnosticeerd: een gecollapste marge, een user-agent-standaard die je vergat (de insprong en de marges van een lijst, het vet van een kop), of padding waar je margin dacht. De inspector-hoverbadges naast elementen in de Elements-boom verklappen bovendien het karakter van elke box (block, inline, en straks flex en grid). Meten is weten geldt in CSS letterlijk, en het meetinstrument zit gratis in elke browser.

Onder de motorkap: waar boxen hun maat vandaan halen

Dit kader gaat dieper dan de theorieles. Je hebt het niet nodig om de oefeningen te maken, wel om te begrijpen waarom boxen zich gedragen zoals ze zich gedragen.

Auto is een algoritme, geen afwezigheid. De standaardwaarde van width en height is `auto`, en auto betekent niet "geen maat" maar "reken mij uit". Voor de breedte van een blok in de gewone flow lost de browser een vergelijking op: margin-left + border + padding + width + padding + border + margin-right = breedte van de ouder-content. Alles wat jij vastlegt, wordt ingevuld; wat op auto staat, krijgt de rest, width als eerste. Zet je width vast én beide marges op auto, dan verdeelt de vergelijking de rest gelijk over beide marges: het centreer-recept van 3.6 is dus geen truc maar een oplossing van deze vergelijking. Hoogte volgt een ander pad: auto-height is de som van de inhoud, regel na regel, box na box. Dat is de diepe reden achter het asymmetrische standaardgedrag van 3.2: breedte wordt verdeeld van buiten naar binnen, hoogte opgebouwd van binnen naar buiten. En het verklaart een klassieke verrassing: `height: 100%` doet niets zolang de ouder zelf geen bepaalde hoogte heeft, want honderd procent van "reken mij uit op basis van mijn inhoud" is een cirkelredenering die de browser weigert.

Vervangen elementen brengen hun eigen maat mee. Een img is voor CSS een *replaced element*: de box omhult content waar CSS niet in kan kijken, met eigen intrinsieke afmetingen (3.9). Rasterformaten ontlenen die aan hun pixelraster, en daar zit ook het beloofde antwoord uit hoofdstuk 1 op de vraag waarom SVG schaalt zonder kwaliteitsverlies. Een rasterafbeelding ís haar pixels: twee keer zo groot tonen betekent pixels bijverzinnen, en dat zie je. SVG (Scalable Vector Graphics) bevat geen pixels maar wiskunde, "een cirkel met deze straal, een lijn langs deze punten", en de browser rekent die beschrijving bij elke maat opnieuw om naar een scherp beeld. Vandaar de regel uit 1.10: logo's en iconen als svg, foto's als raster.

Waarom marges ineenvallen. Margin collapsing (3.6) lijkt een gril tot je zijn herkomst kent: CSS is ontworpen voor doorlopende documenten. Geef paragrafen `margin: 1rem 0` zodat er tussen alinea's 1rem ademt, en zonder collapsing zou er tússen twee paragrafen 2rem staan (bottom + top) maar bóven de eerste slechts 1rem; elk document zou ongelijk ademen. Het ineenvallen maakt "1rem marge rondom" betekenen wat een typograaf bedoelt: nergens minder, nergens dubbel. De regels eromheen zijn genuanceerder dan de theorieles vertelt: ook de marges van een ouder en zijn eerste of laatste

kind kunnen versmelten (waardoor een marge "uit zijn container lekt", het mysterie achter menige F3.6-diagnose), en padding of border op de ouder blokkeert dat versmelten. Wie het onderste uit de kan wil: MDN's *Mastering margin collapsing* staat bij Verder lezen.

Eén box, soms meer fragmenten. Tot slot een subtiliteit die het model afrondt: een inline element genereert wel een box, maar die kan in stukken breken; een lange link wrapt over drie regels en bestaat dan visueel uit drie fragmenten. Daarom weigeren inline boxen width en height (welk fragment zou je meten?) en doen verticale marges er niets: de regelhoogte van de tekst regeert. Zodra je een element `display: inline-block` of `block` geeft, beloof je de browser één onbreekbare rechthoek, en werkt het volledige boxmodel weer. Het display-karakter van 3.7 is dus letterlijk de keuze wélk soort box een element genereert.

Veelgemaakte fouten

De fouten van deze week zie je, anders dan vorige week, wél: een layout die net niet klopt. Wat je niet ziet, is waarom. Het boxmodel-diagram wel. De feedback op labo 3 verwijst naar deze nummers.

F3.1 - De breedte-verrassing. *Symptoom*: twee kolommen van 50% passen niet naast elkaar, een box van 30rem meet er 34. *Oorzaak*: padding en border opgeteld bovenop width, het content-box-model van 3.2, meestal doordat `reset.css` niet gelinkt is. *Remedie*: reset erin; bij oud werk de rekensom van 3.2 maken. Het boxmodel-diagram toont je de optelling zwart op wit.

F3.2 - Hoogte 100% doet niets. *Symptoom*: `height: 100%` en er gebeurt niets. *Oorzaak*: de ouder heeft zelf auto-hoogte; honderd procent daarvan is een cirkelredenering (zie motorkap). *Remedie*: vraag je eerst af of je die hoogte echt nodig hebt (3.8); zo ja, geef de keten een echt vertrekpunt, bijvoorbeeld een hoogte in vh op de buitenste box (3.2).

F3.3 - Padding en margin verward. *Symptoom*: de achtergrondkleur "loopt door" in ruimte die tussen elementen moest zitten, of ademruimte binnen een kaart kleurt niet mee. *Oorzaak*: de verkeerde van de twee ruimtesoorten. *Remedie*: de vraag uit 3.1: hoort de ruimte bij de box of tussen de boxen? Het kleurgecodeerde diagram in DevTools maakt de diagnose in één oogopslag.

F3.4 - Centreren met het verkeerde gereedschap. *Symptoom*: text-align: center op alles gegooid, en het blok zelf staat nog links. *Oorzaak*: inhoud-centreren en box-centreren verward. *Remedie*: inhoud binnen een box: text-align. De box zelf: een breedte plus margin: 0 auto (3.6).

F3.5 - Width op een inline element. *Symptoom*: width, height of verticale padding op een a of span doet niets of doet raar. *Oorzaak*: inline boxen meten zichzelf (3.7, motorkap). *Remedie*: display: inline-block (of block), het navigatielink-patroon van 3.7.

F3.6 - De mysterieuze verticale afstand. *Symptoom*: er zit méér of mínder ruimte tussen twee blokken dan je marges beloven, of een marge lijkt uit zijn container te lekken. *Oorzaak*: margin collapsing (3.6, motorkap). *Remedie*: eenrichtingsstrategie (margin-bottom); selecteer beide boxen in DevTools en lees de oranje lagen.

F3.7 - Verdwenen versus onzichtbaar. *Symptoom*: een verborgen element laat een gapend gat achter, of omgekeerd: buren schuiven op waar dat niet mocht. *Oorzaak*: visibility: hidden en display: none verward. *Remedie*: gat behouden = visibility; plaats vrijgeven = display: none (3.7).

F3.8 - De vervormde afbeelding. *Symptoom*: een uitgerekte of geplette foto. *Oorzaak*: width én height allebei gezet, in een andere verhouding dan het origineel. *Remedie*: zet er één en laat de browser de verhouding bewaren (3.9); voor bijsnijden-in-een-vak bestaat object-fit (hoofdstuk 5).

F3.9 - Spacing met noodgrepen. *Symptoom*: lege paragrafen of br-stapels als witruimte-generator (het verouderde idioom uit het AI-kader van hoofdstuk 1), of overal net iets andere magic-number-marges. *Oorzaak*: ruimte als inhoud behandeld in plaats van als stijl, of geen systeem. *Remedie*: witruimte is CSS (padding en margin), en herhaalde maten horen in je tokenblok; de spacing-schaal van hoofdstuk 7 maakt er een systeem van.

F3.10 - Vaste hoogtes overal. *Symptoom*: afgeknipte tekst zodra content langer wordt, in een andere taal of op een smaller scherm. *Oorzaak*: height gebruikt waar de content de maat hoorde te geven. *Remedie*: de waarschuwing van 3.8: hoogte groeit uit content; wil je een ondergrens, gebruik min-height.

Met AI aan je zij: gegenereerde layout-CSS lezen

Bij boxmodel-werk zijn de kwalen van gegenereerde code goed voorspelbaar; vier patronen om op te screenen.

Vaste maten waar grenzen hoorden. Modellen zetten opvallend graag exacte widths en heights op alles, een erfenis van pixel-perfecte trainingsvoorbeelden. Het resultaat oogt correct in de ene demo en breekt bij de eerste langere tekst (F3.10). Jouw controlevraag per maat: had dit een min- of max-grens moeten zijn, of helemaal niets?

Magic-number-marges. Gegenereerde spacing is vaak een strooisel van net-verschillende waarden (13px hier, 15px daar, 0.9rem verderop) die visueel "kloppen" maar geen systeem vormen en collapsing-verrassingen maskeren met nog een extra marge. Herschrijf naar je tokens en naar de eenrichtingsstrategie, en de helft van de declaraties verdampt meestal.

Reset-archeologie. Vraag om een volledige pagina en je krijgt geregeld een reset-blok uit een ander tijdperk mee: de nuke-reset `* { margin: 0; padding: 0; }` of de volledige Meyer-lijst uit 3.3, float-clearfixes, of lange lijsten verouderde vendor-prefixes (-webkit-, -moz-). Werkend, maar het is sediment van vijftien jaar trainingsdata. Vergelijk met onze vier regels en vraag je bij elke extra regel af welk probleem hij vandaag nog oplost.

Presentatie-elementen en wrapper-boxen. Voor ruimte en lijnen grijpen modellen soms terug naar markup: br-stapels, een leeg div'je als spacer, een hr waar een border-top van CSS hoorde. Dat is de taakverdeling van hoofdstuk 1 die sneuvelt voor visueel gemak, en het is precies wat jij na drie hoofdstukken sneller ziet dan het model het genereert.

Oefeningen

Labo 3 bevat vijf nabouwen, elk met een screenshot en een vergrendeld HTML-bestand, plus de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen.

Drills (zonder AI aanbevolen)

- **D3.1** Rekenoefening op papier: zes boxen met gegeven width, padding, border en margin; bereken telkens de totale visuele breedte, mét en zonder border-box. Controleer daarna twee ervan in het boxmodel-diagram van DevTools.
- **D3.2** Schrijf de vier regels van de projectreset uit het hoofd op en verantwoord bij elke regel wat hij doet en waarom; vergelijk dan pas met `reset.css`. Doe hetzelfde voor drie regels uit de reset van Meyer: welk probleem losten ze op, en bestaat dat nog?
- **D3.3** Bouw hét kaartje: afbeelding, titel, tekstje, met padding, border-radius, subtiele schaduw en correcte binnenruimte, het patroon uit 3.5. Drie varianten: met border, met accentstreep links, met alleen schaduw.
- **D3.4** Centreer een leeskolom (max-width, margin auto) en geef de tekst een consistent verticaal ritme met de eenrichtingsstrategie: alle afstanden via margin-bottom, waarden uit je tokenblok.
- **D3.5** Bouw de navigatiebalk van je site om naar links met royale klikvlakken via display: inline-block, inclusief hover en focus uit D2.5.
- **D3.6** Voorspel-en-controleer: vijf situaties rond margin collapsing en inline gedrag (h2 boven p, marge op een em, ouder-kind-marges). Schrijf je voorspelling op, bouw het na, meet in DevTools.
- **D3.7** Eenhedengymnastiek op een box: een ouder van 40rem breed met een kind van `width: 50%`, `padding: 10%` en `margin-top: 5%`; bereken alle pixelwaarden op papier, dan controleren in het Computed-tabblad. Doe hetzelfde voor een band van `height: 50vh` in een venster van 800 px hoog.

Debug en review (zonder AI)

- **R3.1** Een pagina waarin acht fouten uit het onderdeel "veelgemaakte fouten" verwerkt zitten (minstens F3.1, F3.3, F3.4, F3.6 en F3.10): diagnosticeer elke fout mét het DevTools-bewijs (welke laag, welk element), benoem de oorzaak en herstel.
- **R3.2** Je krijgt twee stylesheets die exact dezelfde kaartcomponent opleveren: één met vaste maten en verspreide magic numbers, één met grenzen, tokens en de eenrichtingsstrategie. Voorspel voor drie scenario's (langere tekst, smaller scherm, extra kaart) wat elk doet, en verifieer.

Bouwen en verantwoorden (met AI of zonder AI)

- **B3.1** Versie drie van je persoonlijke site: `reset.css` uit de template erin, gelinkt vóór `stijl.css` op elke pagina, een gecentreerde leeskolom, minstens drie kaartcomponenten (bijvoorbeeld voor projecten of interesses), consistente spacing via tokens en de eenrichtingsstrategie, en nergens een vaste height op tekstdragende elementen. Bij de AI-route: prompt, onbewerkte output en je verbeterlijst; let dit keer speciaal op vaste maten en magic numbers.

Verder lezen

- MDN, *The box model*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Box_model
- MDN, *Sizing items in CSS*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Sizing
- MDN, *Overflowing content*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/Overflow
- MDN, *Mastering margin collapsing*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_box_model/Mastering_margin_collapsing
- MDN, *box-sizing*: https://developer.mozilla.org/en-US/docs/Web/CSS/box-sizing

Hoofdstuk 4 - Layout 1: de gereedschapskist

In een notendop

Drie hoofdstukken lang vielen je boxen onder elkaar, in de volgorde van je HTML. Deze week krijg je de twee layoutgereedschappen van het moderne web in handen: **flexbox**, dat boxen verdeelt langs één as, en **grid**, dat een raster over twee assen spant. Beide werken op dezelfde manier: één declaratie op de ouder maakt van die ouder een layoutcontainer, en zijn directe kinderen worden items die volgens nieuwe regels geschikt worden. De kunst zit niet in de syntax maar in het kiezen, en de keuze volgt uit één vraag: verdeel ik langs één as, of span ik een raster over twee? Grid voor de pagina, flexbox voor de rij. Aan het einde bouw je er een grondplan, een navigatiebalk en een kaartenrooster mee, en lees je elke layout af in DevTools.

Wat je na dit hoofdstuk kan

De gewone flow, boxen onder elkaar, is voor een leeskolom perfect. Maar een webpagina is meer dan een kolom: een navigatiebalk met links naast elkaar, een rooster van kaarten, een zijbalk naast een artikel, een footer die onderaan blijft. In labo 3 bouwde je zo'n rij nog met inline-block, en je voelde waar dat wringt: een spatie in de HTML die meetelt, een uitlijning die niet klopt. Deze week verdwijnt dat geknutsel. Je leert flexbox en grid samen, want ze beantwoorden dezelfde vraag op twee schaalniveaus, en je leert vooral kiezen.

Na dit hoofdstuk bouw je een volwaardige paginalayout: een grondplan in grid, een navigatiebalk in flexbox, een kaartenrooster met gelijke hoogtes, en alles gecentreerd wat gecentreerd moet; het soort structuur dat je op de toets in november zelfstandig neerzet. Even belangrijk: je kan bij elke layoutvraag beargumenteren wélk gereedschap je pakt en waarom.

Het hoofdstuk heeft drie delen. Eerst het denkraam (4.1 en 4.2): wat een layoutcontainer is, en de ene vraag die elk layoutprobleem opent. Dan de twee gereedschappen zelf: flexbox (4.3 en 4.4) en grid (4.5 tot 4.7), met het grondplan van een pagina als eindpunt. Ten slotte kiezen, combineren en lezen (4.8 en 4.9). In labo 4 zet je beide gereedschappen in: in nabouwen, en op je eigen site.

4.1 De derde laag van display: containers die hun kinderen schikken

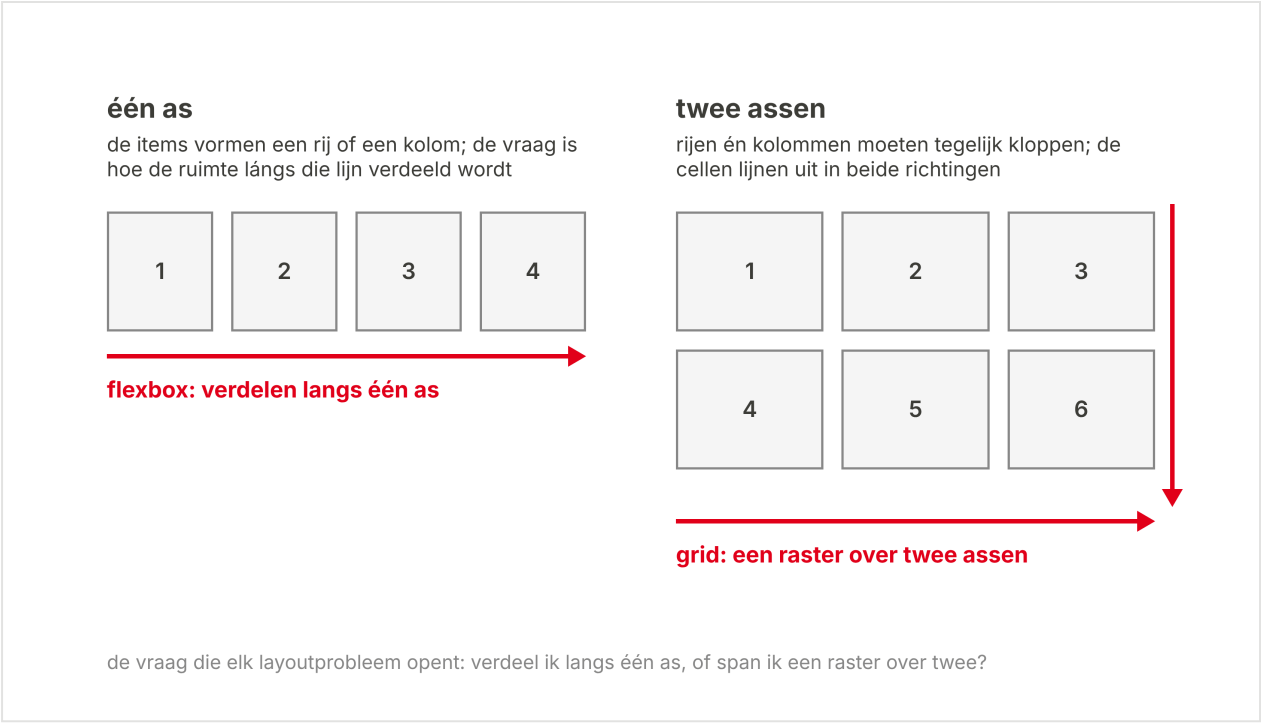
In 3.7 leerde je display kennen als het karakter van een box: block, inline en inline-block bepalen hoe een element zich tegenover zijn *buren* gedraagt. De twee waarden die we toen aankondigden, werken fundamenteel anders: **flex** en **grid** veranderen niets aan het element zelf, maar alles aan wat er *binnen*

gebeurt. Zet je op een element `display: flex` of `display: grid`, dan wordt dat element een **layoutcontainer**, en zijn directe kinderen worden **items** die volgens nieuwe regels geschikt worden.

Twee gevolgen van die formulering moet je meteen scherp hebben, want ze veroorzaken samen de helft van alle beginnersfrustratie met layout. Ten eerste: **de container organiseert, de items ondergaan**. Layoutproperties als "verdeel de ruimte" of "maak drie kolommen" schrijf je op de *ouder*, niet op de kinderen. Wie een rij wil en `display: flex` op de items zelf zet, krijgt niets. Ten tweede: het gaat om **directe kinderen**, en alleen die. Kleinkinderen doen niet mee; die leven in de box van hún ouder. Dit is de boom van hoofdstuk 1 die opnieuw regeert, en het is de reden waarom één onschuldig ogende wrapper-div een hele layout kan breken (F4.1, de meest gestelde vraag van dit lesblok, elk jaar opnieuw).

De twee eigenaardigheden van inline-block uit 3.7 zijn hier meteen verdwenen: een layoutcontainer negeert de witruimte tussen zijn kinderen, en hij lijnt ze uit zoals jij het zegt in plaats van op de baseline van hun tekst. De gewone flow verdwijnt niet; ze blijft het vertrekpunt en voor lopende content het juiste gedrag. Layoutcontainers zet je gericht in, daar waar je schikking nodig hebt, en containers mogen genest worden: een grid-item kan zelf flex-container zijn voor zíjn kinderen. Zo bouw je elke pagina op: grote structuur buiten, fijne verdeling binnen. Dat het web ooit layouts bouwde met misbruikte tabellen en zwevende floats, hoef je alleen te weten om oude tutorials te herkennen en te sluiten; appendix C vat dat verleden samen. Jij leert layout zoals het sinds jaren hoort.

4.2 De vraag die elk layoutprobleem opent



Figuur 4.1 - Eén as of twee assen: flexbox verdeelt items langs één lijn, een rij of een kolom; grid spant een raster waarin rijen en kolommen tegelijk kloppen.

Voor je ook maar één property typt, stel je bij elke layoutwens dezelfde vraag: **verdeel ik langs één as, of span ik een raster over twee?**

Eén as betekent: de items vormen een rij óf een kolom, en wat je interesseert is hoe de ruimte lángs die lijn verdeeld wordt. Een navigatiebalk, een knoppenrij, een logo met tekst ernaast, een verticale stapel met gelijke tussenruimte. Dat is **flexbox**: één as, en de items mogen op die as groeien, krimpen en schuiven naargelang hun inhoud.

Twee assen betekent: rijen én kolommen moeten tegelijk kloppen, cellen moeten uitlijnen in beide richtingen. Een fotorooster, een dashboard, het grondplan van een pagina met header, content en footer. Dat is **grid**: jij beschrijft het raster op de container, en de items voegen zich naar de cellen.

De vuistregel die dit hoofdstuk zijn kernmodel geeft, volgt vanzelf: **grid voor de pagina, flexbox voor de rij.** De grote structuur van een pagina is vrijwel altijd tweedimensionaal; de componenten dáárin, navigaties, knoppenrijen, kaartinhoud, zijn vrijwel altijd eendimensionaal. Er is ook een diepere manier om het verschil te onthouden, en die helpt bij twijfelgevallen: bij flexbox bepaalt de *inhoud* de layout (items nemen de maat van wat erin zit en onderhandelen dan over de restruimte), bij grid bepaalt de *layout* de inhoud (het raster staat er, de items voegen zich). Content-first tegenover layout-first. In 4.8 keren we naar deze vraag terug met geoefende ogen.

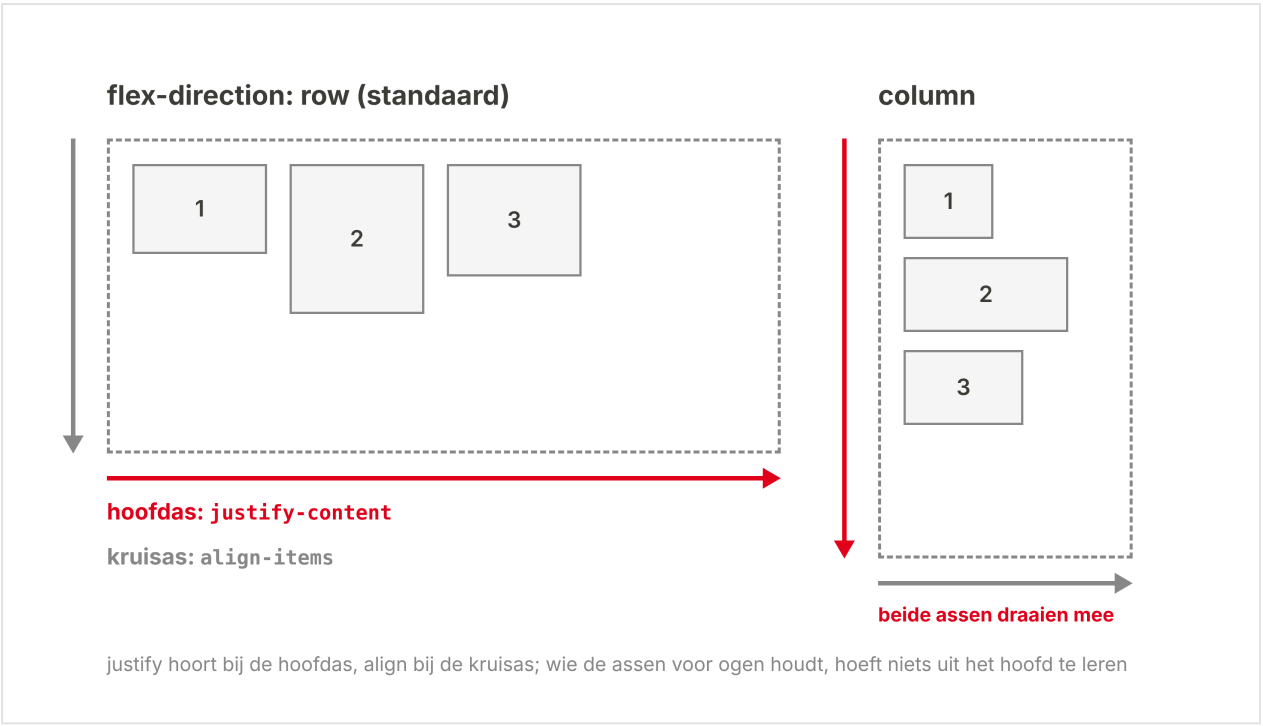
4.3 Flexbox: verdelen langs één as

Eén declaratie op de ouder en de wereld kantelt:

```
<ul class="menu">
  <li><a href="index.html">Home</a></li>
  <li><a href="recepten.html">Recepten</a></li>
  <li><a href="over-mij.html">Over mij</a></li>
</ul>
```

```
.menu {
  display: flex;
}
```

De li's, blokelementen die gisteren nog braaf onder elkaar vielen, staan nu naast elkaar op één rij. Dat is de **hoofdas** (main axis) van de flex-container, standaard horizontaal, van links naar rechts. Haaks daarop staat de **kruisas** (cross axis), standaard verticaal. Deze twee assen zijn het hele denkraam van flexbox: elke property die volgt, werkt op de ene of op de andere, en wie de assen voor ogen houdt, hoeft niets uit het hoofd te leren.



Figuur 4.2 - De twee assen van een flex-container: justify-content verdeelt langs de hoofdas, align-items lijnt uit op de kruisas; flex-direction: column draait beide.

Met **flex-direction** kies je de hoofdas: `row` (standaard) of `column`, en dan draait álles mee, want de hoofdas is dan verticaal en de kruisas horizontaal. De waarden row-reverse en column-reverse bestaan ook; gebruik ze spaarzaam, want ze laten de visuele volgorde afwijken van de DOM-volgorde, en dat verwart toetsenbordnavigatie en schermlezers: de a11y-kanttekening van dit hoofdstuk.

Ruimte tussen de items maak je met **gap**, dezelfde property die je straks bij grid terugziet: één waarde, en er valt tussenruimte tussen elk paar items, alléén ertussen, niet aan de buitenranden. Vergelijk dat even bewust met hoe je dit vorige week had opgelost:

Niet zo:

```
.menu li {
  margin-right: 1rem;
}

.menu li:last-child {
  margin-right: 0; /* undo the last one */
}
```

Zo wel:

```
.menu {
  display: flex;
  gap: 1rem;
}
```

Gap zegt in één declaratie wat het ontwerp bedoelt ("deze items staan 1rem uit elkaar") en laat de marges van je items met rust. Het is op beide layoutgereedschappen de standaardmanier om tussenruimte te maken, en marges tussen items degraderen tot uitzonderingsgereedschap.

Dan de verdeling zelf. **justify-content** verdeelt de items en de restruimte langs de *hoofdas*: `flex-start` (standaard, alles aan het begin), `flex-end`, `center`, en de twee die restruimte uitsmeren: `space-between` (eerste en laatste item tegen de randen, de rest gelijk verdeeld) en `space-around`. **align-items** lijnt uit op de *kruisas*: standaard `stretch` (items rekken zich tot de hoogte van de hoogste; dit is waarom flex-items in een rij automatisch gelijke hoogte hebben, een cadeau waar het oude web jaren om smeekte), verder `flex-start`, `flex-end`, `center` en `baseline` (uitlijnen op de tekstbasislijn, voor rijen met verschillende lettergroottes).

```
.menu {
  display: flex;
  gap: 1rem;
  justify-content: flex-end;  /* links huddle at the end of the row  */
  align-items: center;        /* vertically centred in the bar       */
}
```

Justify hoort bij de hoofdas, align bij de kruisas; bij `flex-direction: column` wisselen ze dus van visuele richting. Wie dat één keer bewust heeft zien gebeuren (D4.2 laat het je doen), verwart ze nooit meer; wie het overslaat, blijft gokken (F4.3).

Hiermee staat het patroon dat je deze week overal gaat toepassen, de **navigatiebalk**: een ul als flex-container, gap ertussen, verticaal gecentreerd, en de links zelf als royale klikvlakken uit 3.7. Drie hoofdstukken kennis in zes declaraties. En de twee helften van de landingspagina uit labo 3, die je met inline-block tegen elkaar moest plakken, zijn nu `display: flex` op main en niets meer.

4.4 Groeien en krimpen: flex

Tot nu behielden de items hun natuurlijke maat en verdeelden we alleen de rúímte errond. Flexbox kan meer: de items zelf laten meebewegen. Dat regelt de **flex**-property, die je, let op, op de *items* zet, niet op de container: het is de enige plek in dit hoofdstuk waar een kind zelf iets te zeggen heeft, namelijk over zijn eigen groeigedrag.

De vorm die je dit semester nodig hebt, is de eenvoudigste en meteen de nuttigste:

```
.kolommen {
  display: flex;
  gap: 2rem;
}

.kolommen > * {
  flex: 1; /* every direct child gets an equal share of the row */
}
```

`flex: 1` betekent: neem een gelijk deel van de beschikbare ruimte, ongeacht je inhoud. Drie kinderen met flex: 1 zijn drie gelijke kolommen; geef er één `flex: 2` en die neemt een dubbel aandeel. Laat je één item op flex: 1 staan en de rest op hun natuurlijke maat, dan wordt dat ene item de rekbare partij die alle restruimte opslorpt: het klassieke recept voor een zoekbalk die groeit tussen een logo en een knop. Hetzelfde recept werkt in een kolom: in een kaart die als flex-column is opgebouwd (afbeelding, titel, tekst, knop), krijgt de tekst `flex: 1`, en de knop staat in elke kaart onderaan, hoe kort of lang de tekst ook is. Dat patroon zie je in 4.8 terug.

Onder deze verkorte notatie liggen drie afzonderlijke properties (flex-grow, flex-shrink en flex-basis) met een subtiel samenspel; dat fijnere mechaniek, samen met het afbreken over meerdere regels met flex-wrap, bewaren we voor hoofdstuk 5. Voor nu is de afspraak: schrijf de verkorte vorm `flex: 1` (of een ander getal) en niets anders. Het is de vorm die vrijwel alle praktijkgevallen dekt én de vorm die je in andermans code het vaakst leest.

4.5 Grid: een raster over twee assen

Nu het tweede gereedschap, voor wanneer het antwoord op de as-vraag "twee" luidt. Ook grid begint met één declaratie op de ouder, plus één die het raster beschrijft:

```
.galerij {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 1.5rem;
}
```

grid-template-columns beschrijft de kolommen: één waarde per kolom, gescheiden door spaties. Je mag elke meeteenheid gebruiken, maar de eenheid die grid zijn kracht geeft is nieuw: de **fr**, een *fraction* van de beschikbare ruimte. `1fr 1fr 1fr` betekent drie gelijke delen; `2fr 1fr` een verdeling van twee derde tegen één derde; `12rem 1fr` een vaste zijkolom naast een hoofdkolom die de rest pakt, het zijbalk-patroon in vier woorden. Waarom fr slimmer is dan percentages (en wat er precies "beschikbaar" is), krijgt zijn volle uitleg in het motorkap-kader; onthoud nu dat fr rekening houdt met gaps en vaste kolommen, en percentages niet (F4.4).

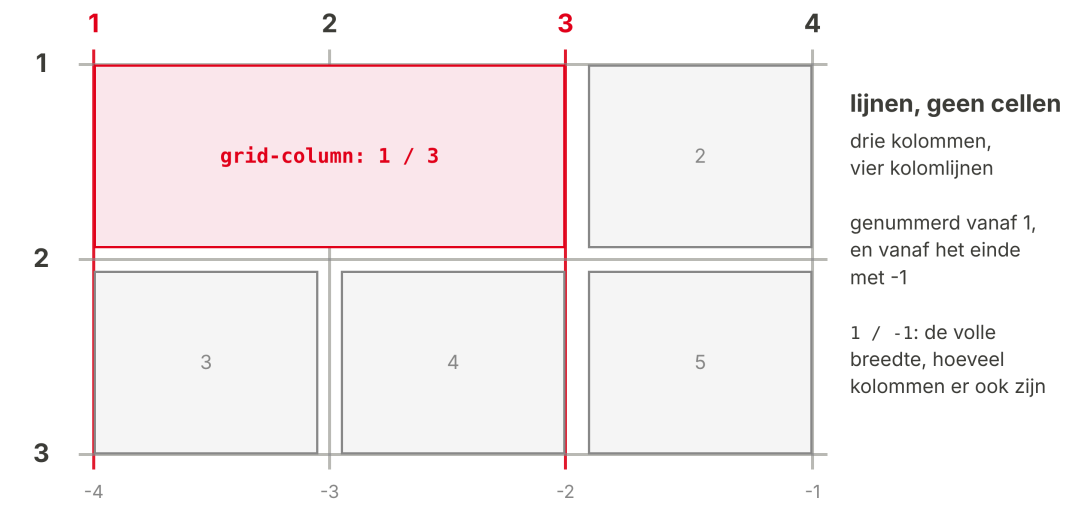
Voor repetitieve rasters bestaat een verkorte notatie die je vooral moet kunnen lézen: `repeat(3, 1fr)` is exact `1fr 1fr 1fr`, en `repeat(4, 12rem)` vier vaste kolommen. In hoofdstuk 6 stopt er iets veel nuttigers in die repeat-functie, het recept voor rasters die zichzelf aanpassen aan de schermbreedte, dus de vorm is het onthouden waard.

En de rijen? Die heb je meestal niet nodig. De items vullen het raster in DOM-volgorde, kolom na kolom, rij na rij, en de browser maakt vanzelf zoveel rijen als er nodig zijn, zo hoog als hun inhoud vraagt, precies zoals boxen altijd al hun hoogte uit content haalden (3.2). We noemen dit de **impliciete grid**, en het is de normale gang van zaken: je beschrijft kolommen, de rijen ontstaan. Expliciete rijen (**grid-template-rows**) declareer je alleen wanneer rijen écht een eigen maat nodig hebben; het grondplan in 4.7 is er het goede voorbeeld van, en "overal rijen vastleggen" is een variant van de vaste-hoogtes-fout uit 3.8.

Voor het overige geldt alles wat je al weet: gap werkt identiek aan flexbox, alleen mag je hier twee waarden geven (rij-gap en kolom-gap), en de items zijn gewone boxen, met padding, borders, kaartschaduw en al, die nu door een raster geschikt worden in plaats van door de flow.

4.6 Plaatsen op het raster

De impliciete vulling, item één in cel één, enzovoort, is vaak precies goed. Maar grid laat je ook gericht plaatsen, en daarvoor moet je één denkstap maken: grid telt niet in cellen maar in **lijnen**. Een raster van drie kolommen heeft vier verticale lijnen, genummerd vanaf 1; een cel ligt tússen twee lijnen. Plaatsing is dus altijd "van lijn tot lijn":



Figuur 4.3 - Grid telt in lijnen, niet in cellen: drie kolommen hebben vier kolomlijnen, genummerd vanaf 1 (en vanaf het einde met -1). Het uitgelichte item loopt van lijn 1 tot lijn 3.

```
.uitgelicht {
  grid-column: 1 / 3; /* from line 1 to line 3: two columns wide */
  grid-row: 1 / 2;    /* the first row                           */
}
```

grid-column en **grid-row** nemen elk een start- en eindlijn, gescheiden door een slash. Er zijn twee handige verkortingen: laat je het tweede getal weg, dan beslaat het item één cel vanaf die lijn, en `span 2` in plaats van een eindlijn betekent "twee sporen breed, waar ik ook sta", vaak natuurlijker dan absolute lijnnummers, omdat het meebeweegt als het raster verandert. Eén item gericht plaatsen heeft trouwens geen domino-effect: de overige items blijven gewoon impliciet vullen en stromen om het geplaatste item heen.

Met plaatsing kan je drie dingen die de impliciete vulling niet kan: een item laten *uitspreiden* over meerdere cellen (de uitgelichte kaart die twee kolommen pakt, de zijbalk die alle rijen bestrijkt), bewust *witruimte* maken door een cel onbezet te laten, en de visuele positie *loskoppelen* van de DOM-volgorde. Bij dat laatste hoort dezelfde waarschuwing als bij row-reverse in 4.3: toetsenbord en schermlezer volgen de DOM, niet je raster. Herschik gerust om een goede reden, maar als je álles aan het herplaatsen bent, is dat meestal een teken dat je HTML-volgorde niet klopt en dáár de fix zit.

Er bestaat nog een tweede, meer visuele plaatsingstaal: je geeft gebieden namen en tekent het raster letterlijk uit in je CSS. Die bewaren we voor hoofdstuk 5, wanneer de grondplannen complex genoeg worden om haar te verdienen. Lijnen eerst: wie in lijnen kan denken, leest élke grid.

4.7 Uitlijnen, centreren, en het grondplan van een pagina

Grid kent dezelfde uitlijningswoorden als flexbox, met dezelfde as-logica: **justify-** werkt horizontaal (langs de rijen), **align-** verticaal (langs de kolommen), en beide bestaan op containerniveau met -content (het hele raster binnen de container) en -items (de inhoud binnen elke cel). Cellen rekken hun items standaard uit (stretch, net als bij flexbox); `align-items: start` laat een kaart zijn natuurlijke hoogte behouden bovenin zijn cel.

Uit deze woordenschat valt het beroemdste recept van moderne CSS, het antwoord op een vraag die het web twintig jaar heeft gekweld: iets perfect centreren, horizontaal én verticaal.

```
.held {
  display: grid;
  place-items: center; /* shorthand for align-items + justify-items */
  min-height: 100vh;
}
```

Drie declaraties. Alles wat in .held zit, staat in het absolute midden van het scherm. (**place-items** is de verkorte vorm van de twee items-properties samen; hij bestaat ook bij -content.) Met flexbox kan het ook, in vier declaraties: `display: flex`, `justify-content: center`, `align-items: center`. Beide zijn goed; de grid-vorm is korter en de vorm die je het vaakst zal lezen.

En daarmee kunnen we het grondplan bouwen dat je site vanaf deze week draagt. Het probleem dat het oplost ken je misschien nog niet bewust, maar je hebt het gezien: op een pagina met weinig inhoud bengelt de footer halverwege het scherm, met een lege zone eronder. De oplossing is een grid van drie rijen op body:

```
body {
  display: grid;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}
```

Lees de rijen: `auto`, de header wordt zo hoog als zijn inhoud; `1fr`, main krijgt álle resterende ruimte; `auto`, de footer wordt zo hoog als zijn inhoud. En `min-height: 100vh` garandeert dat het geheel minstens het scherm vult (min-height, niet height: langere pagina's moeten gewoon kunnen doorgroeien, de les van 3.8; over een subtiliteit van 100vh op mobiele browsers hoor je meer in hoofdstuk 6). Dat body hier zonder witruimte tegen de rand van het venster staat, dank je aan `margin: 0` uit `reset.css`. De footer staat nu onderaan het scherm bij weinig inhoud en onderaan het document bij veel: het **sticky footer**-patroon, en meteen je eerste échte tweedimensionale layout. Drie semantische elementen uit 1.8, drie rijen, nul hulpelementen.

4.8 Kiezen, en combineren

Terug naar de vraag van 4.2, nu met beide gereedschappen in de hand. De heuristiek op een rijtje, van grof naar fijn. Moeten rijen en kolommen allebei kloppen, of beschrijf je het ontwerp vanzelf als "raster"? Grid. Gaat het om één rij of kolom waarin items hun eigen maat mogen hebben en de restruimte verdeeld wordt? Flexbox. Weet je vooraf hoeveel sporen er zijn (jij tekent het raster)? Grid. Bepaalt de inhoud hoeveel er passen en hoe breed ze zijn? Flexbox. En bij echte twijfel: begin met flexbox; zodra je merkt dat je tegen de tweede as aan het vechten bent, is dat het signaal dat het grid moest zijn.

VRAAG	ANTWOORD	GEREEDSCHAP
Moeten rijen én kolommen uitlijnen?	ja	grid
Eén rij of kolom, items met hun eigen maat?	ja	flexbox
Weet je vooraf hoeveel sporen er zijn?	ja	grid
Bepaalt de inhoud hoeveel er passen?	ja	flexbox
Twijfel?	begin met flexbox	vecht je tegen de tweede as: grid

Belangrijker dan de keuze is het besef dat het geen kamp-keuze is. Elke echte pagina gebruikt beide, genest volgens het principe van 4.1: het grondplan van 4.7 is een grid, de navigatiebalk in de header is een flexbox, het kaartenrooster in main is een grid, en bínnen elke kaart schikt een flexbox (richting column) de afbeelding, titel en tekst onder elkaar, met `flex: 1` op de tekst zodat de knop in elke kaart onderaan staat (4.4). Grid voor de pagina, flexbox voor de rij: het kernmodel is letterlijk het bouwplan van je site.

Wat je níét doet, is van de hele pagina één groot allesomvattend raster maken waarin elke knop een cel krijgt. Dat kán, en het wordt een onbeheersbaar monster: elke inhoudswijziging vergt herberekening van het hele raster. Kleine, geneste, begrijpelijke containers, elk met één taak, zijn de professionele norm, dezelfde ontwerplogica als kleine functies in code straks in blok 3.

4.9 Layouts lezen in DevTools

Ook dit hoofdstuk eindigt bij het instrument, en voor layout is dat instrument uitzonderlijk goed. In de Elements-boom draagt elke layoutcontainer een badge: **flex** of **grid**, klikbaar. Eén klik en DevTools tekent de overlay over de pagina: bij grid de lijnen mét hun nummers (de nummering van 4.6, live op je eigen raster), de gaps gearceerd, desgewenst de maten per spoor; bij flex de items en de verdeling langs de hoofdas.

De overlay beantwoordt de twee vragen die je bij elke layoutbug moet stellen. Eén: *is dit element wel de container die ik denk dat het is?* Geen badge betekent geen layoutcontainer: dan is je display-declaratie niet aangekomen (selector? cascade? hoofdstuk 2 weet raad) of zit hij op het verkeerde element. Twee: *zijn dít de items die ik denk?* De overlay toont exact welke kinderen meedoen; een onverwachte wrapper (F4.1) valt hier onmiddellijk door de mand, want de overlay omlijnt de wrapper als één groot item in plaats van de kaarten die erin zitten. Samen met het boxmodel-diagram van 3.10 heb je nu het volledige diagnose-instrumentarium voor de toets: boom, boxen en raster.

Onder de motorkap: hoe fr verdeelt

Dit kader gaat dieper dan de theorieles. Je hebt het niet nodig om de oefeningen te maken, wel om te begrijpen waarom fr doet wat percentages niet kunnen.

Eerst de vaste kosten, dan de verdeling. Wanneer de browser een grid met fr-kolommen legt, rekent hij in twee rondes. Ronde één: alles wat een vaste of inhoudelijke maat heeft, wordt van de containerbreedte afgetrokken: vaste kolommen (12rem), én alle gaps. Wat overblijft heet de *beschikbare ruimte*. Ronde twee: die beschikbare ruimte wordt verdeeld over de fr-kolommen, naar rato van hun getal; bij `2fr 1fr` in delen van twee derde en één derde. Een fr is dus geen breedte maar een *aandeel in de rest*, en daarom kloppen fr-layouts altijd op de pixel: de gaps zijn al betaald vóór er verdeeld wordt. Reken nu na waarom `33.33% 33.33% 33.33%` met gaps wél overloopt: percentages worden berekend op de vólle containerbreedte, gaps komen er nog eens bovenop, en drie keer een derde plus twee gaps is meer dan honderd procent (F4.4). Percentages weten niets van hun buren; fr weet alles van de rest.

De inhoud heeft een vloer. Er is een subtiliteit die iedereen ooit verrast: een fr-kolom krimpt niet voorbij de minimale maat van zijn inhoud. Formeel is 1fr namelijk minmax(auto, 1fr), een bandbreedte met de inhoudsmaat als ondergrens. Meestal merk je daar niets van, tot één cel iets onkrimpbaars bevat: een lang woord zonder spaties, een URL, een brede afbeelding zonder de reset-regel uit 3.3. Dan weigert die kolom smaller te worden dan dat element, de andere kolommen betalen het gelag, en je "gelijke" kolommen zijn het niet meer. De diagnose is met de grid-overlay zo gemaakt (één spoor is breder dan zijn broers); de remedie is de inhoud krimpbaar maken, of, waar dat moet kunnen, expliciet minmax(0, 1fr) schrijven, de vorm zonder vloer. Dit mechanisme, inhoud die haar minimale maat opeist, heet in het jargon de min-content-maat, en het speelt bij flexbox net zo goed: ook flex-items weigeren standaard voorbij hun inhoud te krimpen.

Waarom gap bestaat. Vóór gap bouwde het web tussenruimte met marges op items, en dat model heeft een aangeboren fout: een marge weet niet of het item eerste, laatste of middelste is. Vandaar de klassieke lapmiddelen die je in oude code en oude tutorials ziet: een negatieve marge op de container die de buitenste marges "terugtrekt", of de :last-child-uitzondering uit het Niet zo van 4.3. Gap lost dit op waar het hoort, op containerniveau: de container wéét waar zijn tussenruimtes zitten, want hij legt de items zelf. Het is dezelfde ontwerpwijsheid als bij de cascade en de tokens: leg informatie op het niveau dat haar bezit.

Eén tel-detail tot slot. In 4.6 zei de theorieles dat lijnen tellen vanaf 1; er bestaan ook negatieve nummers, die vanaf het éínde tellen: -1 is de laatste lijn. `grid-column: 1 / -1` betekent dus "de volle breedte, hoeveel kolommen er ook zijn", een idioom dat je vaak zal tegenkomen en dat robuuster is dan het absolute eindnummer, omdat het meebeweegt wanneer het raster van maat verandert.

Veelgemaakte fouten

De fouten van deze week zie je meteen: een rij die een kolom blijft, een rooster dat overloopt. Wat je niet ziet, is op welk element de oorzaak zit. De overlay wel. De feedback op labo 4 verwijst naar deze nummers.

F4.1 - De wrapper breekt de layout. *Symptoom*: display: grid gezet, maar alles staat in één kolom of één reusachtige cel. *Oorzaak*: alleen dirécte kinderen zijn items; er zit een vergeten div tussen container en kaarten. *Remedie*: klik de grid-badge en lees de overlay; je ziet meteen wélk element als item meedoet. Verwijder de wrapper of maak hém de container.

F4.2 - Layoutproperties op het verkeerde niveau. *Symptoom*: justify-content op een item doet niets; flex: 1 op de container doet niets. *Oorzaak*: containerwoorden (display, gap, justify-, align-, grid-template-) horen op de ouder; alleen flex en de plaatsingsproperties (grid-column, grid-row) horen op items. *Remedie*: bij elke declaratie de vraag: organiseert dit, of ondergaat dit?

F4.3 - Justify en align verward. *Symptoom*: je centreert horizontaal en er verandert verticaal iets (of omgekeerd). *Oorzaak*: de as-logica losgelaten; justify volgt de hoofdas, align de kruisas, en bij flex-direction: column zijn die gedraaid. *Remedie*: teken (of denk) eerst de assen, kies dan het woord. D4.2 bestaat om dit één keer bewust mee te maken.

F4.4 - Percentagekolommen plus gap = overflow. *Symptoom*: drie kolommen van 33,33% en een gap, en de derde kolom valt eraf of er verschijnt een horizontale schuifbalk. *Oorzaak*: percentages negeren gaps (zie motorkap). *Remedie*: fr gebruiken; percentages in grid-templates zijn vrijwel altijd het verkeerde gereedschap.

F4.5 - De gestrekte afbeelding in de kaart. *Symptoom*: kaarten in een rij, en één afbeelding is vervormd of één kaart torent uit boven de rest. *Oorzaak*: align-items staat standaard op stretch en rekt items tot de hoogste buur, inclusief afbeeldingen (F3.8 in nieuwe gedaante). *Remedie*: bewuste uitlijning (align-items: start waar stretch niet gewenst is); voor afbeeldingen die een vak mooi moeten vullen komt object-fit in hoofdstuk 5.

F4.6 - Rijen dichtgetimmerd. *Symptoom*: grid-template-rows met vaste maten overal, en content die wordt afgeknipt of cellen vol lege lucht. *Oorzaak*: de impliciete grid niet vertrouwd; vaste hoogtes waar content de maat moest geven (F3.10 op rasterniveau). *Remedie*: beschrijf kolommen, laat rijen ontstaan; expliciete rijen alleen met een reden (grondplan 4.7).

F4.7 - Tussenruimte met marges geknutseld. *Symptoom*: marges op items plus een :last-child-uitzondering, of ongelijke randruimtes. *Oorzaak*: het pre-gap-model (zie motorkap), vaak overgenomen uit oude voorbeelden. *Remedie*: gap op de container; marges op items gereserveerd voor uitzonderingen.

F4.8 - Layout met verouderde middelen. *Symptoom*: float: left op kolommen, position: absolute om een footer onderaan te dwingen, een table om iets naast elkaar te krijgen, of het inline-block van labo 3 waar een flex-container hoort. *Oorzaak*: oude tutorials en oude trainingsdata (zie het AI-kader). *Remedie*: de as-vraag stellen en het juiste moderne gereedschap pakken; het oude arsenaal herkennen volstaat (appendix C).

F4.9 - Herschikt tot de DOM loog. *Symptoom*: visueel klopt alles, maar tabben door de pagina springt wild heen en weer. *Oorzaak*: te veel gericht geplaatst of ge-reversed; de visuele volgorde wijkt structureel af van de DOM-volgorde. *Remedie*: de HTML-volgorde herstellen zodat ze het verhaal van de pagina vertelt; plaatsing voor accenten, niet als correctie van slechte structuur.

F4.10 - De layout die "bijna" werkt is niet gecontroleerd. *Symptoom*: het oogt goed op jouw scherm, en breekt bij de eerste andere vensterbreedte of extra kaart. *Oorzaak*: gebouwd op één momentopname; niet getest met méér en minder content. *Remedie*: elke layout drie keer testen: venster smaller, venster breder, één item erbij en één eraf. Hoofdstuk 6 maakt hier een systeem van.

Met AI aan je zij: gegenereerde layouts lezen

Layout is de plek waar de ouderdom van trainingsdata het hardst doorschijnt. Vier patronen om op te screenen.

Legacy-technieken tussen het moderne. Modellen mengen tijdperken: een prima grid, en drie regels verder een float, een clearfix-class of een position: absolute om iets te bereiken wat gap of align-items in één woord doet. Werkend maar archeologisch. Jouw controle: elke float en elke absolute in een layoutcontext moet zich kunnen verantwoorden, en vrijwel geen enkele kan het.

Flexbox waar het raster hoorde. De trainingsdata bevat veel meer flexbox dan grid, dus modellen bouwen ook tweedimensionale layouts graag als geneste flex-rijen met percentages en flex-wrap. Het resultaat lijnt nét niet uit over rijen heen en vergt lapwerk. Stel zelf de as-vraag van 4.2 en herbouw als grid wat een raster is; het wordt vrijwel altijd korter én robuuster.

Wrapper-lasagne. Gegenereerde layouts stapelen graag containers: een row-div in een container-div in een wrapper-div, elk met een handvol declaraties. Elke laag is een kandidaat-F4.1 en een selector-verzwaring (2.4). Controlevraag per laag: welke layouttaak voert deze container uit? Geen antwoord: weg ermee.

Pixelrasters en marge-wiskunde. Kolommen van 320px, tussenruimtes als berekende marges (calc-uitdrukkingen incluis) waar `repeat(3, 1fr)` plus gap hetzelfde zegt. Dit is de pixel-soep van hoofdstuk 2 op rasterniveau, en het breekt bij de eerste andere schermbreedte. Herschrijven naar fr en gap is bijna mechanisch, en een uitstekende oefening, zie R4.3.

Oefeningen

Labo 4 bevat de opgaven van deze week, met startbestanden, nabouwen en de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen.

Drills (zonder AI aanbevolen)

- **D4.1** De as-vraag als quiz: tien layoutbeschrijvingen in woorden (navigatiebalk, fotorooster, formulierrij, prijstabel, tijdlijn, ...); kies telkens grid of flexbox en verantwoord in één zin. Bespreek daarna met een medestudent waar jullie verschillen.
- **D4.2** Bouw een flex-rij met vier gekleurde boxen en verken systematisch: alle waarden van justify-content, dan align-items, dan flex-direction: column, en voorspel vóór elke wijziging wat er gebeurt. Dit is de assen-drill; doe hem tot je voorspellingen kloppen.
- **D4.3** De navigatiebalk: ul als flex-container, gap, verticaal gecentreerd, klikvlakken uit D3.5, en één variant met de links rechts uitgelijnd naast een sitenaam links.
- **D4.4** Drie grids bouwen zonder te spieken: drie gelijke kolommen; een zijbalk van 12rem naast een 1fr-hoofdvlak; een raster van vier kolommen waarin één kaart twee kolommen beslaat (span) en één de volle breedte pakt (1 / -1).
- **D4.5** Het grondplan: sticky footer met grid-template-rows: auto 1fr auto op je site, en het perfect-centreren-recept op een aparte oefenpagina, één keer met grid en één keer met flexbox.
- **D4.6** Het kaartenrooster: zes kaarten uit D3.3 in een grid van drie kolommen, gelijke hoogtes, met binnen elke kaart een flex-column en `flex: 1` op de tekst, zodat de knop netjes onderaan staat.
- **D4.7** Herbouw de landingspagina van labo 3 met flexbox: main als flex-container, de twee helften als items met `flex: 1`, de navigatie als flex-rij met gap. Vergelijk het aantal regels met je inline-block-versie.

Debug en review (zonder AI)

- **R4.1** Een pagina met zes layoutfouten uit het onderdeel "veelgemaakte fouten" (minstens F4.1 tot en met F4.5): diagnosticeer elke fout mét de overlay als bewijs (screenshot of beschrijving van wat de badge of overlay toont), en herstel.
- **R4.2** Dezelfde kaartlayout, twee inzendingen: één in genest flexbox met percentages, één als grid met fr en gap. Voorspel voor drie veranderingen (kaart erbij, smaller venster, langere titel in één kaart) hoe elk zich gedraagt; verifieer; formuleer een advies.
- **R4.3** Een door AI gegenereerde paginalayout (aangeleverd): identificeer de vier patronen uit het AI-kader, herbouw de layout met grondplan-grid, flex-navigatie en kaartenrooster, en vergelijk het aantal regels CSS voor en na.

Bouwen en verantwoorden (met AI of zonder AI)

- **B4.1** Versie vier van je persoonlijke site: het volledige bouwplan van 4.8, grondplan-grid met sticky footer, flex-navigatie, een kaartenrooster voor je projecten of interesses, en geneste flex-columns binnen de kaarten. Elke container in je CSS krijgt een commentaarregel met zijn taak. Bij de AI-route: prompt, onbewerkte output, verbeterlijst; let dit keer op de vier patronen uit het AI-kader.

Verder lezen

- MDN, *CSS layout - Flexbox*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Flexbox
- MDN, *CSS layout - Grids*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Grids
- MDN, *Basic concepts of flexbox*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Basic_concepts_of_flexbox
- MDN, *Basic concepts of grid layout*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Basic_concepts_of_grid_layout
- MDN, *Relationship of grid layout with other layout methods*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Relationship_of_grid_layout_with_other_layout_methods

Hoofdstuk 5 - Layout 2: verdieping en plaatsing

In een notendop

Vorige week kreeg je flexbox en grid in handen; deze week bouw je de gereedschapskist af. Eerst gaan beide gereedschappen een laag dieper: rasters die je **uittekent** in plaats van uittelt (named areas), flexbox die over meerdere regels mag breken en waarvan je het groeien en krimpen echt begrijpt, en items die individueel mogen afwijken van wat hun container beslist. Dan komt er een derde soort gereedschap bij, voor een ander soort vraag: **positioning**, waarmee een element uit de gewone flow stapt, en z-index, dat beslist wie bovenop ligt. Tot slot twee schulden die we inlossen: afbeeldingen die hun vak vullen zonder te vervormen, en de selector `:has()` die naar binnen kijkt. Elke layoutvraag heeft een passend gereedschap; na deze week ken je ze allemaal, en kiezen is, meer nog dan vorige week, het echte onderwerp.

Wat je na dit hoofdstuk kan

In labo 4 bouwde je het grondplan, de navigatiebalk en het kaartenrooster. Het werkte, en toch bleven er vragen liggen. Waarom waren de drie kolommen van oefening 1 exact even breed, ook met verschillende cijfers erin? Wat doe je met een kaartenrooster dat op een half venster te smal wordt? Hoe krijg je een "nieuw"-label óp een kaart, in plaats van ernaast? En hoe hou je de navigatiebalk in beeld terwijl de pagina scrolt? Dat zijn de vragen van deze week.

Na dit hoofdstuk beheers je het volledige layoutarsenaal van dit vak: je bouwt een grondplan met zijbalk dat je letterlijk kan lezen in je CSS, kaarten met uniforme afbeeldingen die niet vervormen, een header die meescrolt tot bovenaan en daar blijft, en overlappende elementen waarvan jij de stapelvolgorde bepaalt. Dat is de gereedschapskist waaruit je op de toets in november kiest. Even belangrijk: je kan bij elke layoutvraag zeggen of ze om een container, een item, of om plaatsing vraagt, en dus welk gereedschap erbij hoort.

Het hoofdstuk heeft drie delen. Eerst de verdieping van wat je al kent: named areas (5.1), flexbox voorbij `flex: 1` (5.2) en items die afwijken (5.3). Dan het nieuwe gereedschap: positioning (5.4) en stapelen (5.5). Ten slotte de afwerking van de kaart en van het rooster: afbeeldingen in het gelid (5.6), de ouder selecteren met `:has()` (5.7), en plaatsing lezen in DevTools (5.8). In labo 5 zet je alles in: in nabouwen, en op je eigen site.

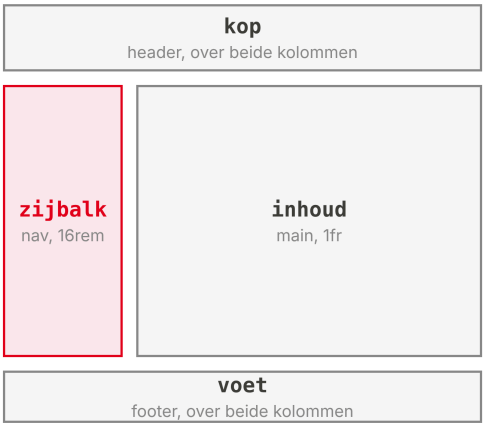
5.1 Named areas: het raster uittekenen

grid-template-areas

```
body {
  display: grid;
  grid-template-columns: 16rem 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "kop     kop"
    "zijbalk inhoud"
    "voet    voet";
}

header { grid-area: kop; }
nav    { grid-area: zijbalk; }
main   { grid-area: inhoud; }
footer { grid-area: voet; }
```

de plattegrond



lees de template hardop en je hoort de pagina; elk gebied is een rechthoek, elk element kiest met grid-area zijn gebied

Figuur 5.1 - Named areas: de grid-template-areas in de CSS is letterlijk de plattegrond van de pagina. Elke naam in de template is een gebied, en elk element kiest met grid-area zijn gebied.

Voor eenvoudige rasters zijn lijnnummers (4.6) prima. Maar zodra een grondplan meer wordt dan een paar kolommen (header over de volle breedte, zijbalk links, content rechts, footer eronder) worden nummers een puzzel die je in je hoofd moet meerekenen. Grid heeft daarvoor een tweede plaatsingstaal die je layout leesbaar maakt als een plattegrond: **named areas**.

Het werkt in twee stappen. Eerst geef je elk te plaatsen item een naam met **grid-area**; daarna teken je op de container met **grid-template-areas** het raster uit, rij per rij, met die namen:

```
body {
  display: grid;
  grid-template-columns: 16rem 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
  grid-template-areas:
    "kop     kop"
    "zijbalk inhoud"
    "voet    voet";
}

header { grid-area: kop; }
nav    { grid-area: zijbalk; }
main   { grid-area: inhoud; }
footer { grid-area: voet; }
```

Lees de template-areas hardop en je hóórt de pagina: bovenaan een kop over beide kolommen, daaronder zijbalk naast inhoud, onderaan een voet over beide kolommen. Een naam herhalen over meerdere cellen laat het gebied uitspreiden (zoals kop en voet hier over twee kolommen); een punt (`.`) markeert een bewust lege cel. Eén beperking bewaakt de leesbaarheid: **elk gebied moet een rechthoek vormen**. Een L-vormig gebied kan niet, en dat is maar goed ook, want een plattegrond met L-stukken is geen plattegrond meer. Vergeet je dat, of tellen je rijen een ongelijk aantal cellen, dan is de hele declaratie ongeldig en valt de browser stil terug op de impliciete plaatsing (F5.6).

Herken je de rijen? `auto 1fr auto` is het grondplan van 4.7 en van labo 4, met één kolom erbij. De sticky footer blijft werken zoals hij werkte; de areas voegen er alleen de plattegrond aan toe.

Wanneer kies je areas boven lijnnummers? **Vuistregel:** nummers voor incidentele plaatsing (één kaart die twee kolommen spant in een verder impliciet rooster), areas voor grondplannen (de vaste structuur van een pagina of een complexe component). Areas dwingen je bovendien om je gebieden te benoemen, en namen zijn documentatie die niet kan verouderen; de les keert terug van classnamen (1.12) en tokens (2.9): `grid-area: zijbalk` liegt nooit over zijn bedoeling. En hou één belofte tegoed: in hoofdstuk 6 zal je zien dat je hetzelfde stel gebieden per schermbreedte compleet anders kan neerleggen door alleen de template-areas te herschrijven. De HTML blijft onaangeroerd, de plattegrond verschuift. Dat is de scheiding van structuur en vormgeving (2.1) op haar sterkst.

Geneste grids horen bij ditzelfde verhaal: een area is een gewone box, en niets belet die box om zelf weer grid- of flex-container te zijn voor zíjn kinderen (4.1). Zo blijft elke container klein en leesbaar: het grondplan kent alleen zijn gebieden, het kaartenrooster alleen zijn kaarten, de kaart alleen haar binnenkant. Wie zich afvraagt of een genest raster kan uitlijnen op het raster van zijn grootouder: dat kan, het heet subgrid, en het bestaat precies voor dat probleem. Je hebt het dit semester niet nodig, maar herken de naam wanneer je hem tegenkomt; het is jonge, inmiddels breed ondersteunde CSS.

5.2 Flexbox verdiept: wrap, en wat flex: 1 echt betekent

Twee zaken bleven vorige week bewust liggen. Het eerste is de vraag wat er gebeurt als flex-items niet meer passen. Standaard: niets goeds. Flexbox perst alles op één regel, items krimpen tot hun minimum en daarna loopt de boel over. Je zag het in labo 4: op een half venster werden de drie kaarten van het dierenasiel zo smal dat de tekst per woord brak. De property **flex-wrap** geeft de container toestemming om te breken:

```
.galerij {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.galerij > * {
  flex: 1 1 16rem; /* grow shrink basis */
}
```

Met `wrap` schuiven items die niet passen naar een volgende regel, en elke regel verdeelt zijn eigen ruimte. Let op wat hier conceptueel gebeurt: dit is nog altijd **eendimensionale layout**, een reeks onafhankelijke regels, en geen raster. De items van regel twee lijnen niet uit met die van regel één. Precies dat verschil maakte in 4.2 het onderscheid met grid, en nu zie je het.

Het tweede is de verkorte vorm zelf. `flex: 1` uit 4.4 is een samentrekking van drie properties, en het voorbeeld hierboven toont ze alle drie. **flex-basis** is de startmaat van het item op de hoofdas: het vertrekpunt van de onderhandeling, vóór groeien of krimpen, en daarmee de flexbox-eigen vervanger van width, die je in een flex-context beter vermijdt (F5.7). **flex-grow** zegt hoe gretig het item de restruimte opeist: 0 is niet, 1 is een gelijk aandeel, 2 een dubbel aandeel. **flex-shrink** zegt hoe gewillig het inlevert bij plaatsgebrek, standaard 1: iedereen krimpt mee. `flex: 1` voluit is `flex: 1 1 0`: groei gretig, krimp gewillig, en start vanaf nul, zodat de verdeling puur op de groeifactoren gebeurt. Dát is waarom de drie kolommen van labo 4 exact even breed waren, hoe groot het cijfer erin ook stond: hun inhoud deed niet mee aan de verdeling, want elke kolom begon vanaf nul.

De combinatie in het voorbeeld, wrap plus `flex: 1 1 16rem`, mag je uit het hoofd kennen: elk item wil zo'n 16rem breed zijn; past er nog één bij, dan groeien ze samen dicht; past het niet meer, dan breekt de regel. Een galerij die zich aanpast aan elke schermbreedte, zonder één media query. Onthoud dit patroon; wanneer hoofdstuk 6 zijn grid-tegenhanger toont (6.8), heb je twee gereedschappen voor zelforganiserende roosters en een scherp oog voor het verschil.

5.3 Items die afwijken: align-self, order en de auto-marge

Containers verdelen collectief, maar soms verdient één item een uitzondering. Daar bestaan drie instrumenten voor, elk op het item zelf.

align-self overschrijft de align-items van de container voor dit ene item: zelfde waarden, zelfde kruisas-logica. Denk aan de knop in de kaarten van labo 4. Daar zette je `align-items: flex-start` op de kaart, zodat de knop niet over de volle breedte werd uitgerekt; maar dat trof meteen ook de titel en de tekst. Met `align-self: flex-start` op de knop alleen, blijft de rest van de kaart stretchen en wijkt enkel de knop af. Het is dezelfde beweging als bij `flex` in 4.4: de container beslist voor allemaal, één item mag voor zichzelf spreken. In grid bestaat ook justify-self, voor de andere as; flexbox heeft die niet nodig, want langs de hoofdas regeert daar de ruimteverdeling zelf.

De **auto-marge** is het sluwste instrument van de drie, en je kent het principe al uit 3.6: auto betekent "verdeel de restruimte hier". In een flex-container werkt dat langs de hoofdas, per item, en het klassieke gebruik is de navigatiebalk waarin één element naar de overkant moet:

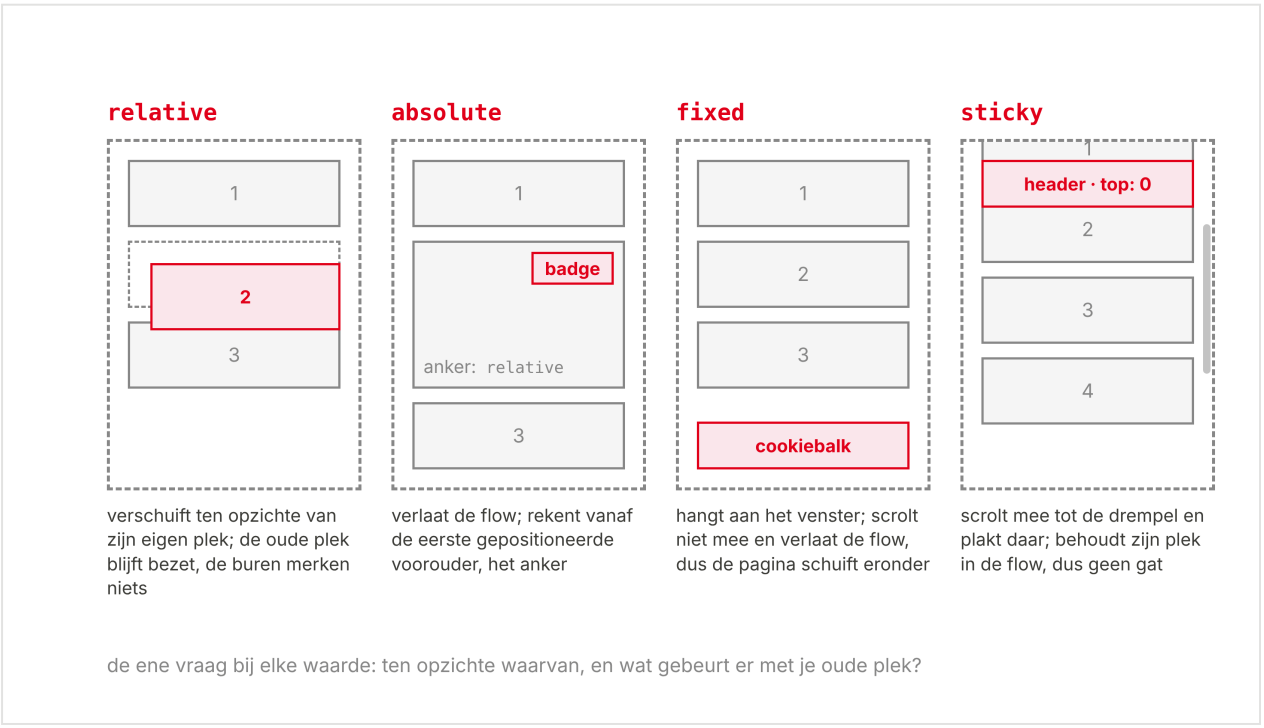
```css
.balk {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.balk .logo {
  margin-right: auto; /* pushes everything after it to the far end */
}
```

Logo links, alle links rechts, één declaratie. De auto-marge slokt alle restruimte op aan die ene kant: het is justify-content op itemniveau, en je ziet hem in professionele navigaties overal. In labo 4 loste je dit op met `justify-content: space-between` op de header, en voor twee kinderen is dat even goed. De auto-marge wint zodra er een derde item bijkomt, een zoekveld of een knop *Inloggen*: space-between smeert de ruimte over alle tussenruimtes uit, de auto-marge duwt precies op één plek.

order ten slotte herschikt items visueel los van hun DOM-volgorde: elk item heeft standaard 0, lagere nummers komen eerst. Het is het flex-equivalent van gerichte grid-plaatsing, en er geldt exact dezelfde waarschuwing als in 4.6: toetsenbord en schermlezer volgen de DOM. Gebruik order voor een klein visueel accent met een goede reden; wie structureel herschikt, moet zijn HTML herzien (F5.8).

5.4 Positioning: uit de flow stappen



Figuur 5.2 - De vier waarden van position naast elkaar: relative verschuift maar houdt zijn plek, absolute verlaat de flow en rekent vanaf zijn anker, fixed hangt aan het venster, sticky scrolt mee tot een drempel en plakt daar.

Alles tot nu toe, flow, flexbox en grid, schikt elementen *ten opzichte van elkaar*: iedereen heeft een plek, niemand overlapt. Maar sommige ontwerpen vragen precies dat: een badge die op de hoek van een kaart ligt, een bijschrift óver een foto, een navigatie die blijft staan terwijl de pagina scrolt. Daarvoor bestaat het derde en laatste gereedschap: de **position**-property, waarmee een element, deels of geheel, uit de gewone flow stapt.

Elk element start op `position: static`: gewoon meedoen met de flow. Elke andere waarde maakt het een *gepositioneerd element*, dat je vervolgens kan verplaatsen met **top**, **right**, **bottom** en **left** (of hun verkorte vorm **inset**). De vier waarden verschillen in één vraag: *ten opzichte waarvan, en wat gebeurt er met je oude plek?*

relative is de mildste: het element verschuift ten opzichte van zijn eigen flow-positie, maar zijn oude plek blijft bezet; de buren merken niets. Op zichzelf gebruik je het zelden voor verplaatsing. Zijn hoofdrol is een andere, die zo meteen volgt.

absolute is de radicaalste: het element verlaat de flow volledig (de buren schuiven bij alsof het niet bestaat) en positioneert zich ten opzichte van zijn dichtstbijzijnde gepositioneerde voorouder. En dáár ligt de sleutel tot vrijwel al het praktische positioneringswerk, dus lees deze zin twee keer: **een absolute element zoekt omhoog in de boom naar de eerste voorouder die zelf gepositioneerd is, en rekent vanaf diens randen.** Is er geen, dan rekent het vanaf het document zelf, en dat is vrijwel nooit wat je wil (F5.1). Vandaar het vaste duo dat je overal zal zien en schrijven:

```
.kaart {
  position: relative; /* anchor: does nothing visually, but... */
}

.kaart .badge {
  position: absolute; /* ...this now positions relative to the card */
  top: 0.75rem;
  right: 0.75rem;
}
```

Dit is het **badge-op-kaart-patroon**: relative op de ouder als anker (het verplaatst niets, het kent alleen ankerstatus toe), absolute op het kind dat erop moet liggen. "Nieuw"-labels, sluitknoppen van dialoogvensters, bijschriften over foto's: allemaal dit duo.

fixed verankert aan het browservenster zelf: het element scrolt niet mee, wat er ook gebeurt. Het klassieke gereedschap voor cookiebalken en zwevende knoppen. Maar voor het beroemdste gebruiksgeval, de blijvende navigatiebalk, is er iets beters, en het verschil is de moeite om naast elkaar te zetten:

Niet zo:

```
header {
  position: fixed;
  top: 0;
  width: 100%;
}

body {
  padding-top: 4rem; /* guess the header height to fill the gap */
}
```

Zo wel:

```
header {
  position: sticky;
  top: 0; /* required: sticky needs a threshold to stick to */
}
```

sticky is de jongste en slimste van de vier: een hybride die zich als relative gedraagt tot het scrollen hem tegen een opgegeven rand duwt, en daar blijft plakken zolang zijn ouder in beeld is. De header scrolt gewoon mee met de pagina, en klikt vast zodra hij de bovenrand raakt. Een fixed element verlaat de flow, dus de pagina schuift eronder en je moet het gat zelf opvullen, met een padding die je gokt en die breekt zodra de header hoger wordt. Een sticky element behoudt zijn plek in de flow: geen gat, geen gok, en geen JavaScript zoals in oude tutorials. Onthoud de twee voorwaarden: een drempel (top, bottom, ...) is verplicht, want sticky zonder top doet niets (F5.2), en het plakken stopt waar de ouder ophoudt, want een element plakt binnen zijn eigen box. Waarom sticky soms toch niet werkt terwijl beide kloppen, is motorkap-stof.

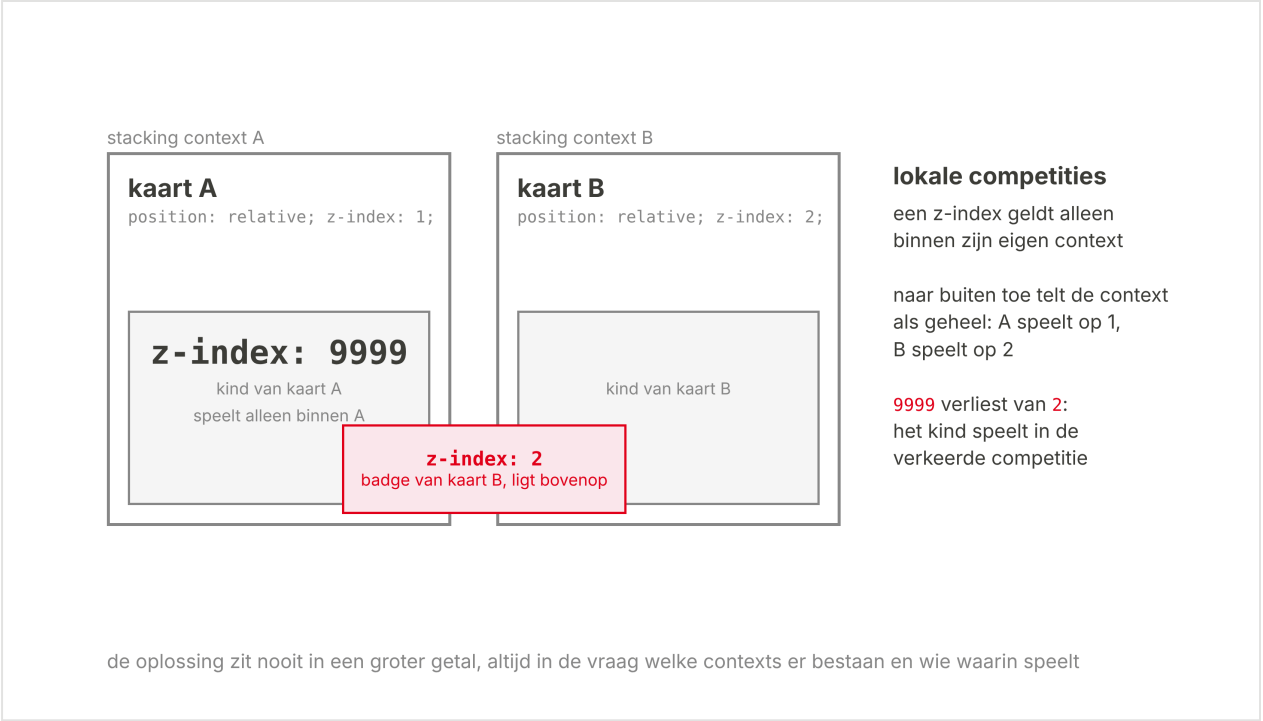
Het gebruikskader tot slot, en dat is streng: **positioning is plaatsingsgereedschap, geen layoutgereedschap.** Paginastructuur bouw je met grid en flexbox; position gebruik je voor het kleine werk erbovenop: overlays, badges, plakkende balken. Oude codebases en oude tutorials (en dus ook gegenereerde code, zie het AI-kader) bouwen hele kolommenlayouts met absolute; jij herkent dat voortaan als het archeologische signaal dat het is (F5.5).

5.5 Stapelen: z-index

Zodra elementen overlappen, ontstaat een nieuwe vraag: wie ligt bovenop? Standaard beslist de DOM-volgorde: wat later komt, ligt hoger. Maar voor gepositioneerde elementen neem jij het over met **z-index**: een getal, hoger wint. De naam zegt het al: na de x-as en de y-as van het scherm is dit de z-as, de denkbeeldige lijn van het scherm naar jouw neus.

```
.kaart .badge {
  position: absolute;
  top: 0.75rem;
  right: 0.75rem;
  z-index: 1; /* above the card image */
}
```

Twee regels voorkomen het grootste deel van de z-index-frustratie. Eén: **z-index werkt alleen op gepositioneerde elementen** (en op grid- en flex-items). Op een gewoon flow-element wordt hij stil genegeerd, het "faalt stil"-motief in een nieuw jasje (F5.3). Twee: gebruik kleine, betekenisvolle getallen (1, 2, 10), geen 9999. De duizendtallen die je online ziet, zijn versteende wanhoop uit escalatie-oorlogen, en ze helpen niet eens, want z-index is geen wereldranglijst: elementen stapelen binnen hun **stacking context**, een soort lokale competitie waarbinnen de nummers gelden.



Figuur 5.3 - Stacking contexts: een z-index geldt alleen binnen zijn eigen context. Het kind met z-index 9999 in kaart A komt nooit boven de badge met z-index 2 in kaart B, want naar buiten toe telt alleen de context van kaart A als geheel, op z-index 1.

Waarom een z-index van 9999 kan verliezen van een 2, is het hoofdstuk van het motorkap-kader; voor de praktijk volstaat de vuistregel dat je stapels klein en lokaal houdt, dan blijft de competitie overzichtelijk.

5.6 Afbeeldingen in het gelid: object-fit en aspect-ratio

Tijd om twee schulden uit eerdere hoofdstukken in te lossen: de vervormde afbeelding (F3.8) en de kaart die uit het gelid springt omdat haar foto een andere verhouding heeft (F4.5). Het onderliggende conflict is altijd hetzelfde: het vák heeft een maat, de fóto heeft een verhouding, en die twee botsen. Twee moderne properties lossen het definitief op.

aspect-ratio geeft een element een vaste verhouding: `aspect-ratio: 16 / 9` maakt de box altijd zestien staat tot negen, welke breedte de layout haar ook geeft; de hoogte volgt vanzelf. Dit is dezelfde verhoudingslogica als de width- en height-attributen van 3.9, maar dan als CSS-gereedschap dat je op elk element kan zetten.

object-fit vertelt de afbeelding vervolgens hoe ze met haar vak moet omgaan wanneer de verhoudingen niet overeenkomen. De standaardwaarde (`fill`) rekt op tot het past: de vervorming die je kent. `contain` past de hele foto in het vak en laat randen open. En de waarde die je vrijwel altijd wil is **cover**: vul het vak volledig en snij bij wat niet past, zonder vervorming. Het is het gedrag dat je van professionele sites kent, op een echt, semantisch img-element mét alt-tekst:

```
.kaart img {
  width: 100%;
  aspect-ratio: 3 / 2;
  object-fit: cover;
}
```

Drie declaraties, en elk kaartenrooster krijgt uniforme, onvervormde afbeeldingen, wat de fotografen ook aanleverden. Dit **drieluik** gaat rechtstreeks je snippets-verzameling in, naast het kaartje (3.5), het grondplan (4.7) en het centreer-recept (4.7). Er bestaat een broertje object-position dat bepaalt welk deel bij het bijsnijden centraal blijft; de standaard, het midden, is meestal juist.

En hiermee lossen we meteen de belofte van 1.11 in: het stijve iframe. Een geplakte YouTube-embed komt met vaste width- en height-attributen en trekt zich niets aan van zijn container, het videobroertje van F3.8. Wie oudere tutorials leest, vindt daarvoor een berucht knutselwerk; zet het naast het moderne recept en je ziet wat aspect-ratio heeft opgelost:

Niet zo:

```
.video {
  position: relative;
  padding-top: 56.25%; /* 9 / 16 as a percentage of the width */
}

.video iframe {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
}
```

Zo wel:

```
.video iframe {
  width: 100%;
  aspect-ratio: 16 / 9;
  height: auto; /* override the pasted height attribute */
}
```

De oude truc misbruikt een percentage-padding (die tegen de breedte rekent, 3.2) om een hoogte te maken, en heeft dan absolute positioning nodig om het iframe erin te leggen: twee gereedschappen voor iets wat één property zegt. Het nieuwe recept is dat van de afbeeldingen, minus object-fit, want

een iframe heeft geen eigen foto om bij te snijden; het schaalt gewoon mee. De embed vult nu de volle breedte van zijn container en houdt de verhouding van het videobeeld vast, op elk scherm. Herken de oude vorm, vervang ze.

5.7 De ouder selecteren: :has() in gebruik

Nog één stuk gereedschap, aangekondigd in 2.5, en het hoort precies hier thuis, want het lost een layoutprobleem op dat je deze week gegarandeerd tegenkomt. Je kaartenrooster bevat kaarten mét afbeelding en kaarten zonder, en die vragen om andere styling: de tekstkaart wil bijvoorbeeld meer padding bovenaan, of een accentkleur. Tot voor kort kon dat alleen door in de HTML een class toe te voegen (`class="kaart-zonder-afbeelding"`): handwerk dat vergeten wordt, en dat vormgevingsinformatie in de structuurlaag lekt.

:has() draait de selecteerrichting om: waar elke andere selector omlaag of opzij kijkt, kijkt :has() naar binnen. Het selecteert een element op basis van wat het bevat:

```
.kaart:has(img) {
  padding: 0; /* image cards: the picture runs to the edges */
}

.kaart:not(:has(img)) {
  border-top: 0.25rem solid var(--color-accent);
}
```

Lees: elke kaart die een img bevat; elke kaart die er geen bevat (de `:not()` eromheen ken je conceptueel al: hij ontkent). Geen extra classes, geen HTML-wijziging: de stylesheet reageert op de werkelijkheid van de inhoud. Dat is waarom :has() jarenlang bovenaan elke verlanglijst stond en waarom het, sinds het overal werkt, zo snel standaardgereedschap werd. Gebruik het zoals hier, gericht en leesbaar, voor echte inhoudsvariatie, en het maakt je CSS eenvoudiger in plaats van slimmer dan goed is. In hoofdstuk 9 bewijst het nog een tweede keer zijn waarde, wanneer formulieren erom vragen ("het veld dat een fout bevat").

5.8 Plaatsing lezen in DevTools

Het instrument van deze week bouwt voort op de badges van 4.9. Gepositioneerde elementen verraden zich in het boxmodel-diagram (3.10) door een extra ring, *position*, met de berekende top- en left-waarden, en het Computed-tabblad beantwoordt bij twijfel de ankervraag: de property *position* toont wat het element denkt te zijn, en wie omhoog klimt door de Elements-boom vindt de eerste voorouder met een position anders dan static, het werkelijke anker van je absolute element. Klopt de badge-positie niet, dan is in negen van de tien gevallen dát de diagnose: **het anker is niet wie jij dacht** (F5.1).

Voor sticky is er een eigen kijktruc: scroll traag en kijk in de Elements-boom. Het element krijgt nergens een class of attribuut bij (sticky is puur visueel), dus je debugt het door de voorwaarden af te vinken. Staat er een drempel (top)? Heeft de ouder hoogte genoeg om ín te plakken? En, de sluipmoordenaar uit het motorkap-kader: staat er ergens tussen element en venster een overflow die het scrollgedrag kaapt? **Drie vinkjes**, en elke sticky-bug is gevonden.

Onder de motorkap: ankers, stapels en plakkers

Dit kader gaat dieper dan de theorieles. Je hebt het niet nodig om de oefeningen te maken, wel om de drie beroemdste positioning-mysteries te kunnen oplossen.

Het containing block, precies. De theorieles zei: absolute rekent vanaf de dichtstbijzijnde gepositioneerde voorouder. De precieze naam voor "datgene waartegen gerekend wordt" is het *containing block*, en elk element heeft er één, ook flow-elementen (daar is het simpelweg de content-box van de ouder). Voor absolute elementen wordt het containing block gevormd door de padding-rand van de eerste voorouder met een position anders dan static; ontbreekt die, dan valt het terug op het *initial containing block*, de box van het document zelf. Twee praktische gevolgen. Ten eerste: `top: 0` op een absolute element betekent "tegen de binnenkant van de padding van mijn anker", niet tegen zijn border; zet je anker padding, dan zie je het verschil. Ten tweede, en verrassender: ook enkele properties die niets met position te maken hebben, promoveren een voorouder tot containing block, waaronder transform en filter (hoofdstuk 8). Vandaar het klassieke raadsel "mijn fixed element scrolt ineens mee": een voorouder kreeg ergens een transform, werd daardoor containing block, en het fixed element is nu aan hém verankerd in plaats van aan het venster. Wie dit kader gelezen heeft, herkent het in één oogopslag.

Stacking contexts: de lokale competities. Waarom verliest z-index 9999 soms van z-index 2? Omdat z-index geen wereldranglijst is maar een reeks lokale competities. Bepaalde elementen openen een *stacking context*: een afgesloten stapelwereldje waarbinnen de z-indexen van hun nakomelingen worden uitgevochten. De bekendste openers: een gepositioneerd element mét z-index, een element met opacity onder 1, en, alweer, transform en filter. Binnen zo'n context stapelen de kinderen onderling; naar buiten toe telt alleen de context als geheel, op de z-index van zijn opener. Concreet, zoals in de figuur van 5.5: krijgt kaart A om wat voor reden dan ook een eigen stacking context met z-index 1, dan kan geen enkel kind van kaart A, hoe astronomisch zijn z-index ook, ooit boven een element uit kaart B met z-index 2 komen. Het kind speelt in de verkeerde competitie. De 9999-reflex is dus niet alleen lelijk maar zinloos: de oplossing zit nooit in een groter getal, altijd in de vraag welke contexts er bestaan en wie waarin speelt. DevTools kan stacking contexts tonen (het Layers-paneel, of een extensie); vaker nog volstaat het besef dat opacity en transform stille context-openers zijn.

Waarom sticky soms weigert. De twee lesvoorwaarden (drempel plus ruimte in de ouder) hebben een derde, stille broer: sticky plakt tegen de dichtstbijzijnde *scrollende* voorouder, en een voorouder met overflow hidden, auto of scroll wordt zo'n scrollcontainer, ook als hij nooit zichtbaar scrolt. Eén `overflow: hidden` ergens tussen jouw header en het venster (ooit toegevoegd om een ander probleempje weg te knippen) en de header plakt voortaan tegen dát element in plaats van tegen het venster, wat er meestal uitziet als "sticky doet gewoon niets". De diagnose-wandeling: klim vanaf het

sticky element omhoog en controleer elke voorouder op overflow. Het is een schoolvoorbeeld van hoe een reparatie op de ene plek een mysterie op de andere baart, en van waarom je `overflow: hidden` alleen met een reden zet (3.8).

Veelgemaakte fouten

De fouten van deze week zijn plaatsingsfouten: iets ligt niet waar je het legde, of plakt niet waar het moest plakken. In negen van de tien gevallen is het antwoord een voorouder die niet is wat jij dacht. De feedback op labo 5 verwijst naar deze nummers.

F5.1 - Absolute zonder anker. *Symptoom*: de badge staat niet op de kaart maar ergens in de hoek van de pagina. *Oorzaak*: geen gepositioneerde voorouder; het element rekent vanaf het document. *Remedie*: `position: relative` op de bedoelde ouder, het ankerduo van 5.4. Controleer het werkelijke anker via het Computed-tabblad (5.8).

F5.2 - Sticky zonder drempel of zonder ruimte. *Symptoom*: `position: sticky` en er gebeurt niets. *Oorzaak*: geen top of bottom opgegeven, of de ouder is precies zo hoog als het element zelf (nergens om ín te plakken), of de stille derde: een overflow op een voorouder (motorkap). *Remedie*: de drie vinkjes van 5.8.

F5.3 - Z-index op een flow-element. *Symptoom*: z-index gezet, stapelvolgorde verandert niet. *Oorzaak*: het element is niet gepositioneerd (en geen grid- of flex-item); z-index wordt stil genegeerd. *Remedie*: `position: relative` erbij als het element enkel ankerstatus en stapelrecht nodig heeft.

F5.4 - De z-index-wapenwedloop. *Symptoom*: een codebase vol 999 en 99999, en nog ligt het verkeerde bovenaan. *Oorzaak*: stacking contexts genegeerd; de nummers spelen in verschillende competities (motorkap). *Remedie*: kleine getallen, en bij een hardnekkig geval de context-wandeling: wie opent er een stapelwereldje tussen mij en mijn rivaal?

F5.5 - Positioning als layoutgereedschap. *Symptoom*: kolommen, headers en footers neergelegd met absolute en handgemeten top-waarden; alles verschuift bij de eerste contentwijziging. *Oorzaak*: het gereedschapskader van 5.4 genegeerd, vaak overgeschreven uit oude voorbeelden. *Remedie*: structuur met grid en flexbox, position alleen voor het plaatsingswerk erbovenop.

F5.6 - Niet-rechthoekige areas. *Symptoom*: de hele grid-template-areas wordt genegeerd; de layout valt terug op impliciete plaatsing. *Oorzaak*: een gebied vormt geen rechthoek, of de rijen tellen ongelijke aantallen cellen; de declaratie is dan in haar geheel ongeldig (het "faalt stil"-motief van 2.10). *Remedie*: rijen uitlijnen in je code zoals in 5.1; de grid-overlay toont meteen of de browser jouw plattegrond tekent dan wel zijn eigen fallback.

F5.7 - Flex-basis en width door elkaar. *Symptoom*: widths op flex-items die "niet kloppen" of onverwacht genegeerd worden. *Oorzaak*: in een flex-context onderhandelt flex-basis, en width verliest die onderhandeling deels. *Remedie*: in flex-items stuur je de maat via de basis (derde waarde van de verkorte vorm), niet via width.

F5.8 - Herschikt tot de DOM loog, reprise. *Symptoom*: visueel perfect, tabvolgorde chaotisch. *Oorzaak*: order en gerichte plaatsing als structuurcorrectie gebruikt. *Remedie*: zelfde als F4.9: herstel de HTML-volgorde; order is een accent, geen verbouwing.

F5.9 - Object-fit vergeten of zonder maat. *Symptoom*: vervormde kaartafbeeldingen (fill), of object-fit gezet en geen effect. *Oorzaak*: het vak heeft geen eigen maat; object-fit heeft pas werk wanneer box en foto van verhouding verschillen. *Remedie*: het drieluik van 5.6: width, aspect-ratio, `object-fit: cover`.

F5.10 - Content onder de plakkende header. *Symptoom*: interne links (naar een `#id`) scrollen hun doel tot achter de sticky header. *Oorzaak*: de header ligt over de bovenrand van het scrollgebied; het doel scrolt naar de rand, niet naar het zichtbare deel. *Remedie*: `scroll-padding-top` op het html-element, ter grootte van de header, een kleine moderne property die precies hiervoor bestaat.

Met AI aan je zij: gegenereerd plaatsingswerk lezen

Positioning is het gereedschap waar gegenereerde code het meest naar grijpt, en het vaakst ten onrechte. Vier patronen om op te screenen.

Absolute als duizenddingendoekje. Het hardnekkigste patroon in gegenereerde layouts: `position: absolute` met handgemeten pixelwaarden voor dingen die uitlijning (4.7), een auto-marge (5.3) of sticky (5.4) in één declaratie doen. Centreren met `top: 50%` en een negatieve marge is de klassieker. Jouw controlevraag bij elke absolute: welk overlap-probleem lost dit op? Geen overlap, verkeerd gereedschap.

Z-index-inflatie. Modellen erven de 9999-cultuur van hun trainingsdata en strooien preventief grote z-indexen, soms op elementen die niet eens gepositioneerd zijn (F5.3). Vervang door kleine getallen en verwijder wat zonder effect blijkt; wat overblijft, vertelt je waar de echte stapels zitten.

Fixed waar sticky hoort. Gegenereerde navigatiebalken zijn opvallend vaak `position: fixed` met een handmatige padding-top op body om het gat te compenseren: het Niet zo van 5.4, twee declaraties lapwerk voor wat sticky in één doet, zonder gat. Ook de scroll-padding-fix van F5.10 ontbreekt in gegenereerde code vrijwel altijd; leg een pagina met interne links ernaast en je vangt het meteen.

Verhoudingen genegeerd. Kaartroosters uit een model komen geregeld zonder object-fit en aspect-ratio: prima met de vierkante placeholderfoto's van het voorbeeld, vervormd bij jouw echte afbeeldingen. Het drieluik van 5.6 is de standaardcorrectie, en een mooi voorbeeld van een fout die pas zichtbaar wordt met echte content. Dat is meteen de les: beoordeel gegenereerde layouts nooit alleen met de placeholder-inhoud waarmee ze geleverd worden.

Oefeningen

Labo 5 bevat de opgaven van deze week, met startbestanden, nabouwen en de inleverinstructies. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen.

Drills (zonder AI aanbevolen)

- **D5.1** Herbouw je sticky-footer-grondplan uit D4.5 met named areas, en breid uit met een zijbalk van 16rem (het plan uit 5.1). Teken de plattegrond eerst op papier, dan pas in code.
- **D5.2** De zelforganiserende galerij: flex-wrap plus `flex: 1 1 16rem` met acht kaarten. Versmal het venster en beschrijf in commentaar wat er bij elke breuk gebeurt, en waarom regel twee niet uitlijnt met regel één.
- **D5.3** Voorspel de verdeling: vijf flex-rijen op papier, met gegeven grow-, shrink- en basis-waarden per item. Bereken wie hoeveel krijgt, bouw na, meet in DevTools.
- **D5.4** Het badge-op-kaart-duo: bouw drie varianten (label linksboven, sluitknop rechtsboven, bijschrift over de onderrand van een foto) op de kaarten van D3.3.
- **D5.5** De plakkende header: maak de navigatiebalk van je site sticky, met scroll-padding voor interne links (F5.10). Test met een lange pagina.
- **D5.6** Uniforme kaartafbeeldingen: pas het drieluik van 5.6 toe op een rooster met opzettelijk uiteenlopende bronafbeeldingen (portret, panorama, vierkant).
- **D5.7** Twee :has()-regels op je kaartenrooster: beeldkaarten zonder padding, tekstkaarten met accentstreep, zonder één class toe te voegen.

Debug en review (zonder AI)

- **R5.1** Een pagina met acht plaatsingsfouten uit het onderdeel "veelgemaakte fouten" (minstens F5.1, F5.2, F5.4, F5.6 en F5.9): diagnosticeer met het bijbehorende DevTools-bewijs en herstel.
- **R5.2** Het z-index-mysterie: een gegeven pagina waar een element met z-index 9999 onder een element met z-index 2 ligt. Verklaar exact waarom (welke stacking context, wie opent hem) en los op zonder ook maar één z-index te verhogen.
- **R5.3** Een door AI gegenereerde landingspagina (aangeleverd): identificeer de vier patronen uit het AI-kader, en herschrijf het plaatsingswerk met de gereedschappen van dit hoofdstuk. Tel de verwijderde declaraties.

Bouwen en verantwoorden (met AI of zonder AI)

- **B5.1** Versie vijf van je persoonlijke site: grondplan met named areas (met of zonder zijbalk, jouw ontwerpkeuze, beargumenteerd in commentaar), sticky navigatie met scroll-padding, minstens één badge-op-kaart, uniforme kaartafbeeldingen via het drieluik, en één :has()-regel die echte inhoudsvariatie opvangt. Bij de AI-route: prompt, onbewerkte output, verbeterlijst; let dit keer op absolute-misbruik en z-index-inflatie.

Verder lezen

- MDN, *Grid template areas*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout/Grid_template_areas

- MDN, *Controlling ratios of flex items along the main axis*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Controlling_ratios_of_flex_items_along_the_main_axis
- MDN, *position*: https://developer.mozilla.org/en-US/docs/Web/CSS/position
- MDN, *Stacking context*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_positioned_layout/Stacking_context
- MDN, *:has()*: https://developer.mozilla.org/en-US/docs/Web/CSS/:has

Hoofdstuk 6 - Responsive: het onbekende scherm

In een notendop

Je weet niet op welk scherm je pagina straks staat: een telefoon, een tablet, een laptop, een breedbeeld, een venster dat iemand half naast een ander venster klemt. **Responsive web design** omarmt dat gegeven: één HTML-document, één stylesheet, en een vormgeving die zich op elke breedte gedraagt. De kern is klein. Je schrijft eerst de smalle versie (mobile-first), je voegt met media queries per breekpunt toe wat er op een breder scherm verandert, en je weet wanneer iets vloeiend mag meebewegen en wanneer het trapsgewijs mag springen. In 6.5 komt dat samen met alles van de voorbije hoofdstukken in één skelet van een pagina: boxmodel, grid, flexbox, plaatsing en responsive, in de volgorde waarin je ze op de toets ook bouwt. Het vervolg van het hoofdstuk verfijnt dat skelet: wat je site al vanzelf doet, vloeiende waarden met `clamp()`, het zelforganiserende raster, queries die meer meten dan breedte, de juiste afbeelding voor het juiste scherm, en testen op een echt toestel. Dit hoofdstuk sluit blok 1 af: na deze week is je site van telefoon tot breedbeeld in orde, en ben je klaar voor de toets.

Wat je na dit hoofdstuk kan

Open de koffiebar van labo 5 en maak het venster traag smaller. De zijbalk van 16rem blijft 16rem, de drie kaartkolommen worden strookjes, en de fotolijst van 36rem schuift op een telefoon gewoon buiten beeld. Niets daarvan is fout volgens hoofdstuk 5; alles daarvan is fout op een telefoon. Deze week leer je wat er moet gebeuren, en vooral: hoeveel er vanzelf gebeurt als je de juiste gereedschappen kiest.

Na dit hoofdstuk maak je een pagina die van 320 CSS-pixels tot breedbeeld klopt: een plattegrond die op een telefoon stapelt en op een breed scherm kantelt, een navigatie die van kolom naar rij gaat, een kaartenrooster dat zelf uitrekent hoeveel kolommen er passen, titels die vloeiend meegroeien, en afbeeldingen die niet alleen passen maar ook niet te zwaar zijn. Je kiest je breekpunten op je content, schrijft ze in rem, en test op een echt toestel. Dat is precies wat de toets in november van je vraagt: een volwaardige responsieve layout, zelfstandig, zonder AI.

Het hoofdstuk heeft twee delen. Het eerste deel is de **kern**, en is wat labo 6 en de toets van je vragen: waarom responsive geen feature is (6.1), de viewport (6.2), mobile-first en de plattegrond die kantelt (6.3), media queries met min-width en het verschil tussen vloeiend en trapsgewijs (6.4), en ten slotte één skelet van een pagina waarin boxmodel, grid, flexbox, plaatsing en responsive samenkomen (6.5). Het tweede deel is het **vervolg**: de vloeibare basis die je al hebt (6.6), `clamp()` (6.7), het zelforganiserende raster (6.8), queries voorbij de breedte (6.9), responsive afbeeldingen (6.10) en

testen op het onbekende scherm (6.11). In de theorieles behandelen we de kern en het testen; het vervolg lees je zelf, en labo 6 en B6.1 gebruiken er stukken van, telkens met een verwijzing naar de sectie.

6.1 Responsive is geen feature

Eerst het kader, want de naam zet je op het verkeerde been. "Responsive design" klinkt als een extraatje dat je aan een site toevoegt; in werkelijkheid is het sinds jaren gewoon wat webdesign is. Meer dan de helft van het wereldwijde webverkeer komt van telefoons; webshops zien hun klanten vaker op een klein scherm dan op een groot; en de toegankelijkheidsplicht uit hoofdstuk 1 omvat mede dat content bruikbaar blijft bij ingezoomde weergave en smalle vensters. Er bestaat geen keuze tussen "een site" en "een responsive site". **Er bestaat een goede site en een kapotte.**

De aanpak rust op drie pijlers, en de volgorde ervan is de moderne les. Pijler één: een **vloeibare basis**, layouts uit relatieve verhoudingen (procent, fr, vw) die meebewegen met de ruimte. Pijler twee: **flexibele media**, afbeeldingen die hun container volgen in plaats van hem op te blazen. Pijler drie: **media queries**, stijlregels die pas gelden vanaf een bepaalde schermbreedte. Oudere cursussen behandelden de derde pijler als de ster van de show; vandaag draaien we dat om. Hoe meer je site uit zichzelf kan (pijler één en twee, plus de zelforganiserende gereedschappen van 5.2 en 6.8), hoe minder queries je nodig hebt, en elke query die je niet schrijft, is een breekpunt dat nooit kapot kan gaan. **Vuistregel:** queries zijn het vangnet, niet het fundament.

6.2 De viewport: hoe breed is een telefoon?



Figuur 6.1 - Zonder en met de meta-regel: zonder tekent een mobiele browser de pagina op een denkbeeldig canvas van zo'n 980 CSS-pixels en toont ze uitgezoomd; met de regel is de viewport zo breed als het toestel echt is, en meten je media queries wat er op het scherm staat.

Eerst het woord. De **viewport** is het venster waardoor je de pagina ziet: op je laptop het browservenster (maak het smaller en de viewport wordt smaller), op je telefoon het hele scherm. Alles wat deze week volgt, media queries voorop, meet die viewport. Daarom moet ze eerlijk zijn, en dat is ze op een telefoon niet vanzelf.

Tijd om een schuld uit hoofdstuk 1 in te lossen. Sinds week één staat in elk van je documenten deze regel, tot nu toe op goed vertrouwen:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

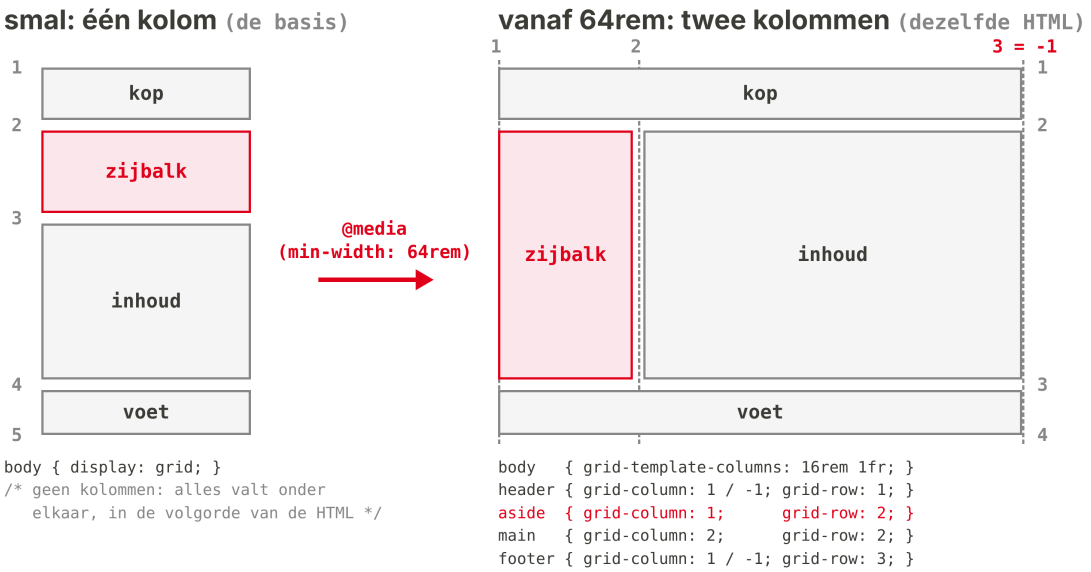
Waarom bestaat die? Toen de eerste iPhones verschenen, was het web gebouwd voor desktopmonitoren. Mobiele browsers losten dat op door te doen alsof: ze namen aan dat hun scherm zo'n 980 pixels breed was, tekenden de pagina op dat denkbeeldige canvas, en toonden het geheel uitgezoomd, als een foto van een krant. Leesbaar was anders, maar het "werkte". Dat is nog altijd het standaardgedrag, en jouw meta-regel is het afgesproken signaal om het uit te zetten: *maak de viewport zo breed als het toestel echt is* (`width=device-width`) *en start op honderd procent zoom* (`initial-scale=1`). Zonder deze regel krijgt je zorgvuldig gebouwde site alsnog de krantenfoto-behandeling en lijken al je media queries kapot (F6.1). Het is berucht lastig te herkennen als je niet weet waar je moet kijken. **Probeer dit:** haal de regel uit je `over-mij.html`, push, en open de pagina op je telefoon (D6.1). Zet ze daarna terug.

Hoe breed is die echte breedte dan? Niet wat de verpakking zegt. Een iPhone 17 Pro heeft een schermresolutie van 1206 bij 2622 pixels en meldt zich bij jouw CSS als 402 bij 874: de browser rekent in **CSS-pixels**, en op dat toestel gaan er drie schermpixels in één CSS-pixel, in elke richting, zodat tekst op elk scherm even groot blijft. De site yesviz.com toont die cijfers per toestel (resolutie, viewport, ratio) en is een goede plek om het verschil één keer met eigen ogen te zien. De les: je media queries meten de 402, nooit de 1206; **je ontwerpt tegen CSS-pixels, niet tegen schermresoluties.** Hoe die verhouding precies werkt, staat in het motorkap-kader.

Nu de viewport een eerlijke maat heeft, verdienen de viewport-eenheden uit 3.2 hun kleine lettertjes. Op desktop is 1vh keurig een honderdste van het venster. Op mobiel is "het venster" echter een beweeglijk ding: adresbalk en knoppenbalk schuiven in en uit beeld tijdens het scrollen. Wat is 100vh dan? De browsers kozen voor de grootste variant, het venster mét ingeschoven balken, met als berucht gevolg dat een element van 100vh bij het openen van de pagina nét niet past: de onderrand valt achter de balk. CSS heeft daar inmiddels eerlijke eenheden voor: **svh** (small: het venster mét balken in beeld), **lvh** (large: zonder) en **dvh** (dynamic: volgt de werkelijkheid van moment tot moment). De praktische afspraak voor dit vak: schrijf voor "minstens schermhoog" `min-height: 100svh`, de veilige ondergrens, en onthoud dat een achteloos `height: 100vh` op mobiel vrijwel altijd nét te hoog is (F6.2). Deze eenheden werken in alle browsers sinds 2022 (Safari 15.4, Firefox 101, Chrome en Edge 108). Je grondplan uit 4.7 en 5.1 verdient bij deze gelegenheid die kleine upgrade, en in labo 6 gebruik je `min-height: 100svh` in de kleurvlakken en de tribune; meer dan dat ene woord heb je er dit semester niet aan.

6.3 Mobile-first, en de plattegrond die kantelt

De strategie die dit hele hoofdstuk ordent, heet **mobile-first**, en ze bestaat uit twee afspraken. Eén: de *basisstijlen*, de regels buiten elke query, beschrijven de smalle versie van je pagina, die voor een telefoon. Twee: *media queries* voegen daar per breekpunt alleen toe wat er op een breder scherm verandert. De rationale is dubbel. Praktisch: de smalle versie is vrijwel altijd de eenvoudigste (één kolom, alles onder elkaar, de gewone flow doet het werk), dus die als vertrekpunt nemen minimaliseert je code; desktop-first werken betekent op elk klein scherm complexiteit zitten afbreken, en dat is altijd meer werk. Ontwerpmatig: wie eerst voor het kleinste scherm ontwerpt, moet kiezen wat essentieel is, en die scherpte komt de brede versie ten goede.



Figuur 6.2 - Mobile-first op het raster: in de basis één kolom, waarin de vier elementen onder elkaar vallen in de volgorde van de HTML; vanaf het breekpunt twee kolommen, en elk element krijgt zijn kolom en zijn rij. De HTML verandert niet.

Het grondplan van 5.1, dat van koffiebar Kaai 12 in labo 5, laat zien wat die twee afspraken in code betekenen:

```
body {
  display: grid;
  min-height: 100svh;
  grid-template-rows: auto auto 1fr auto; /* one column: header, aside, main, footer fall
in DOM order */
}

@media (min-width: 64rem) {
  body {
    grid-template-columns: 16rem 1fr;
    grid-template-rows: auto 1fr auto;
  }
  header { grid-column: 1 / -1; grid-row: 1; } /* across both columns */
  aside  { grid-column: 1;      grid-row: 2; }
  main   { grid-column: 2;      grid-row: 2; }
  footer { grid-column: 1 / -1; grid-row: 3; }
}
```

Lees het van boven naar onder. Eerst de basis: body is een grid zonder kolommen, dus met één kolom, en de vier elementen vallen vanzelf onder elkaar in de volgorde van de HTML: header, aside, main, footer. Alleen de rijen krijgen een maat, zodat main de resthoogte krijgt. Dat is de telefoonversie, en ze heeft geen query en geen plaatsing nodig. Dan het blok dat met `@media` begint. Dat is een **media query**: *vanaf een venster van 64rem breed geldt wat hier tussen de accolades staat.* Hoe je zo'n blok precies schrijft, en waar die 64rem vandaan komt, is de stof van 6.4; lees hem voorlopig gewoon als een voorwaarde met stijlregels erin. Vanaf 64rem krijgt het raster twee kolommen, en zeg je per

element op welke kolom en rij het komt, op lijnen zoals in 4.6: kop en voet van lijn 1 tot lijn -1, dus over beide kolommen; de zijbalk in kolom 1 en de inhoud in kolom 2, allebei op rij 2. De HTML is niet aangeraakt; header, aside, main en footer staan er nog exact zoals in labo 5. Alleen de beschrijving van het raster veranderde: de scheiding van structuur en vormgeving uit 2.1, nu over schermbreedtes heen. Dezelfde plattegrond kan je ook met named areas uittekenen (5.1, en zo deed labo 5 het); lijnen zijn hier bewust de keuze, omdat je dan letterlijk leest welk element waar komt.

En let op wat er *in* de query staat: alleen wat verandert. Kleuren, lettertypes, padding, de flex-balk in de header, dat alles staat één keer in de basis en erft gewoon door. Een query is geen tweede versie van je stylesheet maar een lijstje verschillen (F6.8). **Vuistregel:** de basis is smal, elke query voegt toe, en in een query staat alleen wat verandert. Dat is het hele mobile-first-contract. Dit patroon, plus de nav-omslag van 6.4 en het zelforganiserende raster van 6.8: meer heeft een volwaardige responsieve site meestal niet nodig.

6.4 Media queries: ingrijpen waar het breekt

In 6.3 las je een media query op goed vertrouwen. Hier is de grammatica, en de vraag die elke query oproept: waar leg je het breekpunt? Sommige veranderingen aan een layout zijn structureel: op een telefoon hoort de zijbalk niet naast maar onder de inhoud, en wil de navigatie een kolom zijn in plaats van een rij. Voor zulke omslagpunten bestaat de **media query**: een blok CSS dat alleen geldt wanneer het scherm aan een voorwaarde voldoet.

```
nav ul {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

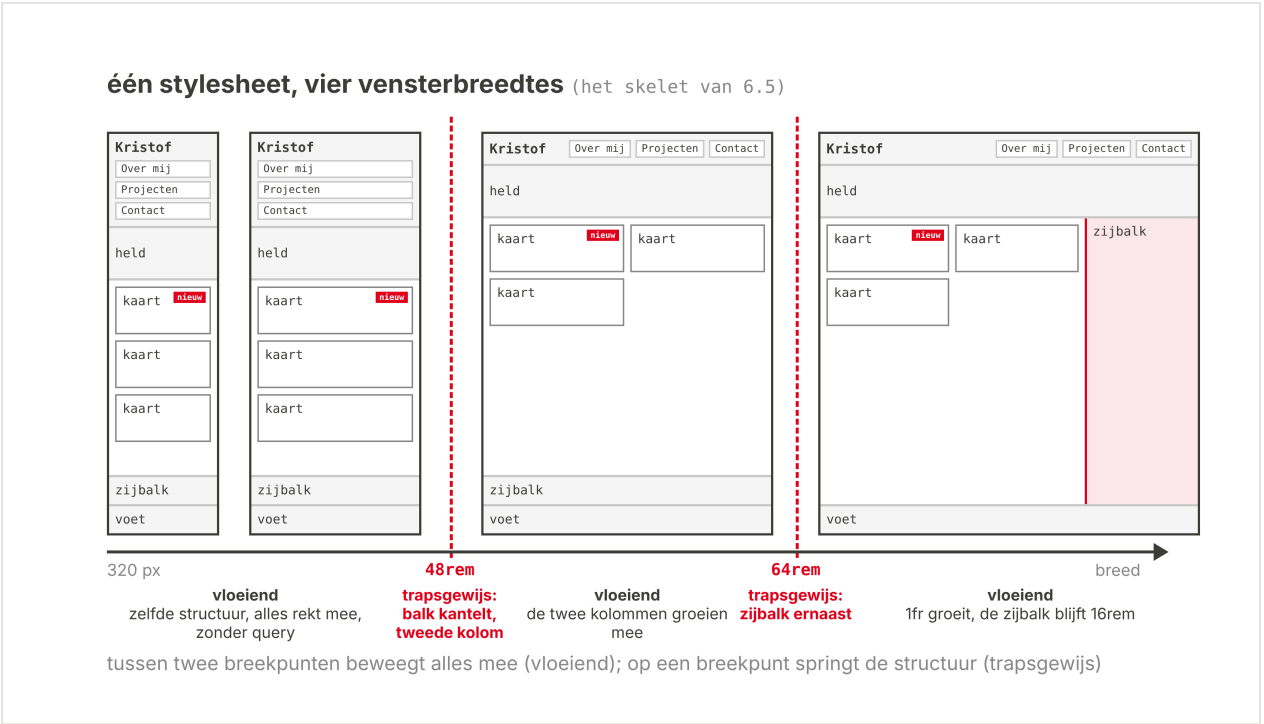
@media (min-width: 48rem) {
  nav ul {
    flex-direction: row;
    gap: 1.5rem;
  }
}
```

De syntax: `@media`, een voorwaarde tussen haakjes, en daarbinnen gewone stijlregels met een extra laag accolades (insprongen bewaken: de F2.3-familie ligt hier op de loer). Wij schrijven voorwaarden vrijwel altijd als **min-width**: "vanaf deze breedte en breder". En begrijp precies wat een query wel en niet doet, want hier komt hoofdstuk 2 terug: **een media query voegt geen specificiteit toe.** De regels erbinnen doen gewoon mee aan de cascade, met hun eigen selectorgewicht; het enige wat de query doet, is ze aan- of uitschakelen. Dat de row-versie hierboven wint op brede schermen, komt puur doordat ze *later* in het bestand staat: stap drie van de cascade-ladder, volgorde, bewust bespeeld zoals in 2.7 al werd aangekondigd. Daarom is de vaste plek van queries onder de basisregels geen stijlkwestie maar een werkingsvoorwaarde: zet je ze erboven, dan verliezen ze van elke gelijkwaardige regel die volgt, en "doet de query het niet" (F6.4).

Breekpunten, de gekozen omslagwaarden, kies je op je **content**, niet op toestellen. Maak je venster traag smaller en breder, kijk waar het ontwerp *breekt* (regels te lang, elementen te krap, een rij die wringt), en leg daar je breekpunt. Lijstjes "standaard-breekpunten voor iPhone en iPad" die je online en in gegenereerde code ziet, zijn een anti-patroon: toestellen wisselen elk jaar, jouw content niet (F6.6). Twee tot drie breekpunten volstaan voor de meeste sites, en schrijf ze in rem (48rem in plaats van 768px), zodat ze meeschalen met de tekstinstelling van de gebruiker en dezelfde taal spreken als de rest van je stylesheet. Waarom dat zo goed uitpakt, legt het motorkap-kader uit.

Vloeiend of trapsgewijs?

Een layout kan op twee manieren op de breedte van het scherm reageren, en het verschil is de sleutel tot alles wat volgt. **Vloeiend** (fluid) wil zeggen: de waarde beweegt mee met elke pixel die het venster breder of smaller wordt. Dat doen procenten, fr, een leeskolom met max-width en auto-marges, gap, en de afbeeldingen dankzij de reset (zie hieronder): geen query, geen sprong, de browser rekent continu. **Trapsgewijs** wil zeggen: de layout blijft gelijk tot een breekpunt, en springt daar naar een andere structuur. Dat doet een media query, en alleen een media query.



Figuur 6.3 - Vloeiend of trapsgewijs: tussen twee breekpunten beweegt alles mee met het venster, zonder query; op een breekpunt springt de structuur. Een goede responsive site doet beide, elk op zijn plaats.

	VLOEIEND	TRAPSGEWIJS
Wat gebeurt er	de waarde beweegt continu mee met het venster	de layout springt op een breekpunt naar een andere structuur
Gereedschap	procent, fr, max-width met auto-marges, gap, de reset op img; straks ook `min()` en `clamp()` (6.7) en `auto-fit` (6.8)	`@media (min-width: ...)`
Waarvoor	maten en witruimte: kolommen die mogen krimpen, tekst die mag lopen	structuur: een zijbalk die ernaast komt, een navigatie die kantelt, een extra kolom kaarten
Kan het kapot	niet op een breekpunt, want er is er geen	ja: test op én tussen de breekpunten

De vuistregel die daaruit volgt: **vloeiend binnen een trede, trapsgewijs tussen de treden.** Alles wat mee kan bewegen, laat je meebewegen; een query bewaar je voor wat echt van structuur verandert. Wie dat omdraait en elke maat per breekpunt vastzet (`width: 20rem` vanaf hier, `width: 30rem` vanaf daar), krijgt een site die klopt op de drie breedtes waarop hij testte en wringt op alle andere. Vandaar de sleeptest van labo 6 en 6.11: tussen twee breekpunten hoort alles te bewegen, op een breekpunt mag het springen, en nergens mag een schuifbalk verschijnen. Hoeveel er in je site al vloeiend is, bekijk je in 6.6; de gereedschappen die hele treden overbodig maken, komen in 6.7 en 6.8.

Flexibele media: wat de reset al doet

De tweede pijler van 6.1, flexibele media, heb je sinds hoofdstuk 3 aan boord zonder dat het je opviel. In `reset.css` staat:

```
/* reset.css, rule 3: media never spills out of its container */
img, picture, video, canvas, svg {
  display: block;
  max-width: 100%;
}
```

Lees het als twee beslissingen. `max-width: 100%` zegt dat een afbeelding nooit breder wordt dan haar container: is de container smaller dan het bestand, dan krimpt de afbeelding mee en volgt de hoogte vanzelf in verhouding. Dat is vloeiend gedrag, zonder query. `display: block` haalt de afbeelding uit de tekstregel (3.3), zodat er geen reep witruimte onder hangt en marges doen wat je verwacht. Zonder die twee regels is een foto van 1600 pixels de klassieke saboteur op een telefoon: ze duwt de pagina open en levert de horizontale schuifbalk (F6.3). Zet daarnaast, zoals in 3.9, `width` en `height` als attributen op het img-element: de browser reserveert dan de juiste verhouding vóór het bestand binnen is, en de pagina springt niet tijdens het laden. Wat de reset niet doet, is het juiste *bestand* kiezen: de telefoon die de foto op 360 CSS-pixels toont, downloadt nog altijd de volle 1600. Dat probleem, en `srcset` als antwoord, is 6.10.

De grammatica voorbij het basisgeval

Queries kunnen meer dan één ondergrens stellen. Voorwaarden combineer je met `and`; een komma tussen twee complete voorwaarden betekent *of*; en `not` ontkent. Naast width bestaan er meer meetbare kenmerken (features): `orientation` (portrait of landscape), aspect-ratio, en de voorkeuren en invoerkenmerken die in 6.9 volgen. En sinds enkele jaren kent CSS ook een **range-syntax** die voorwaarden leesbaarder maakt dan de min- en max-woorden:

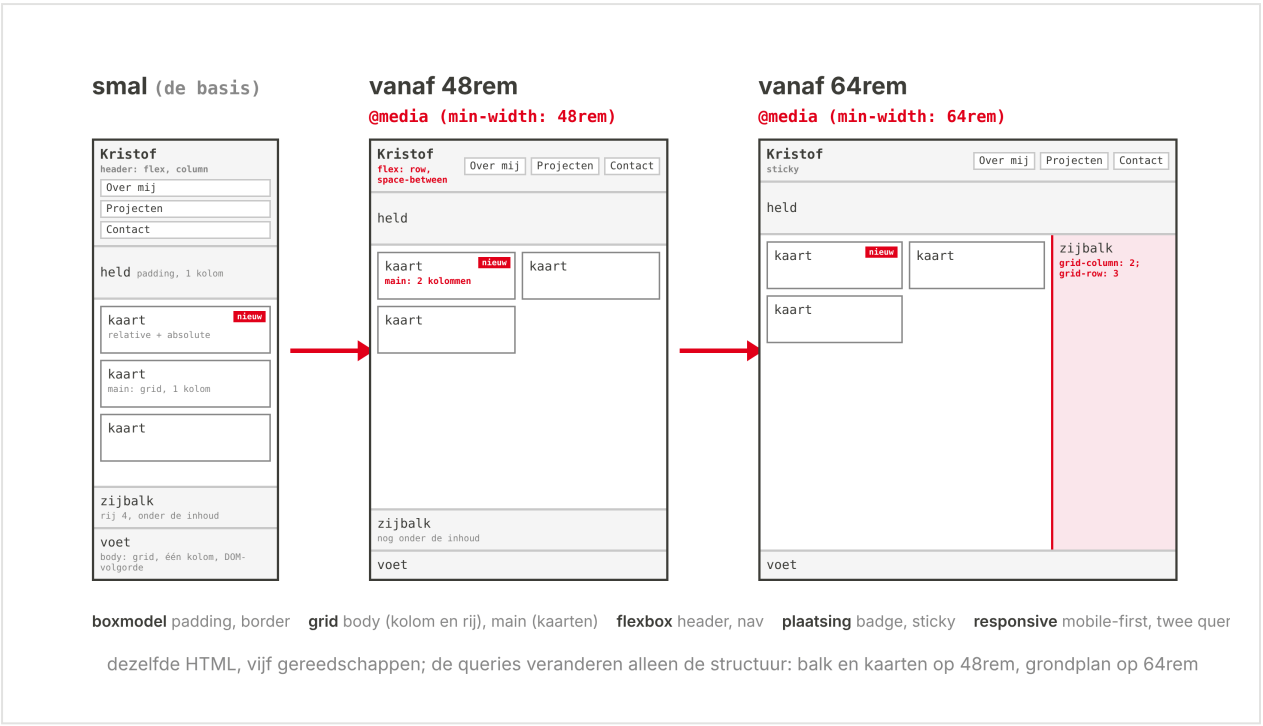
```
@media (width >= 48rem) {
  /* modern spelling of min-width: 48rem */
}

@media (48rem <= width < 64rem) {
  /* only between the two breakpoints, no min/max juggling */
}
```

Vooral die tweede vorm, een gesloten interval in één regel, is winst: het klassieke equivalent (`min-width` én `max-width` in één query, met de vraag of de grenswaarde nu wel of niet meetelt) was foutgevoelig genoeg om een eigen folklore te hebben. De range-syntax is widely available; wij gebruiken beide spellingen door elkaar, en jij moet vooral beide kunnen lezen. Oudere codebases en veel gegenereerde code spreken nog uitsluitend min-width.

6.5 Alles samen: het skelet van een pagina

Tot hier kreeg je de gereedschappen één voor één: het boxmodel (hoofdstuk 3), grid en flexbox (4), plaatsing (5), en nu mobile-first met media queries. Op je eigen site, en op de toets, komen ze samen in één pagina. Hier is die pagina, bewust zonder kleur en zonder inhoud: een **wireframe**, het skelet van je persoonlijke site, en in grote lijnen van elke toetsopgave. Lees deze sectie twee keer: één keer om te zien waar elk gereedschap zit, en één keer als bouwvolgorde.



Figuur 6.4 - Het skelet van een pagina op drie breedtes. Smal: alles gestapeld in de volgorde van de HTML, de navigatie een kolom, de kaarten onder elkaar. Vanaf 48rem: de balk is een rij en de kaarten staan in twee kolommen. Vanaf 64rem: twee kolommen in het grondplan, en de zijbalk krijgt kolom 2, rij 3, naast de inhoud. Dezelfde HTML; elk onderdeel draagt het gereedschap waarmee het gebouwd is.

De HTML is het grondplan van 1.5 en 5.1, met main als kaartenrooster en één badge op de eerste kaart:

```
<!-- index.html, the skeleton: real content comes once the layout works -->
<body>
  <header>
    <a class="naam" href="index.html">Kristof</a>
    <nav><ul>
      <li><a href="over-mij.html">Over mij</a></li>
      <li><a href="projecten.html">Projecten</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul></nav>
  </header>
  <section class="held"><h1>...</h1><p>...</p></section>
  <main>
    <article class="kaart"><span class="badge">nieuw</span><h2>...</h2><p>...</p></article>
    <article class="kaart"><h2>...</h2><p>...</p></article>
    <article class="kaart"><h2>...</h2><p>...</p></article>
  </main>
  <aside>...</aside>
  <footer>...</footer>
</body>
```

De CSS bestaat uit drie delen, en de volgorde is geen toeval. Deel één is de smalle versie van het grondplan en de balk; de tokens op `:root` (2.9) staan er zoals altijd boven en zijn hier weggelaten:

```css
/* stijl.css, part 1: the base is the narrow version, no query needed */
body {
  display: grid;                      /* one column: the five parts fall in DOM order */
  min-height: 100svh;                 /* 6.2 */
  grid-template-rows: auto auto 1fr auto auto;
}

header, nav ul {
  display: flex;                      /* a column on a phone */
  flex-direction: column;
  gap: 0.5rem;
}
nav ul { list-style: none; padding: 0; }    /* the reset leaves lists alone (3.3) */
header { position: sticky; top: 0; background: var(--color-papier); }
header, .held, main, aside, footer { padding: 1rem; }    /* box model: breathing room */
```

Deel twee is het kaartenrooster, in de smalle versie één kolom, en de badge op de hoek van een kaart:

```css
/* stijl.css, part 2: the cards, one column on a phone, and the badge */
main {
  display: grid;                      /* no columns declared: one column */
  gap: 1rem;
}
.kaart {
  position: relative;                 /* the anchor for the badge (5.4) */
  padding: 1rem;
  border: 1px solid var(--color-rand);
}
.badge {
  position: absolute;                 /* on the card's corner */
  top: 0.5rem;
  right: 0.5rem;
}
```

Deel drie, onder de basis, zijn de twee queries, elk met alleen de delta:

```css
/* stijl.css, part 3: media queries, below the base, each one only the delta */
@media (min-width: 48rem) {
  header { flex-direction: row; justify-content: space-between; align-items: center; }
  nav ul { flex-direction: row; gap: 1.5rem; }
  main   { grid-template-columns: 1fr 1fr; }
}

@media (min-width: 64rem) {
  body {
    grid-template-columns: 1fr 16rem;
    grid-template-rows: auto auto 1fr auto;
  }
  header { grid-column: 1 / -1; grid-row: 1; }  /* across both columns */
  .held  { grid-column: 1 / -1; grid-row: 2; }
  main   { grid-column: 1;      grid-row: 3; }
  aside  { grid-column: 2;      grid-row: 3; }
  footer { grid-column: 1 / -1; grid-row: 4; }
}
```

Lees de drie delen nu als een bouwvolgorde, want zo pak je het op de toets ook aan. Deel één is het grondplan in de smalle versie: body als grid zonder kolommen, dus één kolom (4.5), waarin de vijf delen vanzelf onder elkaar vallen in de volgorde van de HTML, en `min-height: 100svh` zodat de voet onderaan blijft (6.2). De kop is een flex-kolom met de naam boven de links, en de lijst in de navigatie is zelf ook een flex-kolom; de lijstopmaak haal je weg, want de reset doet dat bewust niet (3.3). Padding op de vijf gebieden geeft ademruimte, en dankzij border-box uit de reset telt die niet bij de breedte (3.1). De balk blijft plakken met sticky (5.4); een achtergrond is dan verplicht, anders schuift de inhoud er zichtbaar doorheen. Deel twee: main is een grid zonder kolommen, dus één kolom (4.5), en elke kaart is een box met padding en rand; de badge zit met het ankerduo op de hoek (5.4). Deel drie, onder de basis: vanaf 48rem kantelen de kop en de navigatie naar een rij met de naam links en de links rechts (4.3), en krijgen de kaarten een tweede kolom; vanaf 64rem krijgt het grondplan twee kolommen en zeg je per element op welke kolom en rij het komt (4.6, 6.3): kop, held en voet over beide kolommen, de inhoud in kolom 1 en de zijbalk in kolom 2, allebei op rij 3. In elke query staat alleen de delta; al de rest erft van de basis.

ONDERDEEL	GEREEDSCHAP	WAAR HET STAAT
grondplan van de pagina	grid: één kolom in de basis, twee vanaf 64rem, elk element op zijn kolom en rij	4.5, 4.6, 6.3
balk: naam links, links rechts	flexbox met space-between	4.3
navigatie van kolom naar rij	flex-direction in een query	6.4
kaarten: één kolom, dan twee	grid, de kolommen in een query	4.5, 6.4
badge op de hoek van een kaart	relative op de kaart, absolute op de badge	5.4
balk die blijft plakken	sticky	5.4
ademruimte, maten, randen	padding, gap, border-box uit de reset	3.1, 3.3
minstens schermhoog	min-height: 100svh	6.2
eerst smal, dan de delta's	mobile-first	6.3

Zo bouw je het, in deze volgorde:

- reset gelinkt, tokens op `:root`, viewport-meta in de head
- het grondplan in de smalle versie: één kolom, alles onder elkaar in de volgorde van de HTML
- de balk en de navigatie als kolom
- de kaarten in één kolom, de badge met het ankerduo
- slepen van 320 px naar breed: nergens een schuifbalk? dan pas de queries, van smal naar breed, alleen de delta
- slepen opnieuw, op de breekpunten én ertussen

Eén ding ontbreekt in dit skelet bewust: het klassieke **hamburgermenu**, navigatie die op mobiel achter een knop schuilgaat. Een goed hamburgermenu vergt interactie (openen, sluiten, focus vasthouden), en dat is JavaScript-werk voor blok 3, of platform-UI-werk met de moderne elementen van hoofdstuk 8. Tot dan is een compacte navigatie in de flow, de kolom-naar-rij-omslag hierboven, de eerlijke en volwaardige oplossing; voor een handvol links is ze zelfs gebruiksvriendelijker dan een verstopmenu.

Hiermee heb je alles wat labo 6 en de toets van je vragen: een plattegrond die stapelt en kantelt, een balk die van kolom naar rij gaat, een kaartenrooster, een badge, en nergens een schuifbalk. De rest van dit hoofdstuk verfijnt dat skelet. Je leest er waarom zoveel al vanzelf werkte (6.6), hoe titels en witruimte vloeiend meegroeien zonder query (6.7), hoe het kaartenrooster zelf uitrekent hoeveel kolommen er passen (6.8), welke queries meer meten dan breedte (6.9), hoe je het juiste beeldbestand levert (6.10) en hoe je alles test, in DevTools en op een echt toestel (6.11). Labo 6 en B6.1 lenen er stukken van; de tips in het labo verwijzen naar de sectie.

6.6 De vloeibare basis: wat je al hebt

Zet je site open en maak het venster traag smaller. Verrassend veel gedraagt zich al: de leeskolom (max-width plus auto-marges, 3.6) vult smalle schermen en begrenst brede; de afbeeldingen volgen hun container (de resetregel uit 3.3 en 6.4); fr-kolommen herverdelen zich; de citaten van labo 5 breken netjes naar nieuwe regels; gap blijft gap. Dat is geen toeval maar de oogst van vijf hoofdstukken bouwen met grenzen in plaats van maten, de mindset van 3.8, die hier haar naam krijgt: **intrinsiek ontwerp**, vormgeving die uit haar eigen regels weet hoe ze zich moet gedragen, zonder dat jij per schermbreedte instructies schrijft. Het is de vloeiende helft van 6.4, en ze is groter dan je denkt.

Wat die vloeibaarheid saboteert, is altijd hetzelfde: een vaste maat die ergens hard tegen de stroom in staat. De fotolijst van labo 5 is er een schoolvoorbeeld van: `width: 36rem` klopte op 1280 pixels en levert op 400 pixels venster de horizontale schuifbalk die elke responsive site verraadt (F6.3). De diagnoseroutine bij zo'n schuifbalk hoort bij je toetsvoorbereiding: maak het venster smal, open DevTools, en klim door de boom op zoek naar het element dat breder is dan zijn ouder; de inspector-highlight toont hem in seconden. De remedie is vrijwel altijd de vaste maat vervangen door een grens of een verhouding: `width: min(36rem, 100%)` zegt "36rem, maar nooit breder dan mijn ouder", en de lijst krimpt mee tot ze past.

6.7 clamp(): vloeiend, maar met grenzen

Voor waarden die met het scherm mogen meegroeien, leverde 3.2 al de vw-eenheid, met een belofte erbij dat lettergroottes erop moesten wachten. Hier is waarom, en hier is de oplossing.

Niet zo:

```
h1 {
  font-size: 3vw; /* tiny on a phone, huge on a wide screen, ignores zoom */
}
```

Zo wel:

```
h1 {
  font-size: clamp(1.75rem, 4vw + 1rem, 3rem);
}

main {
  padding-inline: clamp(1rem, 5vw, 4rem); /* inline = left and right */
}
```

Pure vw-typografie heeft twee kwalen: op een smal scherm wordt ze onleesbaar klein en op breedbeeld absurd groot, én, erger want onzichtbaarder, ze schaalt niet mee wanneer een gebruiker inzoomt of zijn lettergrootte verhoogt, wat haar tot een toegankelijkheidsprobleem maakt (F6.7). De functie **clamp()** repareert beide in één beweging. Ze neemt drie waarden, een ondergrens, een gewenste (vloeiende) waarde en een bovengrens, en levert de middelste zolang die tussen de grenzen

blijft. Lees de h1-regel: minstens 1.75rem, liefst "4% van de viewport plus 1rem", en nooit meer dan 3rem. De titel groeit soepel mee van telefoon naar breedbeeld, maar binnen grenzen die jij als ontwerper stelt, en de rem-term in het midden houdt de zoom- en instellingsgevoeligheid aan boord die pure vw mist. Dit heet **fluid typography**, en hetzelfde patroon werkt voor witruimte (tweede voorbeeld): royale paginamarges op breedbeeld, compacte op een telefoon, zonder één query. Het is vloeiend in de zin van 6.4, met grenzen erbij. In hoofdstuk 7 bouwen we hier schalen mee; voor nu volstaan de twee voorbeeldpatronen, plus de wetenschap dat `clamp()` twee eenvoudiger broertjes heeft, `min()` en `max()`, die simpelweg de kleinste of grootste van hun argumenten kiezen. Je zag `min()` in 6.6; je zal ze vooral in andermans code lezen.

6.8 Het zelforganiserende raster

In het skelet van 6.5 kregen de kaarten hun tweede kolom via een query. Dat werkt, en voor twee toestanden is het prima. Maar zodra je drie of vier kolommen wil, groeit het uit tot een trap, en daarvoor bestaat iets beters. Twee hoofdstukken lang is het beloofd; hier is de grid-tegenhanger van wrap-plus-basis, en het is misschien wel de nuttigste regel CSS van heel blok 1. Zet hem naast wat je zonder hem zou schrijven:

Niet zo:

```
.galerij { display: grid; grid-template-columns: 1fr; gap: 1.5rem; }

@media (min-width: 40rem) { .galerij { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 60rem) { .galerij { grid-template-columns: repeat(3, 1fr); } }
@media (min-width: 80rem) { .galerij { grid-template-columns: repeat(4, 1fr); } }
```

Zo wel:

```
.galerij {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr));
  gap: 1.5rem;
}
```

Ontleed de tweede van binnen naar buiten. **minmax(16rem, 1fr)** beschrijft een kolom met een ondergrens en rek: minstens 16rem, en anders een gelijk aandeel van de ruimte, dezelfde bandbreedte-logica die je in het motorkap-kader van hoofdstuk 4 al tegenkwam. En **auto-fit** in de `repeat()` zegt: maak zoveel van die kolommen als er passen. De browser rekent per schermbreedte uit hoeveel 16rem-kolommen erin gaan, legt ze neer, en laat ze de rest opvullen. Vier kolommen op breedbeeld, twee op een tablet, één op een telefoon, zonder dat het woord "breedte" ook maar ergens in jouw code voorkomt. De kolommentrap van het Niet zo doet hetzelfde in vier blokken, met drie breekpunten die kapot kunnen (F6.9). Het broertje `auto-fill` houdt bij weinig items lege sporen open in plaats van de items te laten uitzetten; het verschil zie je alleen bij minder items dan er kolommen passen, en `auto-fit` is vrijwel altijd wat je bedoelt.

Vergelijk nu bewust met de citaten van labo 5, want dit is de as-vraag van 4.2 in haar mooiste gedaante. Beide bouwen een zelforganiserend rooster; het verschil is de uitlijning. De flex-versie is een reeks onafhankelijke regels: een halfvolle laatste regel verdeelt zijn ruimte en zijn items worden breder dan de rest. De grid-versie is een echt raster: elke kaart valt in het gelid, ook op de laatste rij, en kolommen lijnen uit over rijen heen. **Vuistregel:** moet de laatste regel meedelen in de ruimte, dan flex; moet ze in het gelid blijven, dan grid. Daarmee heb je voor elk kaartenrooster het juiste antwoord.

6.9 Queries voorbij de breedte

De naam "media query" verklapt dat er meer te bevragen valt dan schermbreedte, en drie families daarvan horen tot de basisuitrusting van een moderne developer.

De eerste familie meet **gebruikersvoorkeuren**. Besturingssystemen laten gebruikers voorkeuren instellen, donkere modus, verminderde beweging, en CSS kan daarnaar vragen: `@media (prefers-color-scheme: dark)` matcht bij een donkere systeeminstelling, `@media (prefers-reduced-motion: reduce)` bij een gebruiker die animaties wil temperen (een toegankelijkheidsinstelling voor onder meer mensen met evenwichts- en concentratieproblemen). Dit is conceptueel hetzelfde instrument als je breekpunten, conditionele CSS, maar de conditie is nu een mens in plaats van een venster. Beide krijgen hun volwaardige behandeling in blok 2 (dark mode in hoofdstuk 7, bewegingsvoorkeuren als vaste reflex in hoofdstuk 8); onthoud hier het principe: **een query kan luisteren naar het scherm én naar de persoon erachter.**

De tweede familie meet het **invoerapparaat**, en lost een probleem op dat je op je eigen telefoon kan zien. Hover-effecten (2.5) veronderstellen een muis; op een touchscreen bestaat "eroverheen zweven" niet, en een tik laat het hover-effect blijven plakken tot de volgende tik: je knop blijft in zijn zweefkleur hangen. De query `(hover: hover)` matcht alleen op apparaten die echt kunnen zweven:

```
@media (hover: hover) {
  .kaart:hover {
    transform: translateY(-0.25rem); /* lift only where hovering exists */
  }
}
```

Zo krijgen muisgebruikers hun effect en blijven touchgebruikers verschoond van het blijf-plakken-gedrag (F6.11). Het broertje `(pointer: coarse)` detecteert grove aanwijzers (vingers) en wordt gebruikt om klikvlakken te vergroten; voor dit vak volstaat het hover-patroon als vaste gewoonte bij decoratieve hover-effecten. Merk op dat `:focus` hier buiten blijft: focus bestaat op elk apparaat, dus focus-stijlen blijven onvoorwaardelijk. Nog een reden waarom 2.5 hover en focus als paar aanleerde maar ze niet aan elkaar vastklonk.

De derde is geen feature maar een **medium**: print. Elke pagina die mensen willen bewaren of afdrukken, een recept, een ticket, jouw cv-pagina straks, verdient vijf minuten printzorg, en die investering is bescheiden omdat het opnieuw gewoon cascade is:

```css
@media print {
  nav, footer, .geen-print {
    display: none;      /* navigation is meaningless on paper */
  }

  body {
    color: black;
    background: white;  /* save toner, maximise contrast */
  }
}
```

Navigatie en decoratie verdwijnen, kleuren worden inktvriendelijk, en de gewone flow doet de rest: papier is per definitie één kolom, dus je mobile-first-basis is al bijna een printversie. Test via het afdrukvoorbeeld van de browser, dat gewoon deze query evalueert. Het bestaan van media-*types* verklaart trouwens de spelling die je in oude code ziet: `@media screen and (min-width: ...)`. Het woord screen mag weg omdat het de standaard is, maar nu weet je wat er stond.

Een vierde, jonge familie meet niet het venster maar de **container**. Media queries meten het *scherm*, maar een component weet daarmee nog niets over zijn eigen situatie: dezelfde kaart staat op breedbeeld soms in een smalle zijbalk. Daarvoor bestaan sinds kort **container queries**, `@container (min-width: 24rem)`, waarmee een component op de breedte van zijn eigen container reageert in plaats van op het venster. Het is jonge maar breed ondersteunde CSS en het logische volgende hoofdstuk van het intrinsieke verhaal; wij houden het dit semester bij deze kennismaking (verdieping: de MDN-link onderaan), maar herken `@container` wanneer je hem tegenkomt. Je zal hem steeds vaker zien.

Eén verwante query moet je tot slot herkennen, al bevraagt ze geen medium maar de browser zelf: **@supports** vraagt of de browser een bepaalde declaratie kent (`@supports (aspect-ratio: 1)`), en past de regels erbinnen alleen toe als het antwoord ja is. Het is het nette gereedschap achter *progressive enhancement*: een werkende basis voor iedereen, een verrijking voor browsers die het aankunnen. In dit vak heb je het zelden nodig: alles wat wij als leerstof aanbieden is widely available (2.1), en jonge features als `:has()` gebruiken we zo dat er zonder hen gewoon een werkende, iets soberder pagina overblijft. Maar in professionele codebases en gegenereerde CSS kom je `@supports`-blokken geregeld tegen, en nu lees je wat er staat: dit deel is de verrijking, het deel erbuiten is de basis.

6.10 Responsive afbeeldingen: het juiste bestand voor het juiste scherm

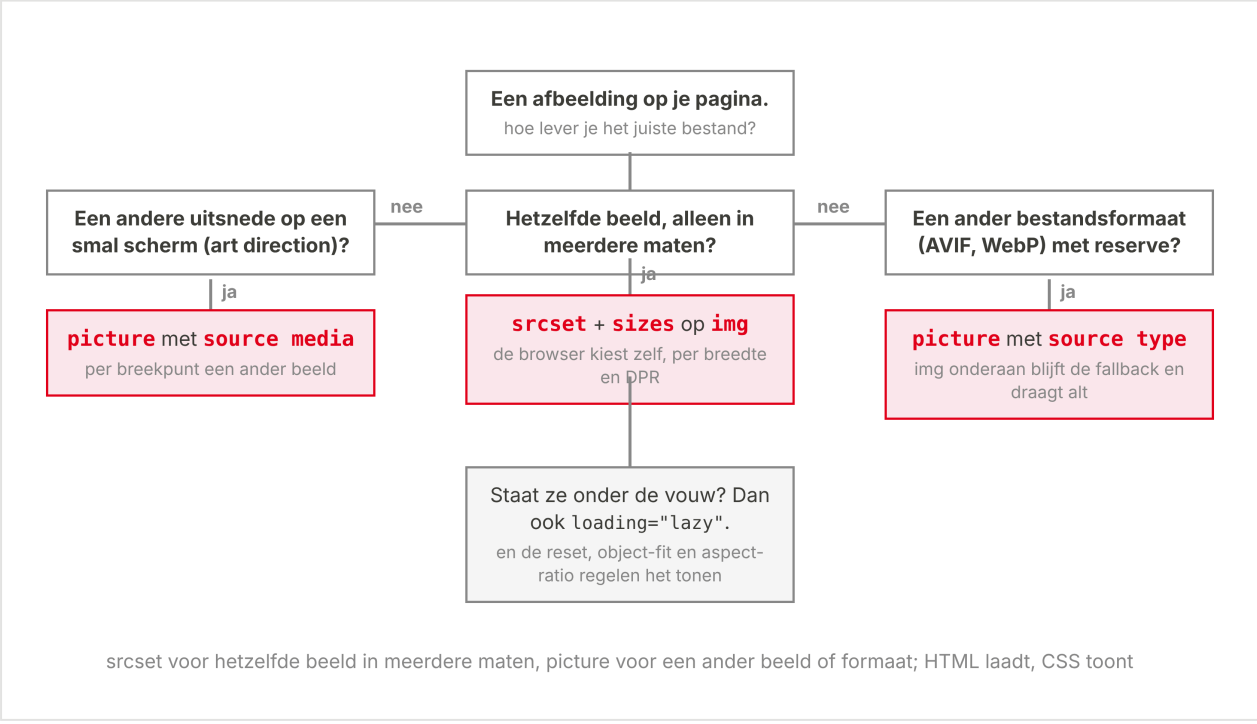
In 6.4 zag je dat de reset een afbeelding laat meekrimpen: `max-width: 100%` (3.3), en visueel klopt het. Maar er is een verborgen kost: de telefoon die jouw foto op 360 CSS-pixels breed toont, downloadt nog altijd het volledige bestand van 1600 pixels breed. Op mobiele data is dat pure verspilling, en afbeeldingen zijn op de meeste sites veruit de zwaarste bestanden. Het omgekeerde

probleem bestaat ook: één bestand van 800 pixels oogt scherp op een gewoon scherm maar wazig op een scherm met DPR 2 (motorkap), dat eigenlijk 1600 beeldpixels kan tonen. De kern van responsive afbeeldingen is dus niet *tonen* maar *leveren*: **het juiste bestand voor deze situatie.**

HTML lost dat op met twee attributen op het img-element dat je al kent. **srcset** biedt de browser een menu van bestandsversies aan, elk met zijn werkelijke breedte (de `w`-descriptor), en **sizes** vertelt hoe breed de afbeelding in jouw layout ongeveer wordt:

```
<img
  src="pasta-800.jpg"
  srcset="pasta-400.jpg 400w, pasta-800.jpg 800w, pasta-1600.jpg 1600w"
  sizes="(min-width: 64rem) 32rem, 100vw"
  alt="Bord spaghetti carbonara met verse pecorino">
```

Lees het als een gesprek. Srcset: "ik heb deze foto op 400, 800 en 1600 pixels breed." Sizes: "in deze layout word ik op brede schermen zo'n 32rem breed, en daaronder de volle schermbreedte." De browser, die het scherm, de DPR én de netwerksituatie kent, kiest zelf het gepaste bestand; jij levert opties en informatie, geen bevel. Het src-attribuut blijft staan als vangnet voor situaties zonder srcset-begrip. En let op de rolverdeling: srcset en sizes regelen het *laden* (HTML, want het gaat over de resource), CSS regelt het *tonen* (de reset, `object-fit` en `aspect-ratio` uit 5.6). De width- en height-attributen van 3.9 blijven er gewoon naast staan voor de layoutreservering.



Figuur 6.5 - De beslisboom voor responsive afbeeldingen: hetzelfde beeld in meerdere maten vraagt om srcset en sizes op img; een ander beeld of een ander bestandsformaat vraagt om picture met source-elementen. Onder de vouw komt daar loading="lazy" bij.

Voor twee verdere situaties bestaat het **picture**-element, een omhulsel dat de keuze expliciet maakt in plaats van haar aan de browser te laten. De eerste is **formaat-fallback**: moderne formaten als AVIF en WebP (1.10) zijn flink compacter, maar je wil een reserve voor de zeldzame browser die ze niet kent:

```
<picture>
  <source srcset="pasta.avif" type="image/avif">
  <source srcset="pasta.webp" type="image/webp">
  <img src="pasta.jpg" alt="Bord spaghetti carbonara">
</picture>
```

De browser pakt de eerste source die hij begrijpt; het img-element onderaan is de fallback én blijft de drager van alt en styling. De tweede situatie heet **art direction**: op een smal scherm wil je soms niet dezelfde foto kleiner, maar een andere uitsnede (de close-up in plaats van het brede tafereel); een source met een media-attribuut (`media="(min-width: 48rem)"`) regelt dat. **Vuistregel:** srcset voor hetzelfde beeld in meerdere maten, picture voor een ander beeld of formaat. Dat is de hele beslisboom.

Eén attribuut maakt het verhaal af, en het is de goedkoopste prestatiewinst van het hele semester: **loading="lazy"** op afbeeldingen die niet in het eerste scherm staan. De browser stelt het downloaden uit tot de gebruiker in de buurt scrolt: één woord, en een lange pagina laadt merkbaar sneller. Niet gebruiken op de afbeeldingen bovenaan (die wil je juist meteen); wel op alles daaronder, je kaartenroosters voorop.

Wees hier ten slotte mild voor jezelf in de praktijk: meerdere bestandsversies aanmaken is werk, en voor de zes foto's van je persoonlijke site is één goed gekozen maat plus lazy loading vaak verdedigbaar. Het punt van deze sectie is dat je het mechanisme kent, want op elke professionele site is dit standaard, gegenereerde code laat het stelselmatig liggen (zie het AI-kader), en de vraag "waarom is deze site zo traag op mobiel" heeft opvallend vaak dit antwoord.

6.11 Testen op het onbekende scherm

Het instrument van deze week is tweeledig. In DevTools opent het toestel-icoon (of Ctrl/Cmd-Shift-M) de **device toolbar**: je pagina in een instelbaar kader, met voorinstellingen voor courante toestellen, een sleepbare vrije maat, en, onderschat, de mogelijkheid om traag te slepen en je ontwerp continu te zien herverdelen. Dat trage slepen is de echte test: niet drie momentopnames maar de hele film, want breuken zitten juist tussen de formaten waarop iedereen test. Het is de sleeptest van 6.4 in de praktijk: tussen twee breekpunten beweegt alles mee, op een breekpunt springt de structuur. Maak er een routine van, samen met de drie-testen-regel van F4.10 (smaller, breder, meer en minder content).

En dan het voordeel dat jij op vrijwel elke tutorial-schrijver hebt: jouw site staat live sinds week één. Pak je telefoon, open je GitHub Pages-adres, en test op een echt toestel, met echte duimen, echte browserbalk-capriolen (6.2), echt zonlicht op het scherm. **De emulator is gereedschap; het toestel is de waarheid.** Vraag ook eens het toestel van iemand anders: andere schermbreedte, andere standaardinstellingen, gratis testlab. Voor de toets geldt de checklist die je hiermee opbouwt letterlijk als generale repetitie: geen horizontale schuifbalk op geen enkele breedte, leesbare regellengtes overal, klikvlakken die een duim aankunnen, en een plattegrond die op elk formaat klopt.

Onder de motorkap: pixels die geen pixels zijn

Dit kader gaat dieper dan de theorieles. Je hebt het niet nodig om de oefeningen te maken, wel om te begrijpen wat een "pixel" eigenlijk meet en waarom queries doen wat ze doen.

CSS-pixels tegenover toestelpixels. Een moderne telefoon adverteert een schermresolutie van pakweg 1170 bij 2532, en toch meldt hij zich bij jouw media queries als zo'n 390 CSS-pixels breed. De verklaring is de *device pixel ratio* (DPR): sinds schermen fysiek steeds fijnere pixels kregen, is de CSS-pixel losgekoppeld van de hardware en gedefinieerd als een *optische* maat, grofweg wat een pixel van een klassiek 96dpi-scherm was op leesafstand. Een telefoon met DPR 3 gebruikt drie fysieke pixels per CSS-pixel in elke richting. Dit is de reden dat je nooit tegen hardwareresoluties programmeert, dat scherpe schermen tekst zo mooi tekenen (negen fysieke pixels per CSS-pixel om letterranden mee te ronden), en dat een "2x-afbeelding" bestaat: een foto die je op halve CSS-maat toont zodat elke CSS-pixel uit meerdere beeldpixels put. Het is precies het probleem dat `srcset` in 6.10 oplost: de browser kent de DPR en kiest per scherm de bestandsversie die scherp genoeg is zonder te zwaar te zijn.

De 980-pixel-aanname, formeel. Met dit vocabulaire is het verhaal van 6.2 precies te vertellen: zonder meta-tag zet de mobiele browser de *layout viewport* op zo'n 980 CSS-pixels en schaalt het resultaat naar de *visual viewport* (wat er werkelijk op glas staat). Jouw meta-regel stelt de layout viewport gelijk aan de toestelbreedte in CSS-pixels. Media queries meten de layout viewport; vandaar dat ze zonder de meta-tag "kapot" lijken: een telefoon van 390 CSS-pixels rapporteert zich als 980 en activeert vrolijk je desktopregels.

Queries en de cascade, formeel. De theorieles zei het al: een media query voegt geen gewicht toe. In het cascade-algoritme van hoofdstuk 2 gebeurt bij een query exact dit: bij het verzamelen (stap één) worden declaraties uit niet-matchende queries simpelweg niet meegenomen; wat wel matcht, doet volwaardig en zonder bonus mee aan sorteren op specificiteit en volgorde. Twee gevolgen die je nu kan beredeneren in plaats van onthouden. Eén: een basisregel met een id-selector (F2.7) verslaat een query-regel met een class; de query "doet het niet", maar de echte diagnose is een specificiteitsverlies dat toevallig in een query woont. Twee: bij het herschalen van het venster evalueert de browser de voorwaarden opnieuw en herloopt de cascade, live, zonder herladen; dat is wat je in de device toolbar continu ziet gebeuren.

Waarom rem in queries. Kleine subtiliteit tot besluit: in een media query verwijst 1rem niet naar jouw eventuele font-size op html, maar naar de basislijn van de browser (de gebruikersinstelling, standaard 16px). Queries leven immers buiten het document waarvan ze de stijl bepalen, een kip-en-ei die de specificatie zo doorhakt. Voor ons is dat louter goed nieuws: 48rem in een query betekent "48 keer wat deze gebruiker als basismaat koos", en een gebruiker met grotere tekst krijgt de eenkolomsversie dus al op een fysiek breder scherm, precies wat leesbaarheid vraagt. Het is exact waarom we breekpunten in rem schrijven.

Veelgemaakte fouten

De fouten van deze week zie je zelden op je eigen laptop. Je ziet ze op een telefoon, in de device toolbar, of in de feedback. De feedback op labo 6 verwijst naar deze nummers.

F6.1 - De vergeten viewport-meta. *Symptoom*: op mobiel oogt de site als een uitgezoomde desktoppagina en geen enkele query lijkt te werken. *Oorzaak*: de meta-regel ontbreekt; de browser doet alsof het scherm 980 pixels breed is (6.2, motorkap). *Remedie*: het skelet van 1.5 is niet onderhandelbaar; controleer bij elk "queries kapot op mobiel"-mysterie eerst de head.

F6.2 - 100vh op mobiel. *Symptoom*: de onderrand van een schermvullend element valt achter de browserbalk; een "perfect gecentreerde" held staat nét te laag. *Oorzaak*: vh rekent met de grootste viewport-variant (6.2). *Remedie*: svh (of dvh waar dynamisch gedrag gewenst is); min-height boven height.

F6.3 - De saboteur met de vaste breedte. *Symptoom*: horizontale schuifbalk op smalle schermen. *Oorzaak*: één element met een vaste maat (een width, een brede afbeelding buiten de reset, een lang onbreekbaar woord: de min-content-vloer uit hoofdstuk 4) breekt de vloeibare basis. *Remedie*: de diagnoseroutine van 6.6: smal venster, boom aflopen, grens in plaats van maat.

F6.4 - Queries boven de basisregels. *Symptoom*: de query matcht (DevTools bevestigt het) maar de stijlen winnen niet. *Oorzaak*: gelijke specificiteit plus verkeerde volgorde; de basisregel staat later en wint stap drie van de cascade. *Remedie*: queries onderaan, altijd; en bij hardnekkige gevallen de motorkap-les: check of een te specifieke basisregel (een id) de echte winnaar is.

F6.5 - Desktop-first-mengelmoes. *Symptoom*: een stylesheet met min-width- én max-width-queries door elkaar, en niemand weet nog welke versie de "basis" is. *Oorzaak*: begonnen vanaf het brede ontwerp en teruggewerkt, of code uit verschillende bronnen geplakt. *Remedie*: kies mobile-first en hou vol: basis is smal, min-width voegt toe. Een mengelmoes herschrijven is vervelend maar leerzaam (R6.2).

F6.6 - Toestel-breekpunten. *Symptoom*: queries op 768px en 1024px "omdat dat iPad is". *Oorzaak*: breekpunten gekozen op toestellen in plaats van op content (6.4). *Remedie*: het venster-sleep-ritueel; leg breekpunten waar jouw ontwerp breekt en schrijf ze in rem.

F6.7 - Vloeiende tekst zonder grenzen. *Symptoom*: microscopische titels op een telefoon, schreeuwende op breedbeeld, of tekst die niet meegroeit bij inzoomen. *Oorzaak*: kale vw-typografie zonder `clamp()` en zonder rem-term (6.7). *Remedie*: het clamp()-patroon, altijd met rem in het midden.

F6.8 - De verdubbelde stylesheet. *Symptoom*: binnen de query staan tientallen regels die exact herhalen wat erbuiten al stond, plus één wijziging. *Oorzaak*: de query behandeld als "aparte versie van de site" in plaats van als delta. *Remedie*: in een query staat alleen wat verandert; al de rest erft van de basis. Dat is het hele mobile-first-contract (6.3).

F6.9 - Eén raster dichtgetimmerd met queries. *Symptoom*: vier queries die telkens het aantal grid-kolommen herdeclareren (1, 2, 3, 4). *Oorzaak*: het zelforganiserende raster van 6.8 niet gekend. *Remedie*: `repeat(auto-fit, minmax())` vervangt die hele trap; queries bewaar je voor structurele omslagen zoals de plattegrond-kanteling.

F6.10 - Alleen op de eigen laptop getest. *Symptoom*: "bij mij werkt het", en op de telefoon van de lector niet. *Oorzaak*: geen device-toolbar-routine, geen test op een echt toestel, terwijl de site gewoon live staat. *Remedie*: 6.11: traag slepen, echte telefoon, andermans toestel. Voor de toets is dit geen tip maar de checklist.

F6.11 - Het plakkende hover-effect. *Symptoom*: op een telefoon blijft een knop of kaart na een tik in zijn hover-staat hangen. *Oorzaak*: decoratieve hover-stijlen onvoorwaardelijk geschreven; touchscreens kennen geen zweven (6.9). *Remedie*: decoratieve hover-effecten in `@media (hover: hover)`; focus-stijlen blijven onvoorwaardelijk.

F6.12 - Het loodzware plaatje. *Symptoom*: de site oogt perfect maar laadt tergend traag op mobiele data; één foto blijkt megabytes te wegen. *Oorzaak*: de reset laat alles visueel kloppen (`max-width: 100%`), dus niemand merkt dat elk scherm het grootste bestand downloadt (6.10). *Remedie*: srcset en sizes voor beeldzware pagina's, `loading="lazy"` onder de vouw, en het Network-paneel van DevTools als weegschaal.

Met AI aan je zij: gegenereerde responsiveness lezen

Responsive is de plek waar gegenereerde code er op jouw scherm goed uitziet en op een ander scherm niet. Vijf patronen om op te screenen.

Toestel-breekpunten als reflex. Vraag een model om een responsive pagina en je krijgt vrijwel gegarandeerd 768px en 1024px: versteende toestelmaten uit vijftien jaar tutorials (F6.6). Vraag expliciet om content-gebaseerde breekpunten in rem, of leg zelf de sleeptest aan en verplaats ze.

De verdubbelde stylesheet. Gegenereerde queries herhalen graag complete regelblokken met één gewijzigde declaratie: F6.8 op industriële schaal, en een teken dat het model de delta-logica van mobile-first niet "snapt" maar patronen plakt. Snoeien tot alleen de werkelijke wijzigingen overblijven is een uitstekende reviewoefening (R6.3) en halveert het bestand geregeld.

Query-trappen waar intrinsiek gereedschap hoort. Het kolommen-per-breekpunt-antipatroon (F6.9, het Niet zo van 6.8) is in gegenereerde code alomtegenwoordig, simpelweg omdat het patroon vaker in de trainingsdata zit dan auto-fit en minmax. Ook de wrap-plus-basis- en clamp()-oplossingen ontbreken vaak. De algemene controlevraag: welke van deze queries zou een intrinsieke regel kunnen zijn? Elke query die sneuvelt, is winst.

Vermengde strategieën. Modellen die bestaande code aanvullen, matchen de stijl van wat er staat, en produceren daardoor moeiteloos de desktop-first-mengelmoes van F6.5 bovenop jouw mobile-first-basis. Na elke gegenereerde aanvulling: één blik op de queryrichtingen. Allemaal min-width? Dan pas verder kijken.

Afbeeldingen zonder leverlogica. Gegenereerde pagina's tonen afbeeldingen vrijwel altijd als kaal img met één src: geen srcset, geen sizes, geen `loading="lazy"`, geen formaat-fallback. Visueel onberispelijk en F6.12 in wording. Ook de hover-query (6.9) ontbreekt standaard, dus gegenereerde hover-effecten plakken op touch. Beide zijn typische voorbeelden van wat een model niet "ziet": problemen die pas optreden op een ander apparaat dan waarop je kijkt.

Oefeningen

Labo 6 bevat de opgaven van deze week, met startbestanden, nabouwen en de inleverinstructies; het is meteen de generale repetitie voor de toets. Hieronder staat wat je na dit hoofdstuk moet kunnen, zodat je jezelf kan testen.

Drills (zonder AI aanbevolen)

- **D6.1** De viewport-proef: neem een simpele pagina, verwijder de meta-regel, bekijk op je telefoon (of de device toolbar), en beschrijf in twee zinnen wat je ziet en waarom. Zet terug. Nooit meer vergeten.
- **D6.2** Vervang in je grondplan 100vh door 100svh en pas het clamp()-patroon toe op je h1 en je paginamarges. Controleer met traag slepen dat alles vloeiend meebeweegt.
- **D6.3** Herbouw het kaartenrooster van D4.6 met `repeat(auto-fit, minmax(16rem, 1fr))` en zet het naast de wrap-versie van D5.2: beschrijf in commentaar bij welke vensterbreedtes en item-aantallen de twee verschillen, en waarom.
- **D6.4** De nav-omslag: navigatie als kolom in de basis, als rij vanaf een zelfgekozen (content-gebaseerd, in rem geschreven) breekpunt. Verantwoord je breekpunt in één commentaarregel.
- **D6.5** De kantelende plattegrond: het grondplan van 6.3 op je eigen site, met jouw kolommen en rijen. Mobiel gestapeld, breed met zijbalk, en de HTML blijft onaangeroerd.
- **D6.6** Schuifbalkjacht: een aangeleverde pagina bevat drie saboteurs met vaste maten (F6.3). Vind ze met de DevTools-routine en vervang elk door een grens of verhouding.
- **D6.7** Responsive leveren: je krijgt één foto in drie maten plus een WebP-versie. Bouw (a) een srcset/sizes-versie voor je kaartenrooster en (b) een picture-versie met formaat-fallback; zet `loading="lazy"` op alles onder de vouw, en vergelijk in het Network-paneel wat een smal venster nu werkelijk downloadt.
- **D6.8** Voorbij de breedte: zet je decoratieve hover-effecten in een `(hover: hover)`-query en schrijf een minimale printversie van je site (navigatie en decoratie weg, inktvriendelijke kleuren). Test beide: het eerste op een telefoon of in de device toolbar met touch-simulatie, het tweede via het afdrukvoorbeeld.
- **D6.9** Het skelet uit het hoofd: sluit de syllabus, bouw de HTML en de drie CSS-delen van 6.5 na vanaf een blanco bestand, in de bouwvolgorde van die sectie, en vergelijk daarna regel voor regel met de syllabus. Wat je vergat, is precies wat je voor de toets nog oefent.

Debug en review (zonder AI)

- **R6.1** "De query doet het niet": een pagina met drie query-mysteries, één volgordefout (F6.4), één specificiteitsverlies (id in de basis), één vergeten viewport-meta. Diagnosticeer alle drie mét bewijs uit DevTools en herstel.
- **R6.2** Een desktop-first-mengelmoes (aangeleverd, min- en max-width door elkaar): herschrijf naar zuiver mobile-first en documenteer per verplaatste regel waar hij heen ging en waarom.
- **R6.3** Een door AI gegenereerde responsive pagina: identificeer de vijf patronen uit het AI-kader, elimineer elke query die een intrinsieke regel kon zijn, en rapporteer de tellingen (queries voor en

na, regels voor en na).

Bouwen en verantwoorden (met AI of zonder AI)

- **B6.1** Versie zes van je persoonlijke site, en meteen je generale repetitie: volledig responsive van 320 CSS-pixels tot breedbeeld. Verplicht aanwezig: de kantelende plattegrond of een beargumenteerde eenvoudiger structuur, het zelforganiserende raster, clamp()-typografie, content-gebaseerde breekpunten in rem, hover-effecten achter een `(hover: hover)`-query, `loading="lazy"` op afbeeldingen onder de vouw, geen horizontale schuifbalk op geen enkele breedte, en een test op minstens één echt toestel (vermeld welk). Bij de AI-route: prompt, onbewerkte output, verbeterlijst, met speciale aandacht voor de vijf patronen uit het AI-kader.

Verder lezen

- MDN, *Responsive design*: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- MDN, *Using media queries*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries
- MDN, *clamp()*: https://developer.mozilla.org/en-US/docs/Web/CSS/clamp
- MDN, *Responsive images*: https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Responsive_images
- MDN, *CSS container queries*: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_containment/Container_queries